



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1: Lecture 1

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Introduction to Software Engineering

Setting the context

☐ MRI Scanning machine



☐ Expectations

- Produces detailed images of the inside of the body.
- Uses strong magnetic fields and radio waves.
- The scanner is controlled by a computer kept inside another room

Setting the context

□ An ATM



□ Expectations

- Electronic banking outlet.
- Today's ATMs are technological marvels, many capable of accepting deposits as well as several other banking services.

Setting the context

□ Uber



□ Expectations

- Allows one to book cabs/autos etc.
- Booking can be for an immediate ride or much later.
- You can do this from the comforts of your home.

Setting the context

☐ A missile launcher



☐ Expectations

- Can launch a missile.
- Target can be anywhere within the defined range
- Can be ballistic missiles (ICBM).
- Moving as well as stationary targets

Introduction

□ What is the most obvious driving force behind all these systems?

SOFTWARE (Lots of lines of code)

- A Boeing 787 has **6.5 million** lines behind its avionics and online support systems. This is much more than the sum of all mechanical parts including nuts and bolts.
- Google Chrome (browser) runs on 6.7 million lines of code (upper estimate).
- Facebook runs on 62 million lines of code (minus backend).
- Impact of a 90 minute outage at Amazon?
- \$2.8 Million revenue loss and lots of customers lost

SOFTWARE IS EVERYWHERE

Introduction

- Any software system should have all the functionalities up and running with minimum downtime.
- To build a software of high quality, we need to have the following:
 - Interaction with customers and stakeholders on what exactly is needed
 - A clear understanding of the end users (who are they and how they are going to use the system)
 - Experts in multiple domains
 - Good planning
 - Team work
 - Ability to scale and support
- Hence, need for lots of engineers to work in teams and hence teamwork is most important.
- Looking at the sizes of commonly used software packages (OS, Facebook, MS Office, etc), engineers have a lot of work to do.

Software glitch cost Hamilton victory - Mercedes

25 March 2018

MERCEDES

AUSTRALIA

HAMILTON





Ariane 5 launch

Ariane 5 is a European heavy lift space launch vehicle developed and operated by Arianespace for the European Space Agency (ESA).



A few other case studies

- In 2000, a Zenit 3SL launch failed due to faulty ground software not closing a valve in the rocket's second stage pneumatic system.
- The European Space Agency's CryoSat-1 satellite was lost in a launch failure in 2005 due to a missing shutdown command in the flight control system of its Rokot carrier rocket.
- A famous quote
“To err is human, but to really foul things up you need a computer.”
–Paul Ehrlich (German physician)

Terminology

□ Commonly used terms

- Software - A comprehensive definition

Software is not just the programs but also associated documentation and configuration data that is needed to make these programs operate correctly, serving a computational purpose.

- What is a software system?

A software system usually consists of a number of separate programs, configuration files, which are used to set up these programs, system documentation which describes the structure of the system, and, user documentation, which explains how to use the system and web sites for users to download recent product information.

Terminology...

- Software product:
 - A software product is software that has been developed and maintained for the benefit of a **user base** and often to satisfy a need in the market.

- Categories (Types) of software:
 - System software and Application Software
 - Generic software and Custom software

System software	Application Software
System software is software designed to provide a platform for other software. Examples of system software include operating systems (OS) like macOS, Linux, Android, etc., game engines, search engines, industrial automation, and software as a service applications.	Application software is software that allows users to do user-oriented tasks such as create text documents, play or develop games, create presentations, listen to music, draw pictures or browse the web.

Terminology...

General Software	Custom Software
Stand-alone systems that are developed by a development organization and sold on the open market to any customer who is able to buy them. This can meet many clients' general requirements	Custom software is a bespoke design developed <u>to meet one client's specific needs</u>, based on the budget and requirements predefined by them.

Terminology...

□ Engineering

- Engineering is the application of science and mathematics to solve problems. While scientists come up with inventions, it is engineers who apply these discoveries to the real world. Engineers innovate.

□ What is software engineering?

- Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software, and the study of these approaches; that is, the application of engineering to software.

□ One more definition

- It is also defined as a systematic approach to the analysis, design, assessment, implementation, testing, maintenance and reengineering of software, that is, the application of engineering to software.

Terminology...

- **Software Engineering principle** drives usage of appropriate tools and techniques depending on the problem to be solved, while considering the constraints and resources available.
- Focuses more on techniques for developing and maintaining software that is correct from its inception.

Introduction

□ When did the term “Software Engineering” first appear?

At the 1968 NATO Software Engineering Conference

□ Why?

It was meant to provoke thought regarding the perceived "**software crisis**" at the time.

□ Now what was the “Software Crisis”?

The term was used to describe the impact of rapid increases in computer power and the complexity of the problems that could be tackled.

□ What does this mean?

In essence, it refers to the difficulty of writing correct, understandable, and verifiable computer programs.

The roots of the software crisis are **complexity**, **expectations**, and **change**.

Introduction...

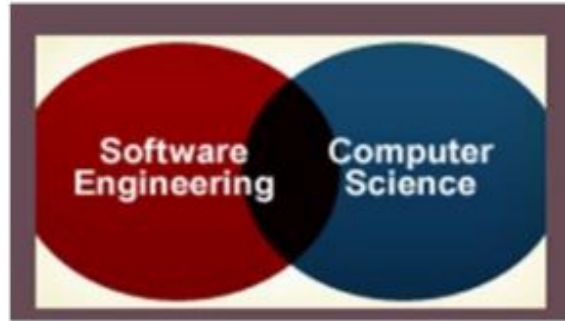
- The causes of the software crisis were linked to the overall complexity of hardware and the software development process.
- The crisis manifested itself in several ways:
 - Projects running over-budget and over-time.
 - Software was very inefficient and often of low quality.
 - Software often did not meet requirements.
 - Projects were unmanageable and code difficult to maintain.
 - Software was never delivered.

An interesting note on state of software projects

- Percentage of projects that fail
 - The statistics for software project delivery show that **only one in three software projects are truly successful.**
 - According to Standish Group's report, 66% of technology projects (based on the analysis of 50,000 projects globally) end in partial or total failure. **Just one in three software projects are considered truly successful with large projects most at risk of failure**
- When can a Software Project be defined as a Failure?
 - The software did not satisfy the requirements of the customer.
 - The software release was later than scheduled (deadline violation).
 - The software had too many bugs.

Computer Science v/s Software Engineering

□ Is Computer Science same as Software Engineering? NO



- Computer Science is the study of computation, automation and information. It spans the theoretical disciplines (such as algorithms, theory of computation, information theory and automation) to practical disciplines (including the design and implementation of hardware and software).
- Software engineering typically deals with the engineering principles of building, designing and testing software products.

Fundamental drivers of Software Engineering

What are the key factors which drive Software Engineering?

- Industrial strength software should
 - be operational,
 - be maintainable,
 - be capable of being moved,
 - have elaborate documentation
 - have no or minimal number of bugs
 - be impactful to the business

- Software is expensive
 - (Good) Software engineers are at a premium and hence expensive
 - Maintenance and rework cost money.

- Life-critical and mission-critical software
 - Can influence the life or death of a person or a mission.
 - Can make or break organizations.

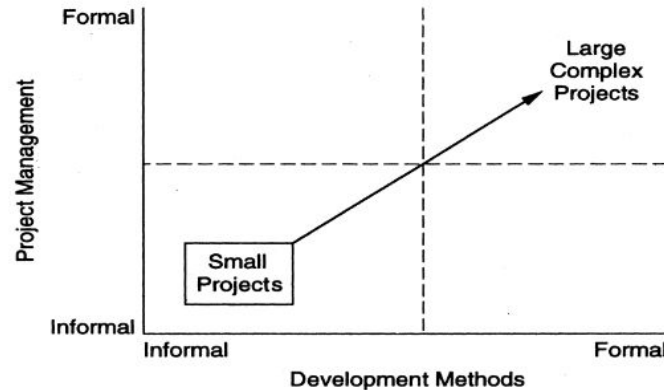
Fundamental drivers of Software Engineering ...

- Heterogeneity
 - Systems should work as distributed systems.
- Security and trust
 - Software should be trustworthy.
 - It should be secure.
- Diversity
 - Different types of software systems.
- Scale
 - Software should be scalable.
- Business and social changes
 - Ability to change existing software and to develop new software
 - The organizations are becoming global.

Fundamental drivers of Software Engineering ...

- Quality and productivity
 - Quality measured through **FLURPS** (Functionality, Localizability, Usability, Reliability, Performance and Security) , Portability, Efficiency/Maintainability.

- Consistency and repeatability



- Late and unreliable.
 - Typically 35% of the computer based projects are runaway

Summary – Why Software Engineering is required

- Developing large programs
- Mastering complexity of big programs
- Efficient development of evolving software
- Ensuring software process supports users effectively and right choices and decisions are made
- Supporting large teams and team work
- Ensuring visibility and continuity

Next Class

□ Take Home Activity - Crowdstrike Case study



THANK YOU

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1: Lecture 2

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

TOPIC: CrowdStrike Case Study

Case Study: CrowdStrike



LaGuardia Airport, New York City

Photo by Smishra1 - Own work, CC BY-SA 4.0,
[https://commons.wikimedia](https://commons.wikimedia.org/wiki/File:Blue_screen_of_death_at_LaGuardia_Airport.jpg)

The New York Times

[See more from our live coverage](#)

Chaos and Confusion: Tech Outage Causes Disruptions Worldwide

Airlines, hospitals and people's computers were affected after CrowdStrike, a cybersecurity company, sent out a flawed software update.



Share full article



951



Travelers waiting to check in at the airport in Hamburg, Germany, on Friday. Bodo Marks/DPA, via Associated Press

Class Discussion

- ☐ What could have gone wrong?
- ☐ How to avoid such events? What could the company have done?
- ☐ Is this Microsoft issue? Explain
- ☐ How could they have minimized the impact?
- ☐ Why did it take so long to get it fixed?

Summary

❑ Defective Update Released

- On July 18, 2024, CrowdStrike pushed a routine Falcon software update to production without adequate testing, embedding a corrupted library that would trigger system crashes

❑ Global Outage Begins

- The next morning, over 8.5 million Windows endpoints worldwide abruptly failed when the update auto-applied

❑ Critical Services Disrupted

- Sectors from banking to healthcare saw systems go offline; airlines halted flight operations, and 911 call centers experienced intermittent failures

❑ Delta Air Lines Impact

- Delta canceled more than 7,000 flights over five days, affecting 1.3 million passengers and estimating \$550 million in losses

❑ Extended Recovery Period

- While some organizations restored service within hours, Delta's outdated infrastructure prolonged its recovery until July 25

❑ Insurer Exposure

- Insurers face an estimated \$1.5 billion in cyber-interruption claims tied to this single incident

❑ Regulatory and Legal Fallout

- Georgia Superior Court allowed Delta's suit alleging gross negligence to proceed; U.S. DOT and EU data-protection regulators opened investigations

What Went Wrong

❑ Inadequate Staging & Testing

- Minimal pre-production validation meant the buggy library went undetected; a simple sandbox test could have flagged the crash

❑ Monolithic Update Deployment

- All customers received the same update simultaneously, eliminating opportunities for canary or phased rollouts

❑ Poor Change Management

- Absence of a formal Change Control Board (CCB) and impact-analysis workflow violated best practices in SCM and risk management

❑ Over-Reliance on Upstream Vendor

- Relying on a single code library without verifying its integrity downstream introduced a single point of failure

❑ Insufficient Monitoring & Alerting

- Telemetry failed to trigger rapid rollback when error rates spiked; incident response teams were only alerted after mass failures

Possible Explanation

- ❑ **SCM & Version Control:** No branching strategy or semantic versioning enforced → untested code path introduced into master
- ❑ **Continuous Integration:** Lack of automated unit/integration tests against real-world configurations → regressions slipped through
- ❑ **Release Management:** No canary or blue-green deployment model → all users hit simultaneously by the bug
- ❑ **Requirements & Quality Assurance:** Missing clear “release-readiness” criteria (code coverage, fault injection) → low confidence in update stability
- ❑ **Risk Management:** No formal risk register or mitigation plans for third-party library failures

How to Avoid

- ❑ **Implement Phased Rollouts**
 - Use canary releases and feature flags to expose changes to a small subset before full-scale deployment.
- ❑ **Automate Testing & Rollbacks**
 - Integrate regression, fault-injection, and canary-pipeline tests; script automatic rollback on threshold breaches.
 - Automated security testing integrated in the CI/CD pipeline throughout the lifecycle.
- ❑ **Enforce SCM Best Practices**
 - Adopt semantic versioning, maintain separate release branches, and conduct CCB-driven impact analyses.
- ❑ **Establish “Release-Readiness” Gates**
 - Define clear QA criteria (test coverage, performance benchmarks, security scans) that must pass before deployment.
- ❑ **Strengthen Monitoring & Incident Response**
 - Real-time telemetry with automated alerts and a documented runbook for immediate mitigation.
- ❑ **Diversify Dependencies**
 - Mirror critical libraries and verify checksum/signatures to avoid single-source risks
- ❑ **Configuration management for code and parameter configs.**

Change Management Process

- Configuration updates treated differently than code changes
 - Critical gap in the kind of testing change was subjected to
- To improve the change management process:
 - Classify changes based on the risk and impact, not type
 - Comprehensive risk assessment for all changes
 - Automated rollback plans for all types of changes
 - Phased deployment approach for high risk changes
 - Enhance monitoring and alerting system to quickly detect issues post deployment.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1 - Lecture 3

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

2

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Software Development Life Cycles & Security Development Life Cycle

How to develop software???



Discuss the
software that
needs to be
written



Write some
code



Test the code
to identify the
defects



Debug to find
causes of
defects



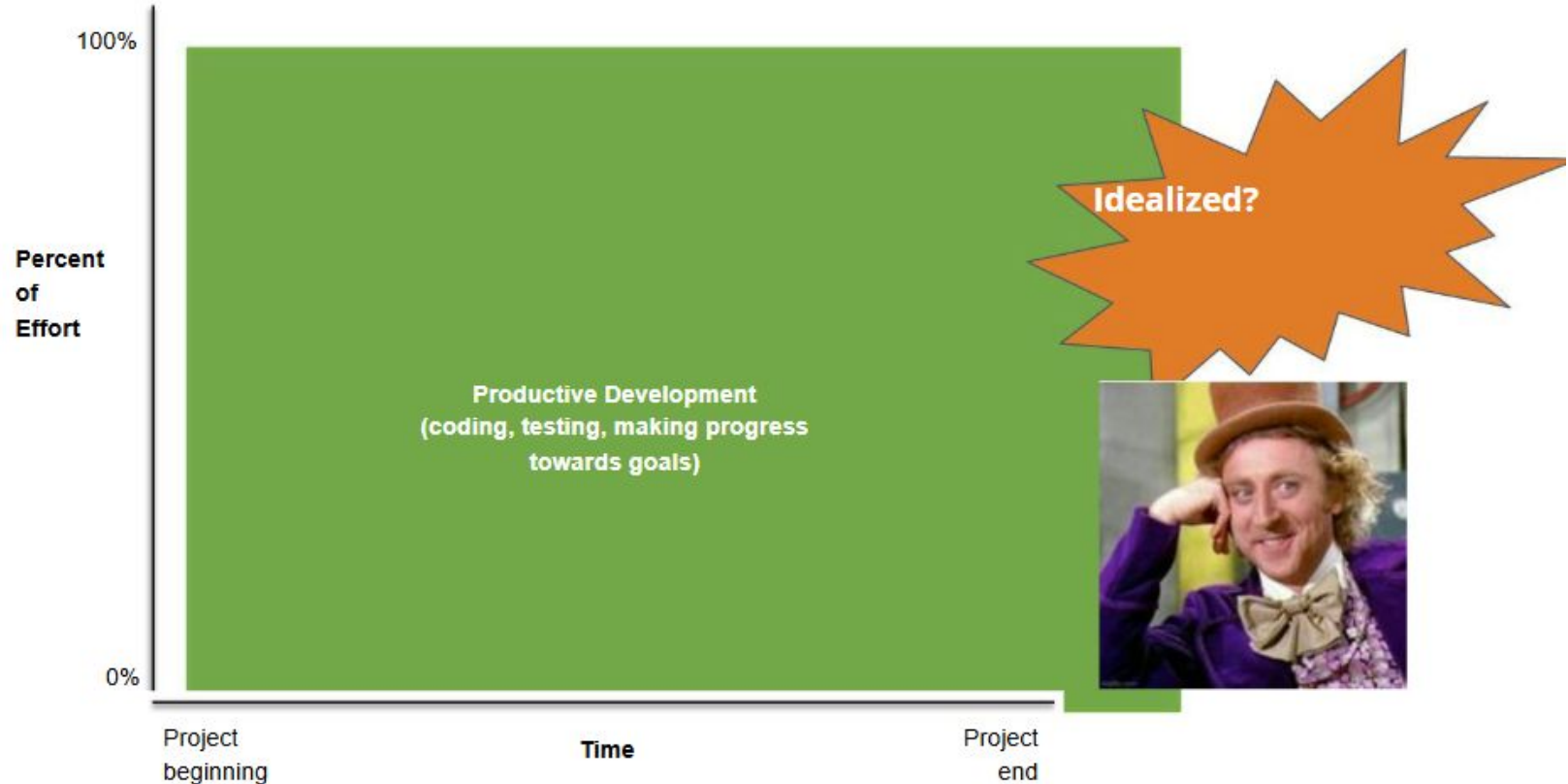
Fix the
defects

What does a software engineer's day look like?

- How many hours do they spend in meetings, coding, testing, debugging, etc.?



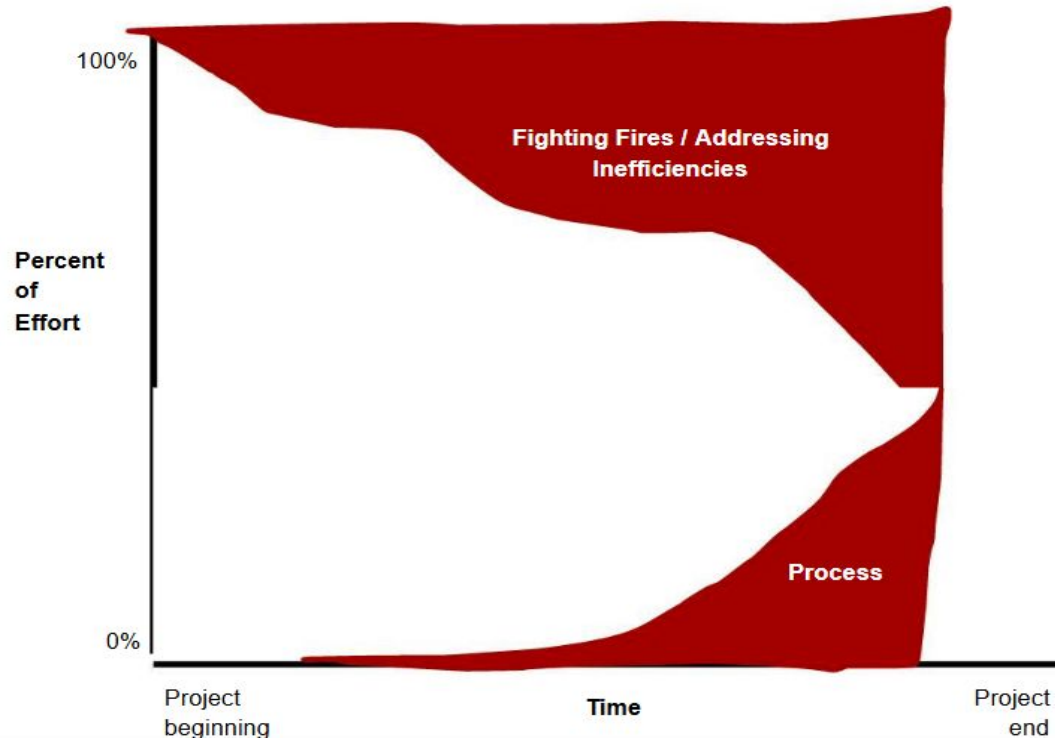
What does a software engineer's day looks like?



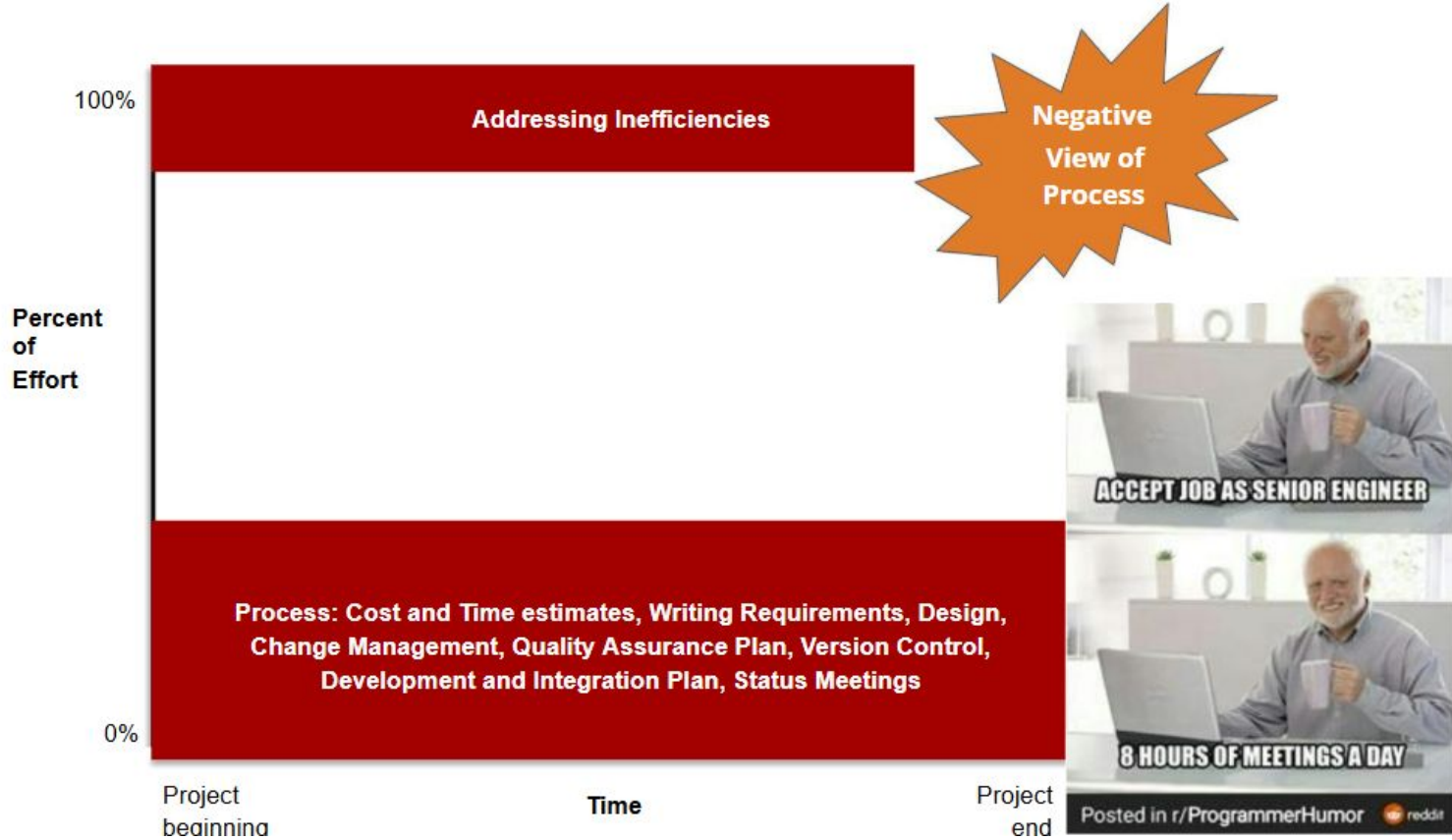
What does a software engineer's day looks like?



What does a software engineer's day look like?



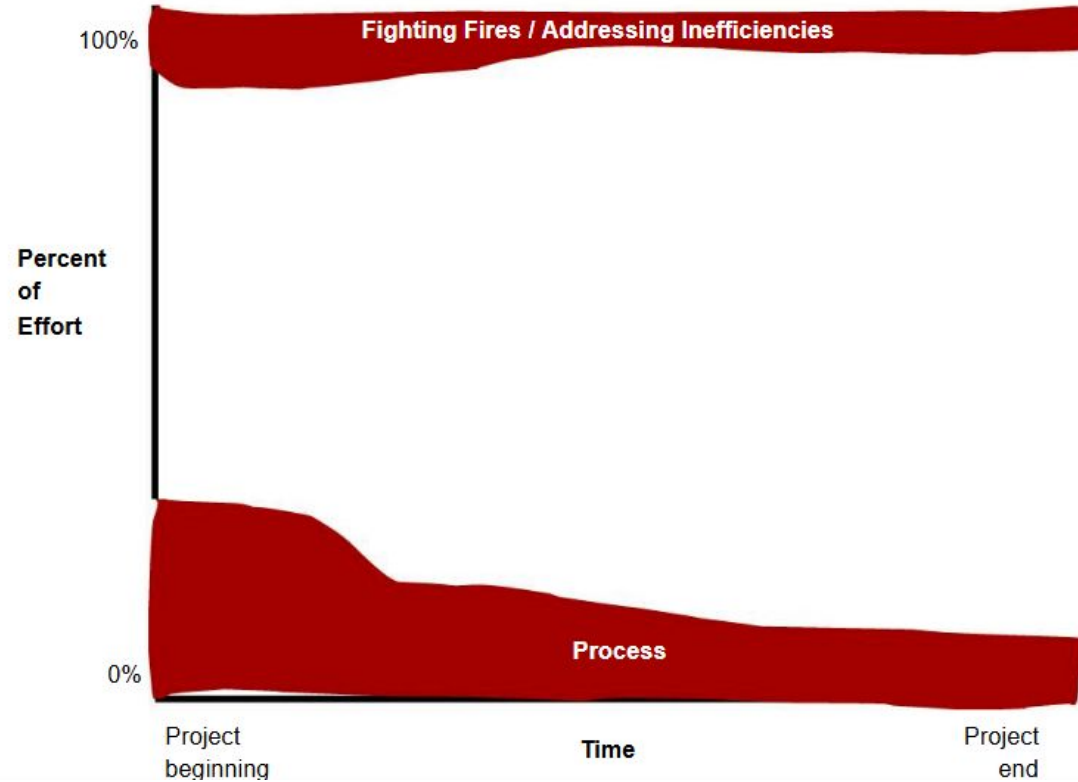
What does a software engineer's day looks like?



Let's improve the reliability of this process

- Writing down all requirements
 - Review requirements
 - Require approval for all changes to requirements.
- Use version control for all changes
 - Code Reviews
- Track all work items
 - Break down development into smaller tasks.
 - Write down and monitor all reported bugs.
 - Hold regular, frequent meetings.
- Plan And conduct quality assurance
- Employ a DevOps framework to push code between developers and operations.

Let's improve the reliability of this process



Hypothesis: Process increases flexibility and efficiency

Ideal Curve: Upfront investment for later greater returns

Is a more structured SDLC necessary?

- It's used to establish an order – provide a model - in which software project events occur from project conception to project delivery
- It forces us to think of the “big picture” and follow steps so that we reach it without glaring deficiencies
- Without it we may make decisions that are individually on target but collectively misdirected
- It allows us to organize and coordinate our work as a team
- It allows us to track progress and risks, and adjust as necessary

Software Project Lifecycle

□ Software process

A software process is a structured set of activities and associated outcomes (intermediate and final), that produces a software product.

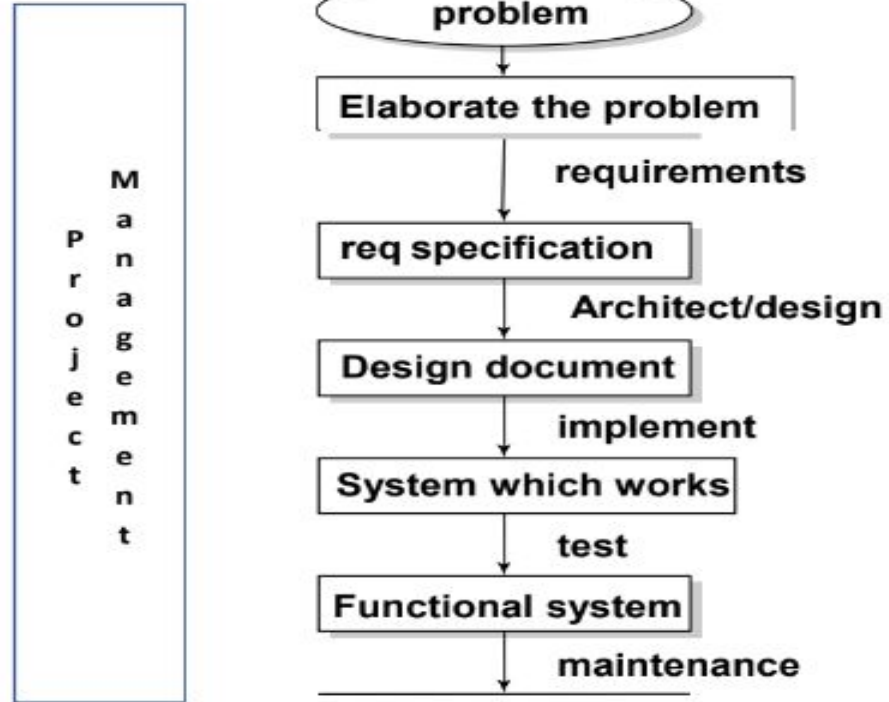
□ Each activity is defined in terms of:

- **Entry criteria**: What conditions must be satisfied for initiating this phase
- **Task and its deliverable**: What should to be done in this phase
- **Exit criteria**: When can this phase be considered done successfully.
- **Who**: Who is responsible
- **Dependencies**: What are the dependencies for this phase ..etc.
- **Constraints**: Time schedule

Software Project Lifecycle ...

Note:

Each step can be a process itself.



Levels of Abstraction

- Requirements:
 - high-level “what” needs to be done
- Architecture (High-level design):
 - high-level “how”, mid-level “what”
- Design (Low-level design, e.g. design patterns):
 - mid-level “how”, low-level “what”
- Code:
 - low-level “how”

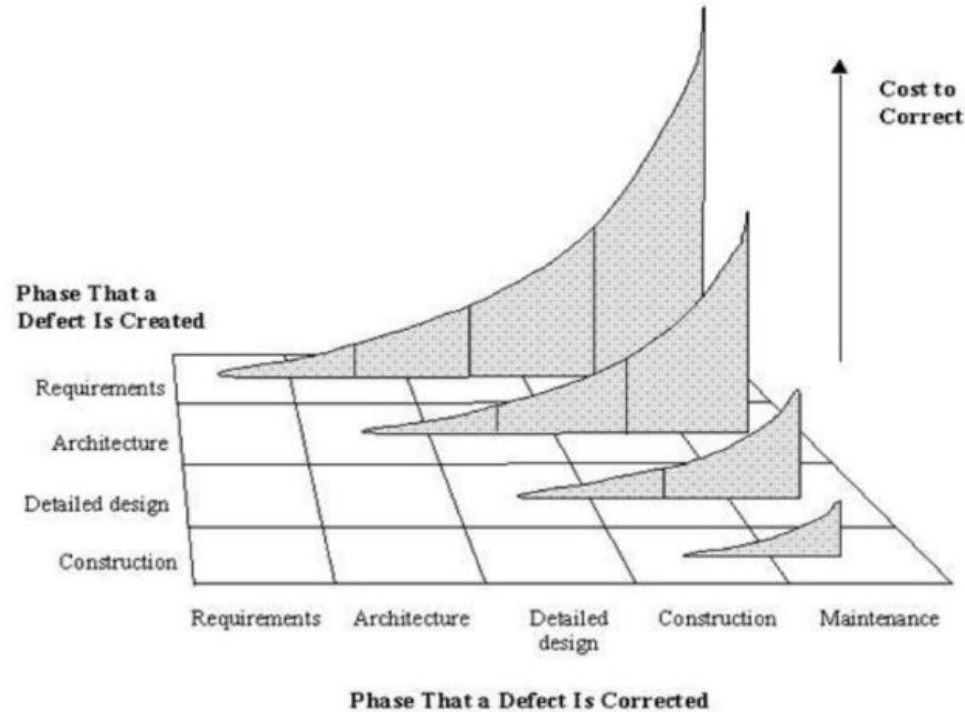
Software Project Lifecycle ...

There are a few other important activities:

- Configuration Management
- Change management

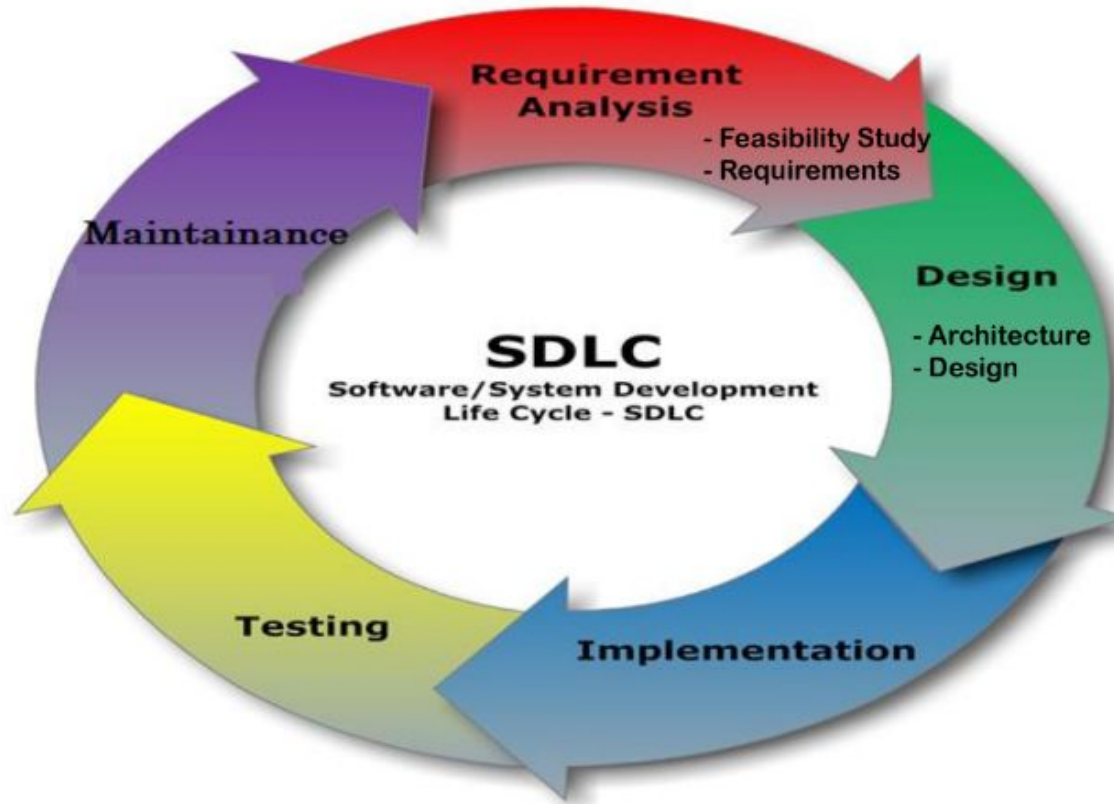
These form part of the Software Development process.

Let's improve the reliability of this process



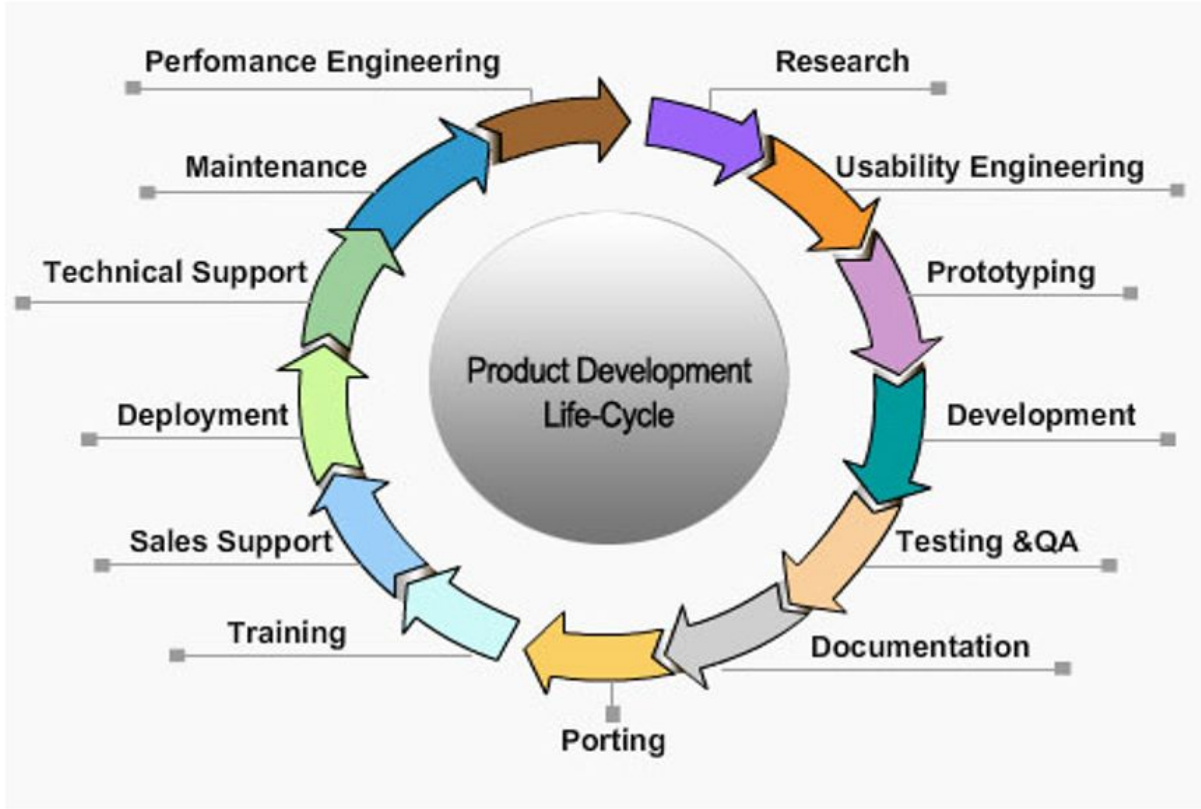
Copyright 1998 Steven C. McConnell. Reprinted with permission from *Software Project Survival Guide* (Microsoft Press, 1998).

Software Development Lifecycle



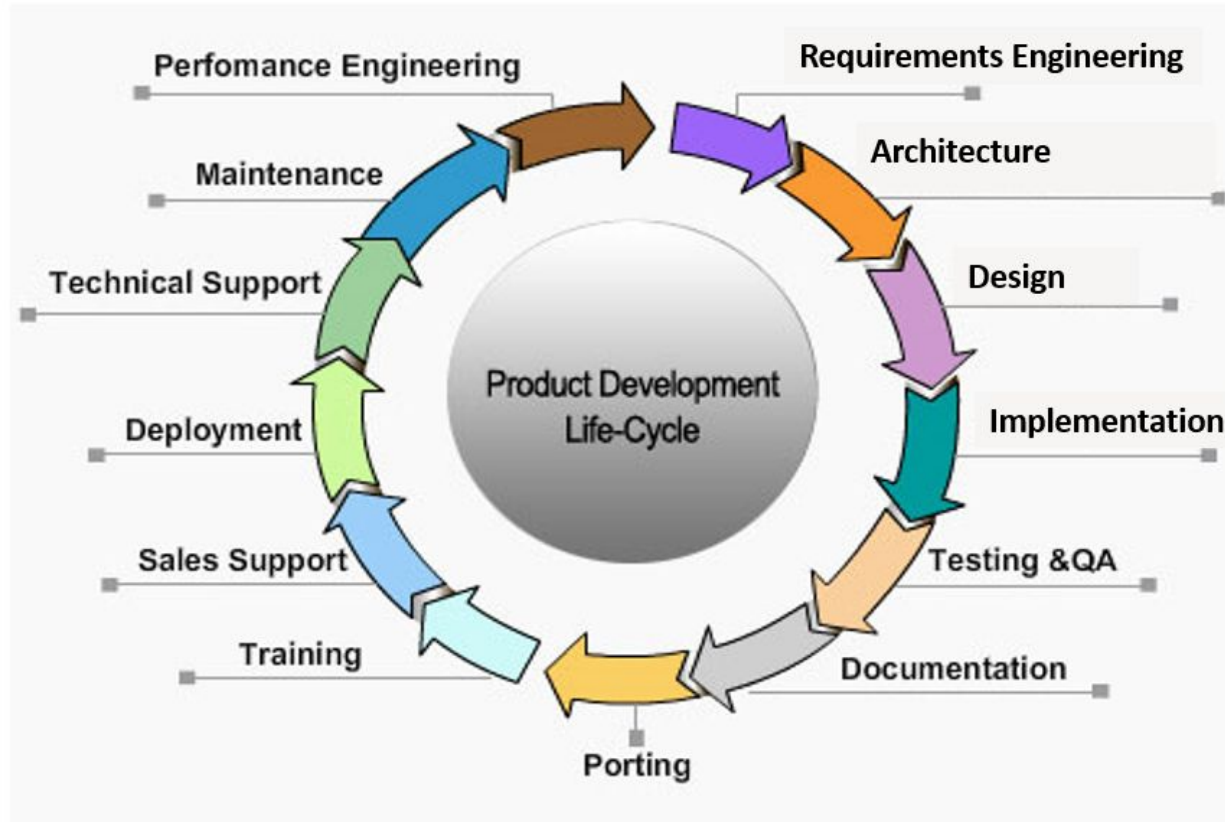
Product Development Lifecycle

Variation 1



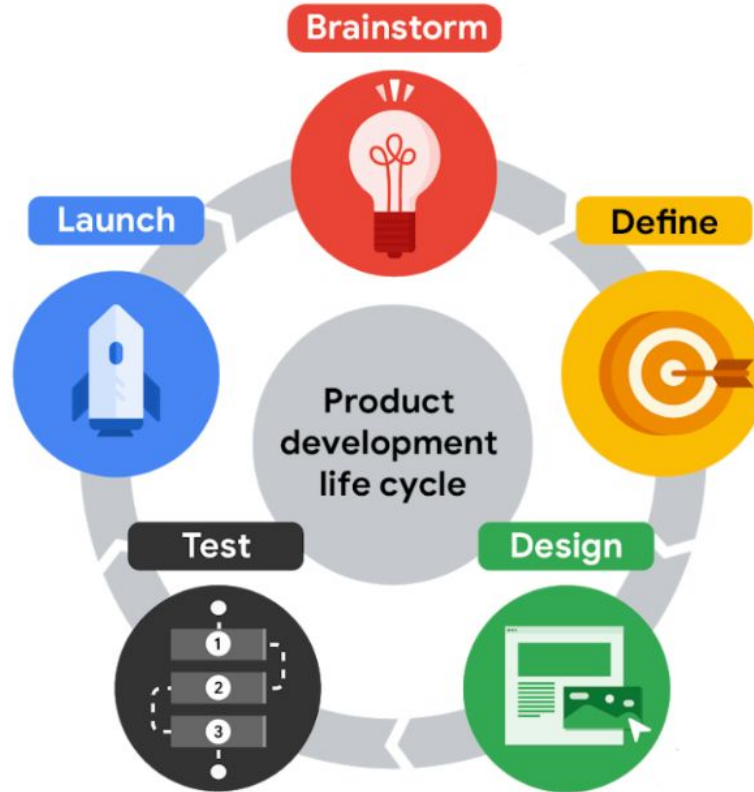
Product Development Lifecycle

Variation 2



Product Development Lifecycle

Variation 3



Brainstorm

An active discovery stage that's all about generating ideas about the user and potential needs or challenges that the user might have, it is important to pay attention to the diversity of your team at this stage.

The brainstorming stage is also an ideal time to check out your product's competitors and identify if there are already similar products available in the market, you want your product to fill a gap in the market or solve a problem better than existing products.

note: A UX designer at a large company might not be very involved in the brainstorming stage but a UX designer at a startup or small business could have a big role to play.

Define

This stage brings UX designers, UX researchers, program managers, and product lead to define the product.

The goal is to figure out the specifications for the product by answering questions like who is the product for? what will the product do? and what features need to be included for the product to be successful?

Design

Here the implementation of insights into new designs using various tools is done. At this stage UX designers develop the ideas for the product, generally, UX designers start by drawing wireframes, which are outlines or sketches of the product, then move on to creating prototypes, which are early models of a product that convey its functionality.

UX writers are also involved in the design stage and might do things like write button labels. At this point in the lifecycle, UX designers make sure to include all of the product specifications that were outlined in the define stage and ensure each part of the design fits together.

Test

Evaluating the product designs based on the feedback of the potential users.

UX designers work with engineers to develop functional prototypes that match the original designs, including details and features that fit the company's brand, like font and color choices. This also means writing the code and finalizing the overall structure of the product.

At this stage, the designs go through at least 3 phases of testing:

internal tests: within the company reviews with stakeholders, and external tests with potential users.

Alpha testing - The team tests the product internally to look for technical glitches and usability problems.

Beta testing - an external test with potential users. This is the time to figure out whether the product provides a good user experience, meaning its usable, equitable, enjoyable, and useful

Launch

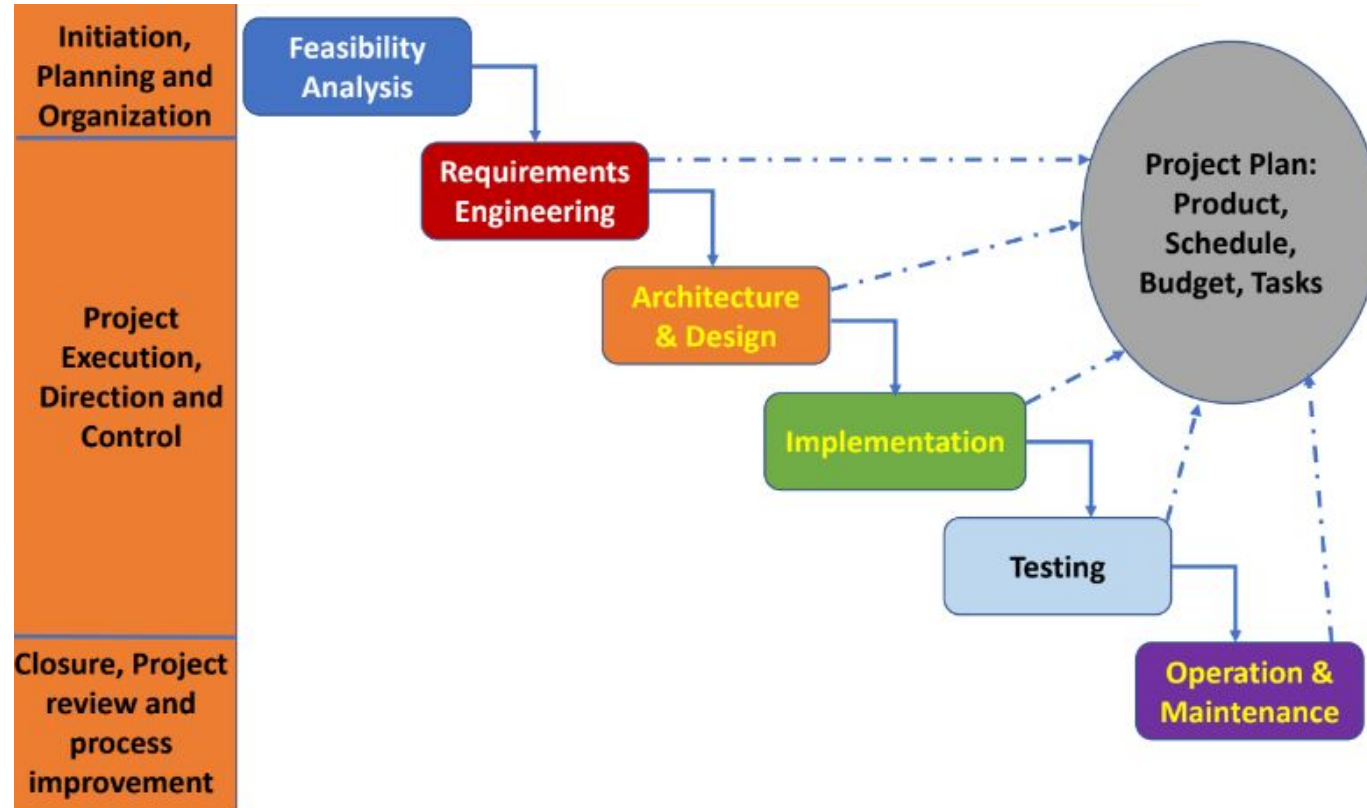
Sharing a finished version of the product with the public.

Marketing professionals on your team might post about the new product on social media or publish a press release. The customer support team might get ready to help new users learn how the product works.

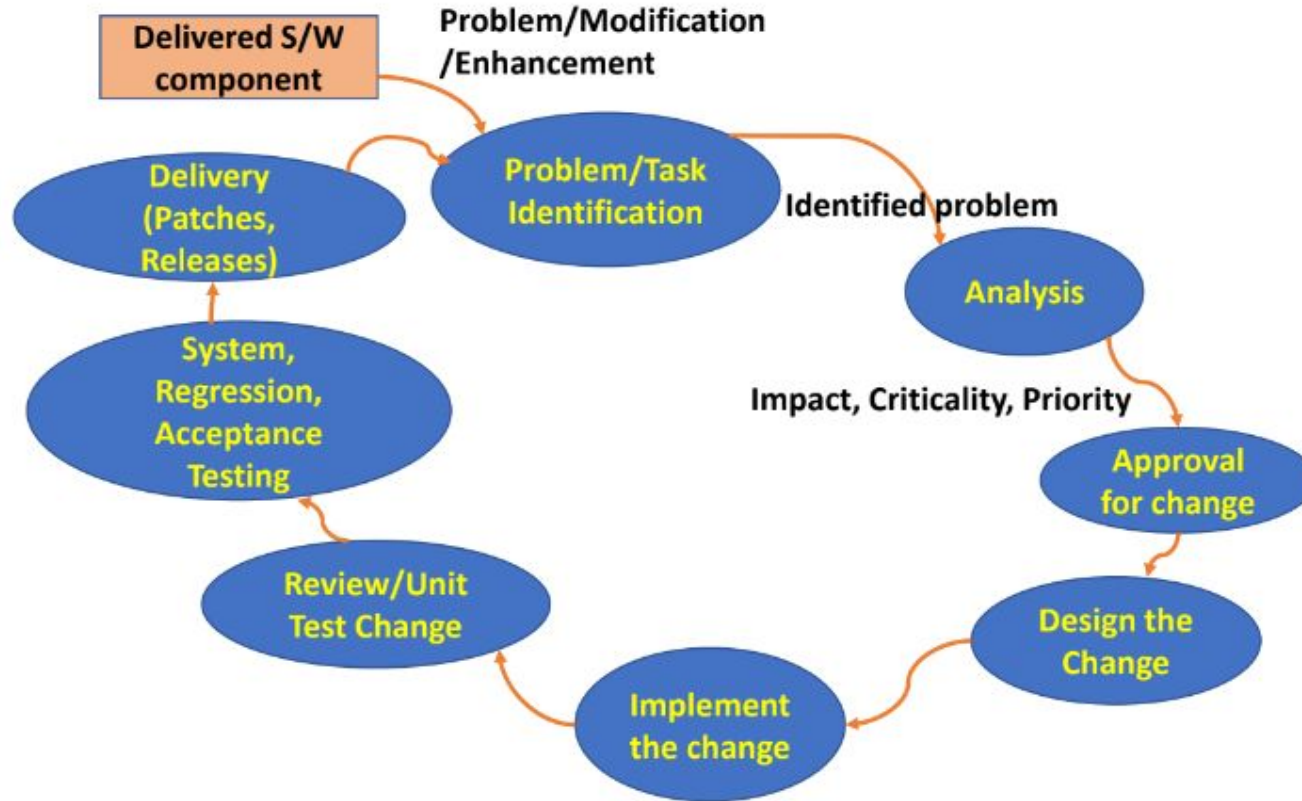
Program managers also meet with cross-functional teams to reflect on the entire product development life-cycle and ask questions like what worked and what could be improved? were goals achieved? were timelines met? making time for this reflection is super important since it can help improve the process going forward.

For a physical product, the launch stage might be the end of the product development life cycle. But for a digital product, like an app or website, launching the product to a wider audience provides another opportunity to improve on the UX. So, after the launch stage, teams will often cycle back to the design stage and testing stages to start working on the next version of the digital products.

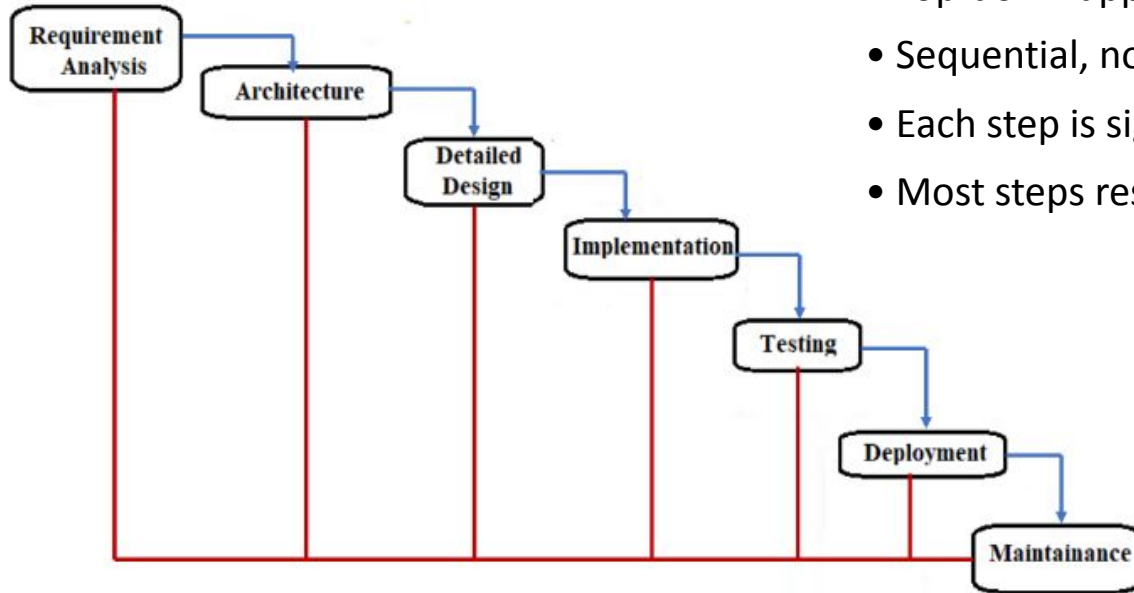
Project Management Lifecycle (PMLC)



Software Maintenance Lifecycle (SMLC)



SDLC: Waterfall model



- Top-down approach
- Sequential, non-overlapping activities and steps
- Each step is signed off on and then frozen
- Most steps result in a final document

SDLC: Waterfall model

ADVANTAGES

Simple

Clear identified phases

Easy to manage due to rigidity

Each phase – specific deliverables + reviews

Easy to departmentalize and control

DISADVANTAGES

Assumes requirements are frozen

Difficult to change & sequential

Poor model for long projects

Big Bang approach

High risk + Uncertainty

When can you use this model?

- ❑ Pure form: Short projects where requirements are well known
- ❑ Product definition is stable & technology is understood
- ❑ Regulatory requirements
- ❑ When product requirements are very well understood and won't change

Honeywell's Flight Management System Selected By Airbus

Honeywell's solution will address the avionics needs of the Airbus A320, A330 and A350 aircraft fleet

Ahjay Rai
May 19, 2022



The logo for the U.S. Food & Drug Administration (FDA), featuring the letters 'FDA' in a blue box.

**U.S. FOOD & DRUG
ADMINISTRATION**

Q. Search

[Home](#) / [Medical Devices](#) / [Device Advice: Comprehensive Regulatory Assistance](#) / [Overview of Device Regulation](#)[Share](#) [Tweet](#) [LinkedIn](#) [Email](#) [Print](#)

Overview of Device Regulation

[A History of Medical Device Regulation & Oversight in the United States](#)

Introduction

FDA's Center for Devices and Radiological Health (CDRH) is responsible for regulating firms who manufacture, repackage, relabel, and/or import medical devices sold in the United States. In addition, CDRH regulates radiation-emitting electronic products (medical and non-medical) such as lasers, x-ray systems, ultrasound equipment, microwave ovens and color televisions.

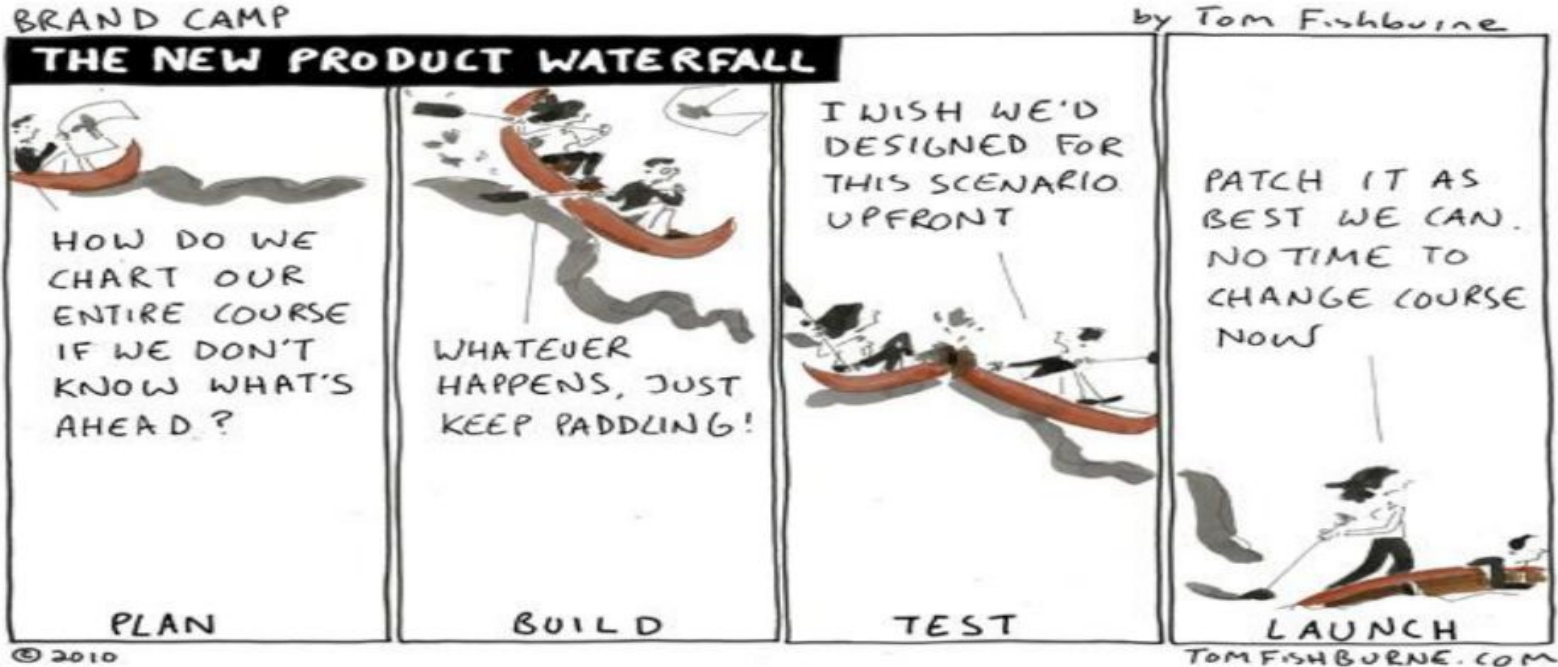
Cont
of:
09/0

Regu
Prod
Medi
Radi
Prodi

Likely parts of their SDLC is waterfall-like due to the upfront and regulated requirements

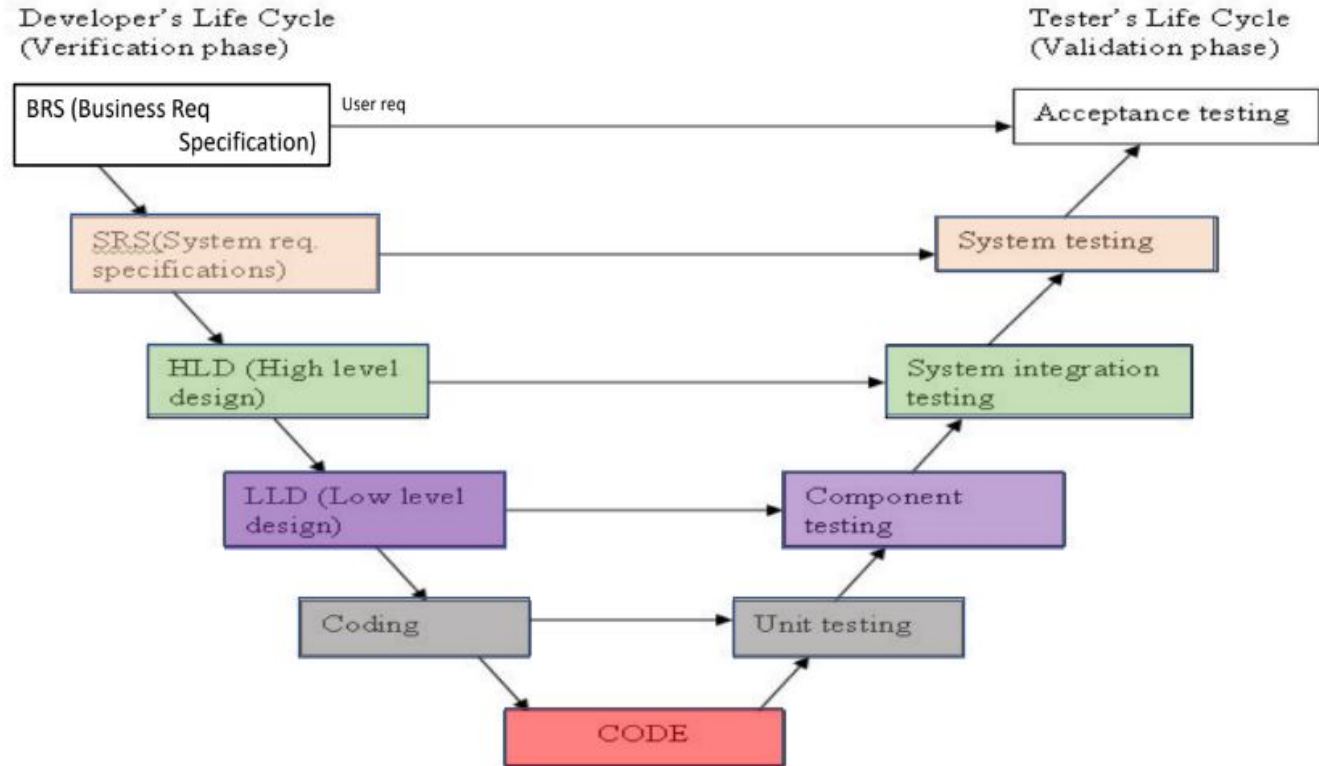


Waterfall model



Legacy Software Development Life Cycles (SDLC)

The V model



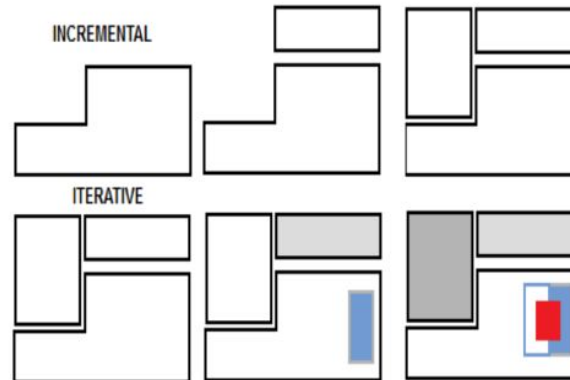
The V model

ADVANTAGES	DISADVANTAGES
Similar to Waterfall model	Similar to Waterfall model
Test development activities can happen before formal testing cycle	No early prototypes of software
Higher probability of success + Increased effectiveness of usage of resources	Change in process => change in test documentation

Usage: Similar to waterfall model

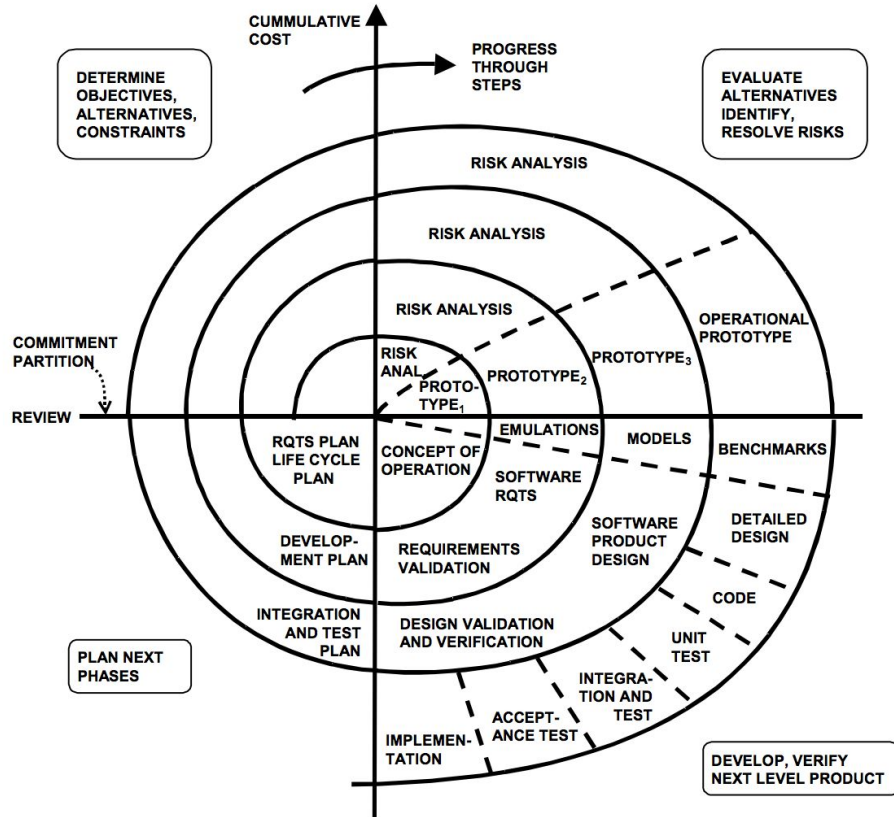
Iterative model vs Incremental model

ITERATIVE MODEL	INCREMENTAL MODEL
Revisit and refine everything	No need to go back and change delivered things
Focus on details of things	Focus on things not implemented yet
Leverage on learnings	Does not leverage on experience or knowledge



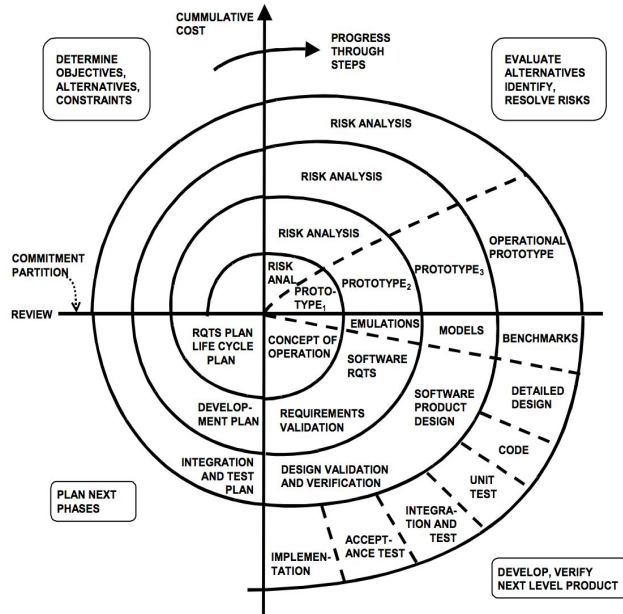
- The spiral model was originally proposed by Boehm.
- Instead of representing the software process as a sequence of activities with some backtracking from one activity to another, the process is represented as a spiral.
- Each loop in the spiral represents a phase of the software process. The innermost loop might be concerned with system feasibility, the next loop with requirements definition, the next loop with system design and so on.

Spiral model



- Incremental/iterative model (combines waterfall and prototyping)
- Iterations called spirals
- Repeat these activities:
 1. Determine objectives (reqs)
 2. Risk analysis
 3. Develop and test
 4. Plan next delivery
- Phased reduction of risks (address high risks early)

Spiral model



Pros:

- Early indication of unforeseen problems
- Allows for changes
 - The risk reduces as costs increase

Cons:

- More complex to run
- Requires proper risk assessment
- Requires more planning and experienced management

Limitations of most legacy lifecycle models

- ❑ Predictive software development methods
- ❑ Upfront planning
- ❑ Do not facilitate periodic customer interaction
- ❑ Suited for large complex projects
- ❑ Regulatory perspectives
- ❑ Suited for global and distributed organizations
- ❑ Product lifecycle and its ecosystem
- ❑ People and skill perspective
- ❑ Suitable for projects with clear definition
- ❑ Suitable when things are not changing too fast



THANK YOU

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1 - Lecture 4

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

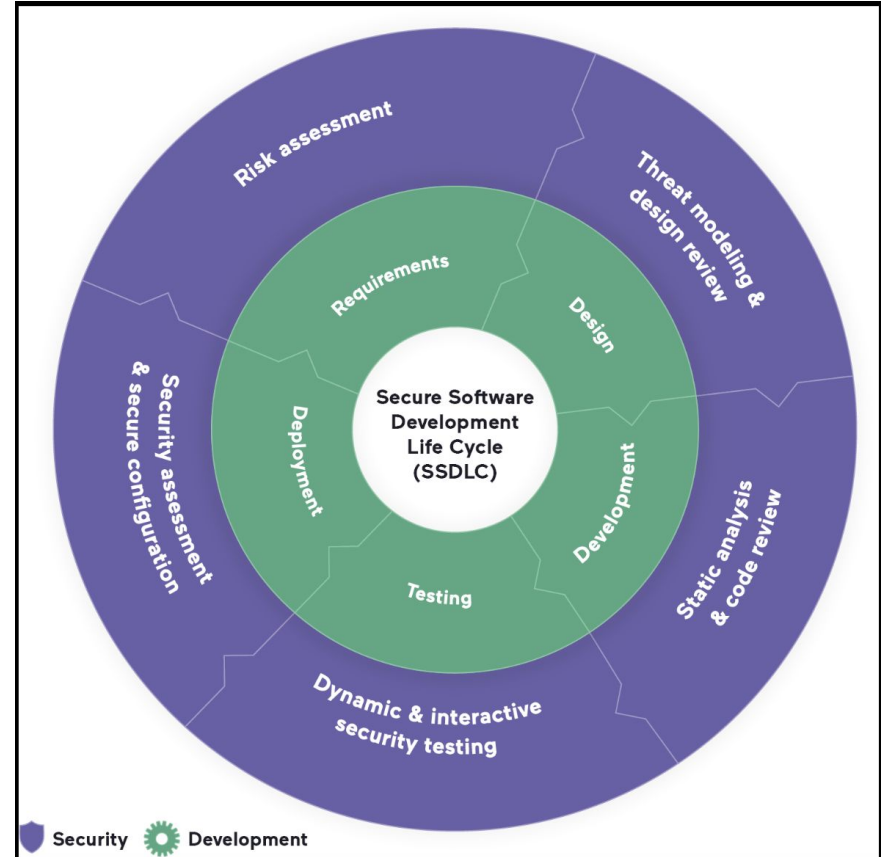
2

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

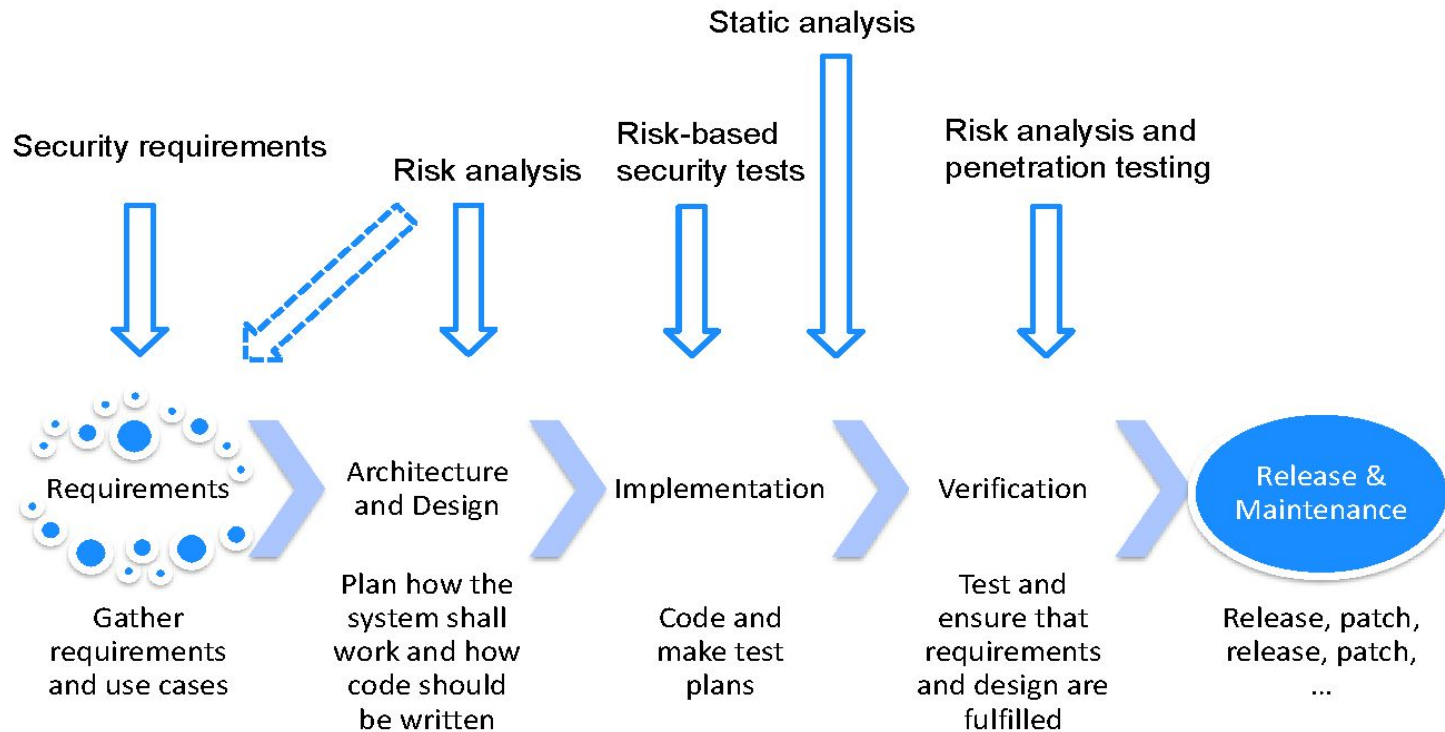
Security Development Life Cycle (SecDLC)

Mapping with SDLC

The Security Development Life Cycle (SecDLC) is a structured set of processes that incorporates security considerations into every phase of the SW Development Life Cycle (SDLC).



Mapping with SDLC

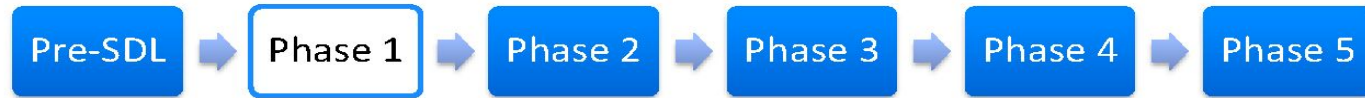


Pre-SDL: Security training



- All members must receive appropriate **training** to **stay informed** about security basics and recent trends in security and privacy.
- Topics include:
 - Threat Modelling (e.g. design implications)
 - Secure Coding (e.g. buffer overruns, cross-site scripting)
 - Privacy (e.g. types of privacy-sensitive data)
- This is only the baseline training, specialization and advanced training may be necessary.

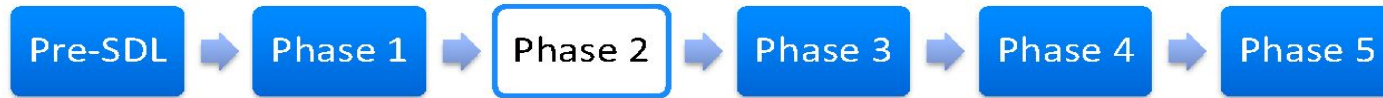
Phase 1: Requirements



Identify functional aspects of the software that require deep review:

- Which portions of the project will require security design reviews before release?
- Which portions of the project will require penetration testing by a mutually agreed upon group that is external to the project team?
- Perform Threat Modeling and Security Risk Assessment (SRA) to define product security requirements.

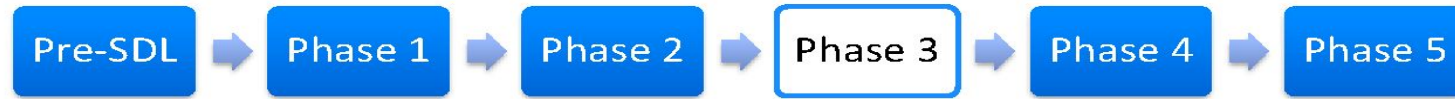
Phase 2: Design



All design specifications should describe how to securely implement all functionality provided by a given feature or function:

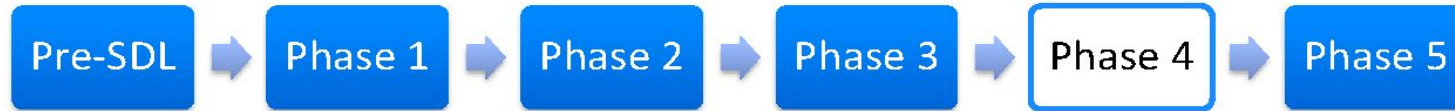
- Attack surface reduction (giving attackers less opportunity to exploit a potential weak spot).
- Defense in depth (design system with multiple layered defenses)
- Risk analysis of components or features that have meaningful security risks (can be defined by the security risk assessment during requirements).

Phase 3: Implementation



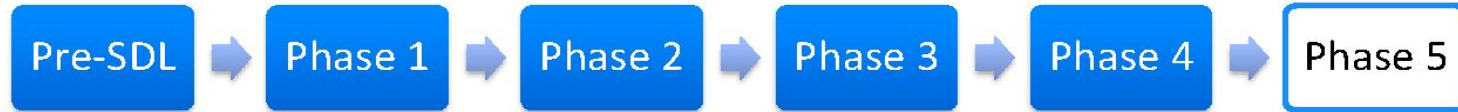
- ❑ Publish a list of **approved tools** and their associated security checks, such as compilers/linker options and warnings.
- ❑ Teams should analyze all functions and APIs that will be used in conjunction with a software development project and **prohibit** those that are determined to be unsafe (including open source).
- ❑ Once a prohibited list is defined, all code should be **scanned** for these functions and APIs and modified accordingly.
- ❑ **Static analysis** of code should be performed.

Phase 4: Verification



- Dynamic program analysis, monitor application problems with memory corruption, user privilege issues, etc.
- **Fuzz testing**, deliberately introduce malformed or random data to an application during dynamic analysis.
- Update threat model and attack surface analysis, account for any design or implementation changes to the system, and assure that any new threats/attack are reviewed and mitigated.

Phase 5: Release



An incident response plan must be in place:

- ❑ A first point of contact in an emergency.
- ❑ On-call contacts with decision-making authority that are available 24-hours a day.
- ❑ Security servicing plans for code inherited from other groups in the organization.
- ❑ Security servicing plans for third-party code (and if appropriate the right to make changes).
- ❑ **Final Security Review (FSR):** Includes an examination of threat models, tool output, performance against quality gates and bug bars.

Additional Material for Home Reading

The 4 Ps in software engineering

- The 4Ps namely, **People, Process, Product** and **Project Environment** are a framework used to describe and organize the key elements of a project.
- They cover everything from the team working on the project, the methodologies used, the tangible outcomes and even the external influences that can impact the project's success.
- The framework is essential because it provides a holistic perspective, helping project managers to better plan, execute and control projects.
- By considering these key elements, project managers can deliver successful outcomes and add real value to stakeholders.

The 4 Ps in software engineering - People

- People are the backbone of any project.
 - Without the right team in place, even the best-laid plans can crumble.

- Who are these crucial individuals that breathe life into a project?
 - Firstly, the project team; this group is a diverse mix of individuals, each bringing their unique skills and expertise to the table.
 - They are the builders, the creators, the problem-solvers, the ones who make things happen. It's their collective efforts that drive the project from inception to completion.

- Then, we have the stakeholders.
 - These are the individuals or groups who have a vested interest in the project. They could be sponsors, providing the necessary funding or clients eagerly awaiting the project's deliverables. They could also be end-users who will ultimately benefit from the project, or vendors supplying essential resources.

The 4 Ps in software engineering - People

- The stakeholders' needs and expectations significantly influence the project's direction and goals.
- And steering this ship is the project manager.
 - He/She is the linchpin, the one responsible for keeping all the moving parts in sync.
 - He/She plans, coordinates, communicates and leads.
 - His/Her role is to ensure that the project stays on track, meets its objectives and delivers value to its stakeholders.
- But these roles aren't isolated.
- They intertwine and interact in a dynamic environment.
- Collaboration is essential.
- The project team must work together, sharing knowledge, resolving conflicts and building on each other's ideas.

The 4 Ps in software engineering - People

- ❑ Communication is key, ensuring everyone is on the same page, understanding the project's goals, progress and challenges.
- ❑ Leadership, provided by the project manager, guides the team, motivates them and fosters a positive, productive environment.
- ❑ The people in project management are more than just cogs in a machine.
- ❑ They are the heart and soul of the project, the human element that makes the difference between success and failure.
- ❑ They bring the project to life, infuse it with their energy, creativity and passion and ultimately shape its outcome.
- ❑ In essence, a project's success largely depends on its people.

- This second P (**Process**) is all about the methodologies, techniques and procedures that guide a project from initiation to closure.
- To kick things off, we have the project management processes.
- These are essentially the roadmap for your project.
- They include a multitude of methods and strategies designed to streamline your project's journey. This could be anything from agile methodologies that prioritize flexibility and adaptability, to more traditional waterfall models that follow a strict, sequential approach.
- Each project will require a unique blend of methodologies, tailored to its specific needs and challenges.
- Next, move onto the project lifecycle.

- Just like any living thing, a project goes through several stages of life – from humble beginnings to fulfilling its purpose.
- These stages typically include initiation, planning, execution, monitoring and closure.
 - Initiation is where we define the project and get the necessary approvals.
 - Planning is where we map out how to achieve our goals. Execution is when the real work happens and our plans start to come to life.
 - Monitoring is where we track progress and make necessary adjustments.
 - Closure, well, that's where we tie up loose ends and celebrate our success.
- But, projects are rarely a smooth sail from start to finish.

The 4 Ps in software engineering - Process

- They are more like a roller coaster ride with twists, turns and the occasional loop-de-loop.
- That's where change management process comes in.
- It helps us navigate these unexpected changes without losing sight of our goals.
- This could involve managing changes to the project scope, schedule or resources.
- The key is to have a robust change management process that allows for flexibility without jeopardizing the project's objectives.
- So, the process pillar is not just about following a set of rules. It's about having a structured approach to managing projects, while still allowing room for change and adaptation.
- It's about understanding that each project is unique and requires a unique approach.
- "A well defined process can steer the project towards its goals".

The 4 Ps in software engineering - Product

- ❑ The **product** or the outcome of the project, is what ultimately delivers value to the stakeholder.
- ❑ The product is the tangible output of a project. It's the reason the project exists in the first place. This can be a physical product, a service or even a result that meets a specific requirement.
- ❑ Think of it as the end game, the goal post, the finish line – it's what everyone on the project team is working towards.
- ❑ Project deliverables are the concrete outputs or the outcomes that the project promises to deliver. They are the tangible aspects of the project that the stakeholder can see, touch and evaluate.
- ❑ Whether it's a newly built skyscraper, a software application or a market research report, these deliverables are the measurable products of all the hard work put into the project.

- But, simply creating a deliverable isn't enough.
- The quality of these deliverables is paramount.
 - Quality, in the context of project management, is the degree to which the deliverables meet the specified requirements and expectations. It's about ensuring customer satisfaction and project success.
- A software application full of bugs or a market research report that's riddled with errors, can hardly be considered successful.
- This brings us to the scope of the project. The scope is the defined boundaries and objectives of the project.
 - It lays down work that needs to be done, the features that need to be included and the constraints that need to be observed.
 - It's like the blueprint of the project.

- ❑ Without a clearly defined scope, a project can easily veer off course; leading to cost overruns, delayed timelines and unhappy stakeholders.
- ❑ So, what exactly is the product?
- ❑ Hence, the product in its essence, is a blend of well-defined deliverables, high quality and a well-managed scope.
- ❑ It's about delivering something tangible that meets the requirements, satisfies the stakeholders and adds value.
- ❑ The product of a project is its tangible value

- The **Project environment** (the 4th P) includes external and internal factors that can impact the project's success.
- About the organizational context (internal factors)
 - This refers to the culture, structure, policies and procedures within your organization.
 - Imagine it as the backdrop of your project.
 - It shapes how projects are initiated, executed and managed.
 - For example, an organization with a culture of innovation, might encourage risk-taking and experimentation in project, while a more conservative organization might prioritize adherence to established processes and standards.
- Then we have external factors.
 - These are influences outside of your organization that can affect your project. They could be market conditions, regulatory requirements, technological advancements or competitive pressures.

- For instance, a sudden change in market trends could necessitate a shift in your project's direction or a new regulatory law might require modifications to your project deliverables.

- Lastly, there's risk management
 - This involves the identification, assessment and mitigation of potential risks and uncertainties that may affect your project objective.
 - It's about foreseeing what could go wrong and having strategies in place to manage and respond to those risks effectively.
 - It could be as simple as having a plan B for a critical project task or as complex as having a detailed risk response strategy for potential legal or financial issues.

The 4 Ps in software engineering – Project environment

- All these elements of the project environment interact and influence each other.
- The organizational context might shape how risks are managed. External factors might create new risks or necessitate changes in the organization's policies.
- Successful project management involves navigating this complex web of influences to deliver successful project outcomes.
- Like a skilled sailor navigating a ship through stormy waters, a project manager needs to understand the project environment, anticipate potential challenges and adjust the project's course as necessary.
- It's not just about managing, tasks and deliverables.
- It's also about understanding the bigger picture and steering the project towards success amidst changing conditions.
- "Understanding the project environment helps it making informed decisions and mitigating risks".



THANK YOU

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1: Lecture 5

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Agile SW Development Life Cycle

Agile philosophy

Agile is an umbrella term used to describe a variety of methods



FDD – Feature Driven Development

DSDM – Dynamic System Development Method

RUP - Rational unified process

XP - Extreme Programming

Agile methods encourage:

1. Continual realignment of development goals with needs and expectations of the customer
2. Reducing massive planning overhead to allow fast reactions to change

Agile philosophy ...

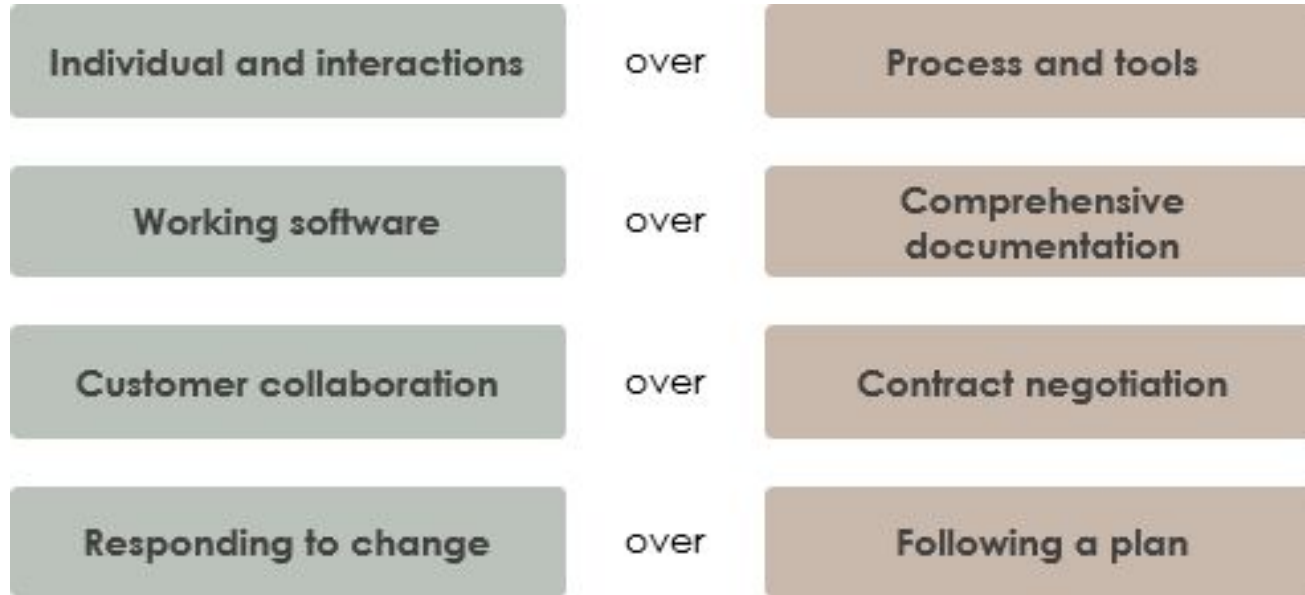
Agile is not a process.

It is a set of values or philosophy.

AGILE key words

- Rapid
- Iterative
- Cooperative
- Quality driven
- Adaptable

Agile manifesto



While there is value in the items on the right, we value the items on the left more.

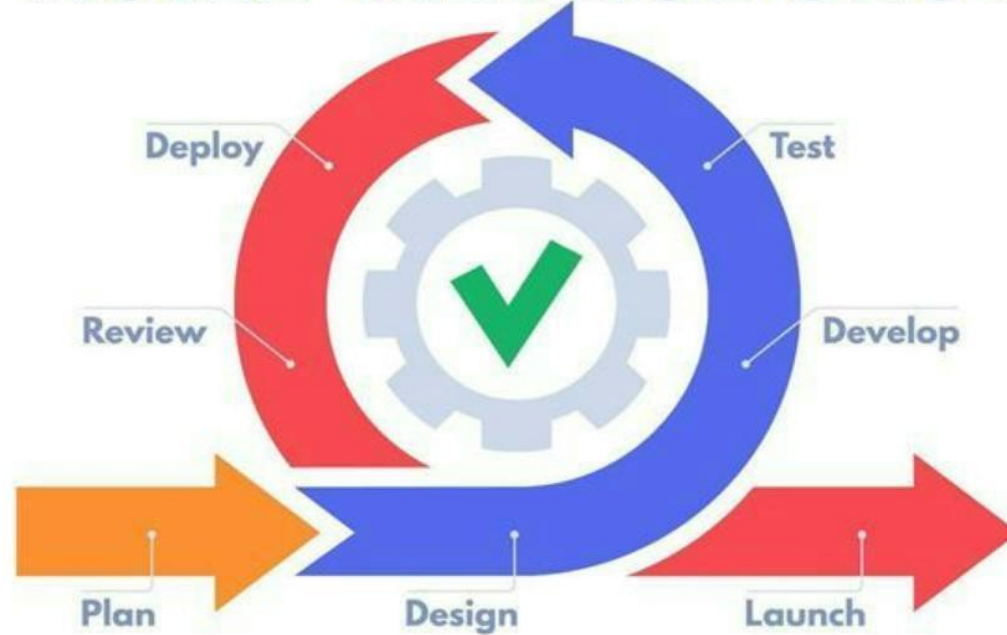
Agile principles

- Customer involvement
 - Customers should be closely involved throughout the development process. Their role is to provide and prioritize new system requirements and to evaluate the iterations of the system.
- Incremental delivery
 - The software is developed in increments with the customer specifying the requirements to be included in each increment.
- People not process
 - The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
- Embrace change
 - Expect the system requirements to change and so design the system to accommodate these changes.
- Maintain simplicity
 - Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

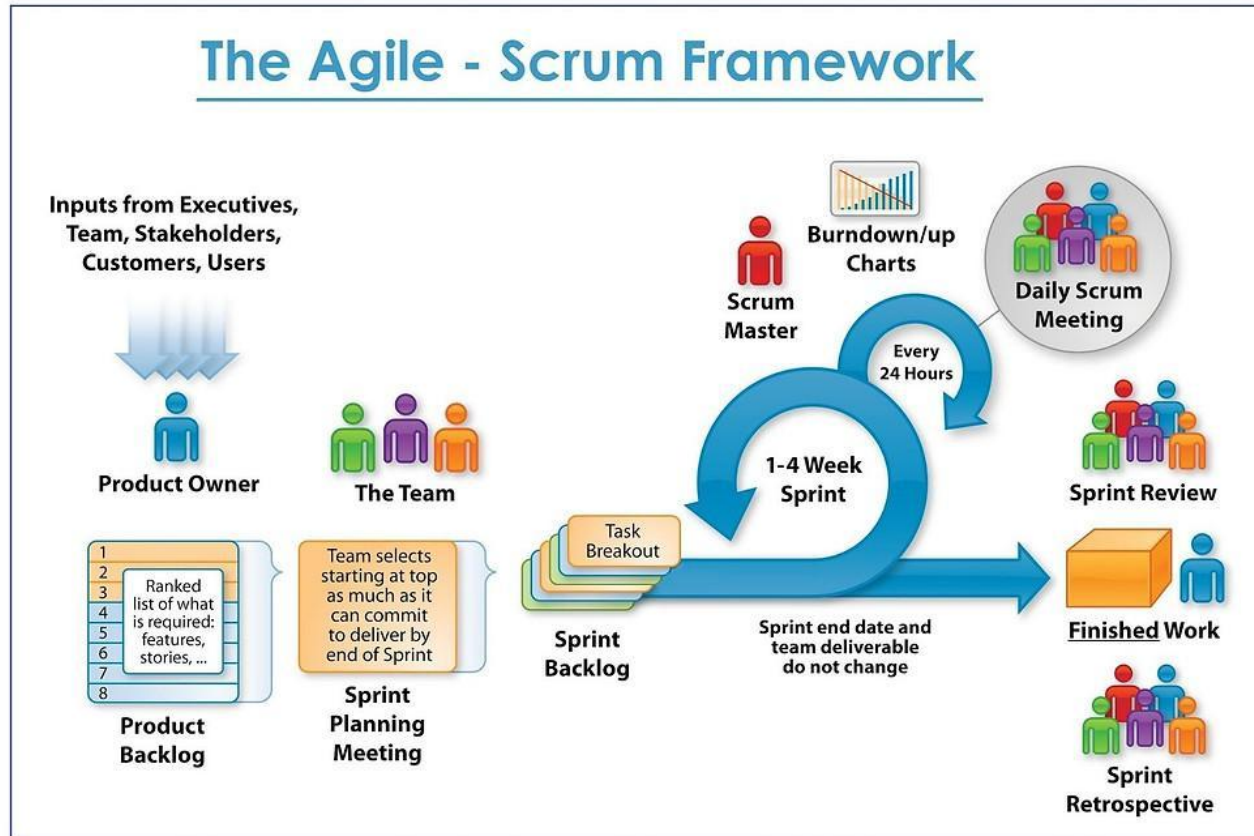
Pros and Cons

Pros	Cons
❑ Is a very realistic approach to software development ❑ Promotes teamwork and cross training. ❑ Functionality can be developed rapidly and demonstrated. ❑ Resource requirements are minimum. ❑ Suitable for fixed or changing requirements ❑ Delivers early partial working solutions. ❑ Good model for environments that change steadily. ❑ Minimal rules, documentation easily employed. ❑ Enables concurrent development and delivery within an overall planned context. ❑ Little or no planning required ❑ Easy to manage ❑ Gives flexibility to developers	❑ Not suitable for handling complex dependencies. ❑ More risk of sustainability, maintainability and extensibility. ❑ An overall plan, an agile leader and agile PM practice is a must without which it will not work. ❑ Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines. ❑ Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction. ❑ There is very high individual dependency, since there is minimum documentation generated. ❑ Transfer of technology to new team members may be quite challenging due to lack of documentation.

AGILE METHODOLOGY



Agile methodologies - Scrum



Scrum in brief



Roles

Scrum Team

- Team is cross-functional and consists of 5-9 people
- There are no set project roles within the team
- Team defines tasks and assignments
- Team is self-organizing and self-managing
- Maintains the Sprint Backlog
- Conducts the Sprint Review



Product Owner (PO)

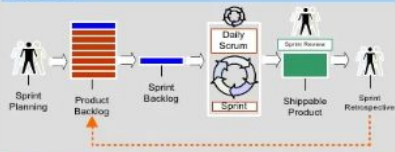
- Accountable for product success
- Defines all product features
- Responsible for prioritizing product features
- Maintains the Product Backlog
- Insures team working on highest valued features



Scrum Master (SM)

- Holds daily 15 minute team meeting (Daily Scrum)
- Removes obstacles
- Shields the team from external interference
- Maintains the Sprint Burndown Chart
- Conducts Sprint Retrospective at the end of a Sprint
- Is a facilitator not a manager

Process



Tools

Task Board

- White Board containing teams Sprint goals, backlog items, tasks, tasks in progress, "DONE" items and the daily Sprint Burndown chart.
- Scrum meeting best held around task board
- Visible to everyone

Artifacts

Product Backlog - (PB)

- List of all desired product features
- List can contain bugs, and non-functional items
- Product Owner responsible for prioritizing
- Items can be added by anyone at anytime
- Each item should have a business value assigned
- Maintained by the Product Owner

Sprint Backlog - (SB)

- To-do list (also known as Backlog item) for the Sprint
- Created by the Scrum Team
- Product Owner has defined as highest priority

Burndown Chart - (BC)

- Chart showing how much work remaining in a Sprint
- Calculated in hours remaining
- Maintained by the Scrum Master daily

Release Backlog - (RB)

- Same as the Product Backlog. May involve one or more sprints dependent on determined Release date

"DONE"= Potentially Shippable!

FAQ

- Who decides when a Release happens?** At the end of any given Sprint the PO can initiate a Release.
- Who is responsible for managing the teams?** The teams are responsible for managing themselves.
- What is the length of a task?** Tasks should take no longer than 16 hours. If longer then the task should be broken down further.
- Who manages obstacles?** Primary responsibility is on the Scrum Master. However, teams must learn to resolve their own issues. If not able then escalated to SM.
- What are two of the biggest challenges in Scrum?** Teams not self-managing, Scrum Master managing not leading.

Meetings

Sprint Planning – Day 1 / First Half

- Product backlog prepared prior to meeting
- First half – Team selects items committing to complete
- Additional discussion of PB occurs during actual Sprint

Sprint Planning – Day 1 / Second Half

- Occurs after first half done – PO available for questions
- Team solely responsible for deciding how to build
- Tasks created / assigned – Sprint Backlog produced

Daily Scrum

- Held every day during a Sprint
- Lasts 15 minutes
- Team members report to each other not Scrum Master
- Asks 3 questions during meeting
- "What have you done since last daily scrum?"*
- "What will you do before the next daily scrum?"*
- "What obstacles are impeding your work?"*
- Opportunity for team members to synchronize their work

Sprint Review

- Team presents "done" code to PO and stakeholders
- Functionality not "done" is not shown
- Feedback generated - PB maybe reprioritized
- Scrum Master sets next Sprint Review

Sprint Retrospective

- Attendees – SM and Team. PO is optional
- Questions – What went well and what can be improved?
- SM helps team in discovery – not provide answers

Visibility + Flexibility = Scrum

Glossary of Terms

- Time Box** - A period of time to finish a task. The end date is set and can not be changed
- Chickens** – People that are not committed to the project and are not accountable for deliverables
- Pigs** – People who are accountable for the project's success
- Single Wringable Neck** – This is the Product Owner!

SCRUM CHEAT SHEET

Estimating

User Stories

- A very high level definition of what the customer wants the system to do.
- Each story is captured as a separate item on the Product Backlog
- User stories are NOT dependent on other stories
- Story Template:**
- 'As a <User> I want <function> So that <desired result>'*
- Story Example:**
- 'As a user, I want to print a recipe so that I can cook it.'*

Story Points

- A simple way to initially estimate level of effort expected to develop
- Story points are a relative measure of feature difficulty
- Usually scored on a scale of 1-10. 1=very easy through 10=very difficult
- Example:**
- "Send to a Friend" Story Points = 2*
- "Shopping Cart" Story Points = 9*

Business Value

- Each User Story in the Product Backlog should have a corresponding business value assigned.
- Typically assign (L,M,H) Low, Medium, High
- PO prioritizes Backlog items by highest value

Estimate Team Capacity

- $Capacity = \# \text{ Teammates (Productive Hrs x Sprint Days)}$
- Example – Team size is 4, Productive Hrs are 5, Sprint length is 30 days.
- $Capacity = 4 (5 \times 30) = 600 \text{ hours}$
- NOTE:** Account for vacation time during the Sprint!

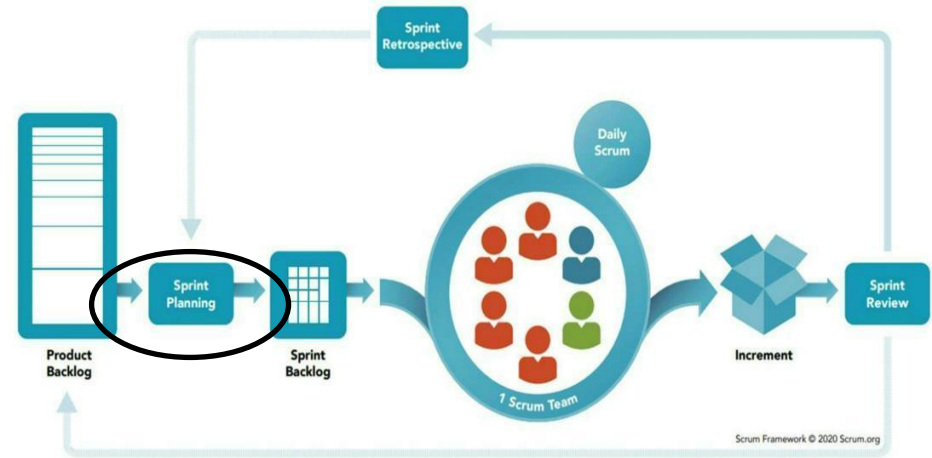
Velocity

- The rate at which team converts items to "DONE" in a single Sprint – Usually calculated in Story Points.

Key scrum activities

1. Sprint planning (30-60 min, biweekly)

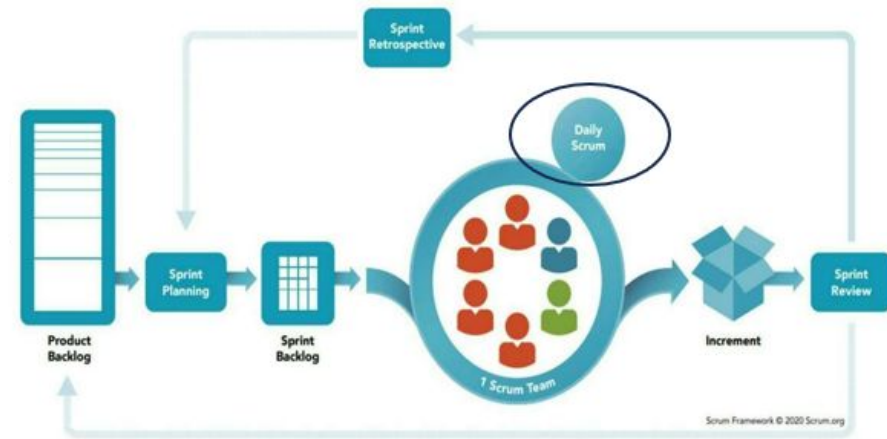
- What features can be delivered this Sprint? (from product backlog)
- What tasks are needed to get these features delivered? (build sprint backlog)



Key scrum activities

2. Daily standup(15 min, daily)

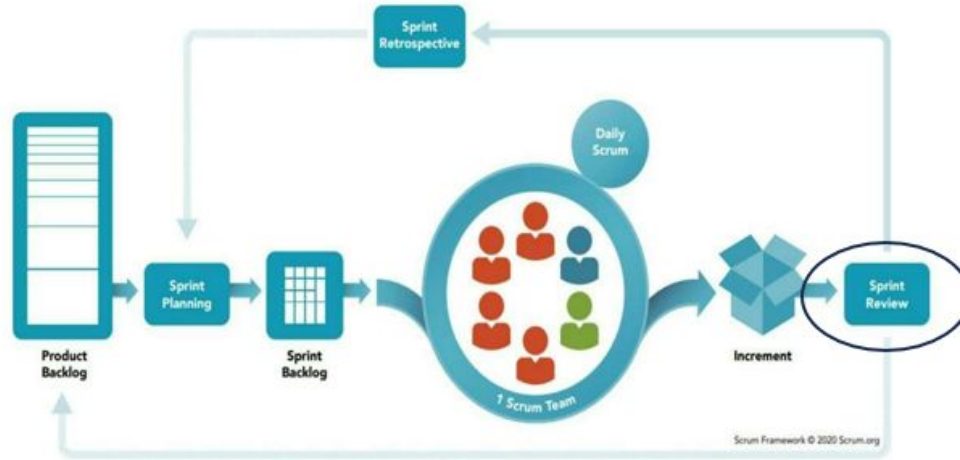
- What did I accomplish yesterday?
- What am I planning to work on today?
- Are there any blockers that are preventing me from making progress?



Key scrum activities

3. Sprint Review (30-60 min, biweekly)

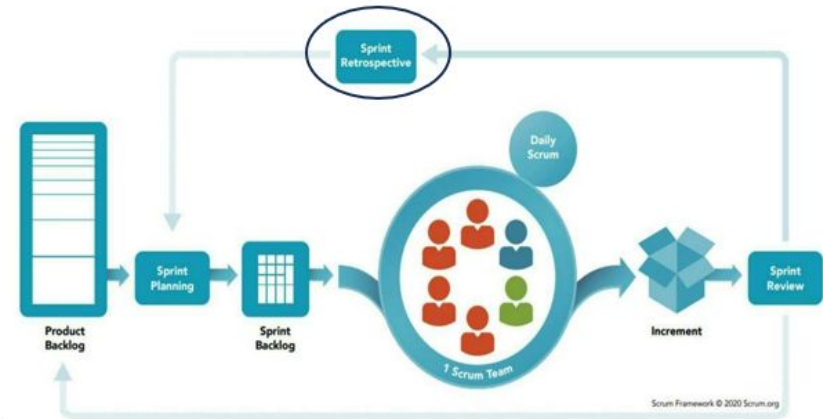
- Demo of working software to product owner



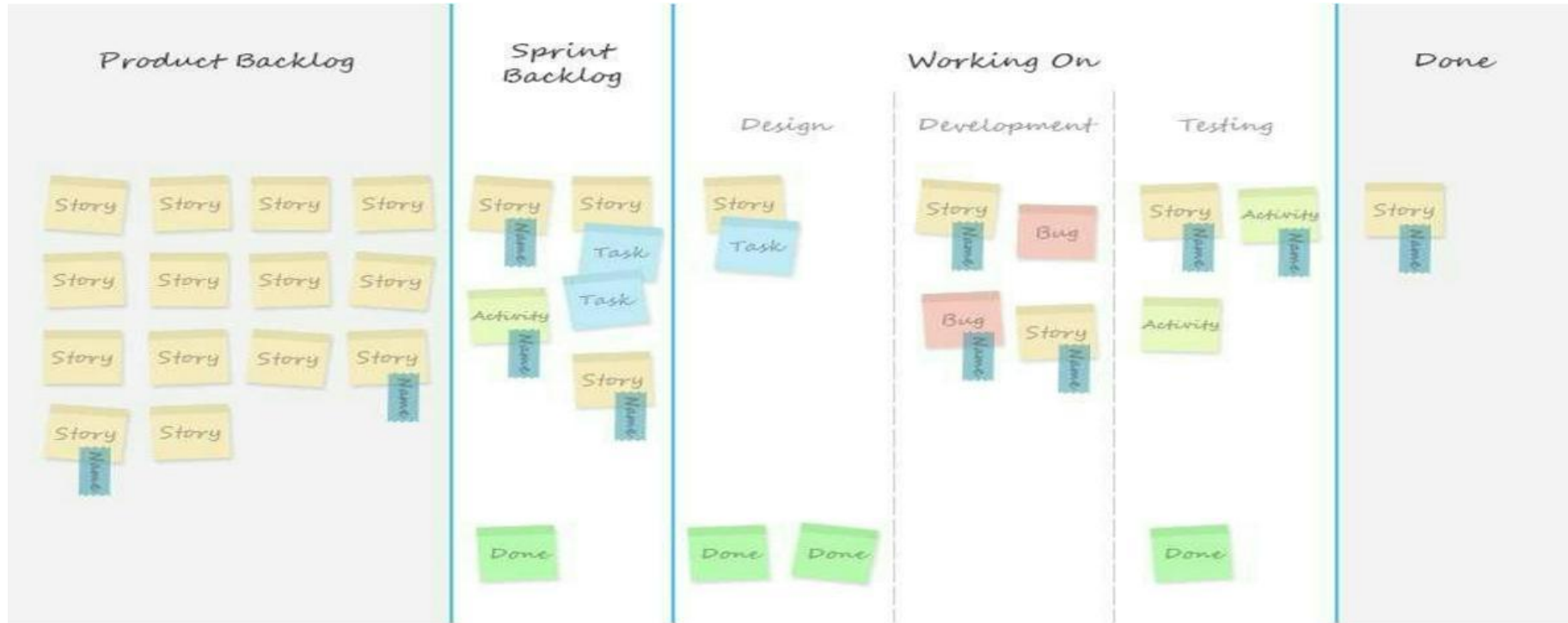
Key scrum activities

4. Sprint retrospective(30-60 min, biweekly)

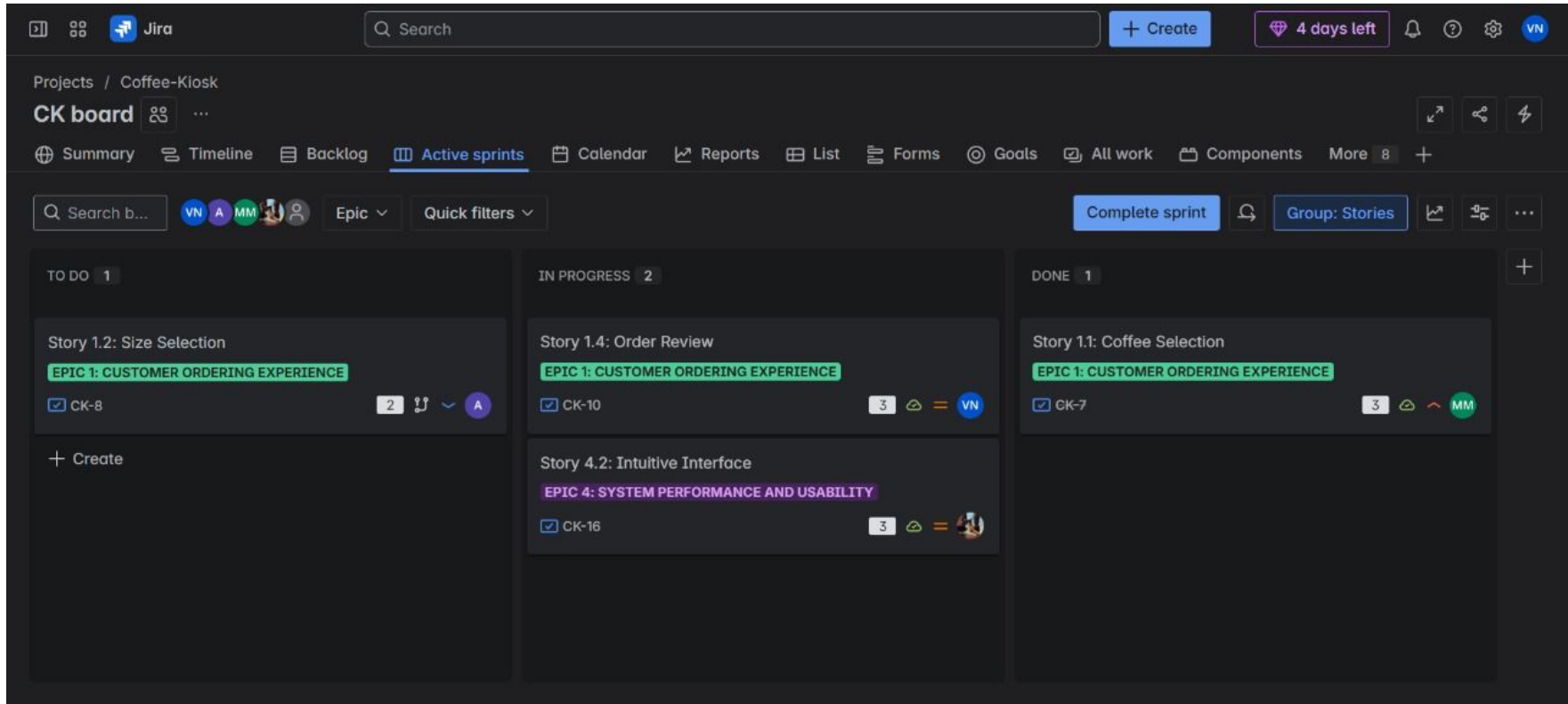
- Reflect on what went well and what could have gone better
- Identify specific actions to improve processes and teamwork in the next sprint



Leverage scrum planning tools



Leverage scrum planning tools



The screenshot displays the Jira interface for a project named "Coffee-Kiosk" (CK board). The top navigation bar includes a search bar, a "+ Create" button, and a "4 days left" indicator. Below the navigation bar, the "Active sprints" tab is selected, showing a sprint progress bar. The main content area is divided into three columns: "TO DO", "IN PROGRESS", and "DONE".

- TO DO (1 item):** Story 1.2: Size Selection. It is linked to "EPIC 1: CUSTOMER ORDERING EXPERIENCE" and has a checkbox for "CK-8".
- IN PROGRESS (2 items):**
 - Story 1.4: Order Review. It is linked to "EPIC 1: CUSTOMER ORDERING EXPERIENCE" and has a checkbox for "CK-10".
 - Story 4.2: Intuitive Interface. It is linked to "EPIC 4: SYSTEM PERFORMANCE AND USABILITY" and has a checkbox for "CK-16".
- DONE (1 item):** Story 1.1: Coffee Selection. It is linked to "EPIC 1: CUSTOMER ORDERING EXPERIENCE" and has a checkbox for "CK-7".

Each story card includes a title, a link to an epic, a checkbox for a specific task, and a status indicator (e.g., "2", "3", "4").

User Story

- ❑ Follow the INVEST guidelines for good user stories
- ❑ Each component ensures that user stories are well-structured, facilitating better planning, execution, and delivery in Agile projects.

Independent

Negotiable

Valuable

Estimable

Small

Testable

Independent

- **Definition:** User stories should be self-contained, with minimal dependencies on other stories.
- **Importance:** Independence allows teams to prioritize and implement stories in any order, reducing bottlenecks and complexities.
- **Example:** Instead of having a story like "Implement payment processing," which might depend on "User registration," consider breaking them into separate, independent stories

A mobile/web application for college students to view events, register, and receive notifications.

- As a student, I want to register for events after creating my profile so I can get updates.
- Problem: Depends on profile creation; not independent

Negotiable

- **Definition:** User stories are not contracts but starting points for discussions between stakeholders and the development team.
- **Importance:** Flexibility in stories encourages collaboration, allowing teams to adapt and refine requirements as more information becomes available.
- **Example:** A story stating "As a user, I want to receive notifications" is negotiable, allowing the team to decide on the best implementation method.

A mobile/web application for college students to view events, register, and receive notifications.

- As a student, I must receive SMS notifications for every event registration.
- Problem: Rigid on SMS only; not open for discussion
- Not Negotiable

- **Definition:** Each user story should deliver value to stakeholders or end-users.
- **Importance:** Focusing on value ensures that the team works on features that provide tangible benefits, aligning with business goals.
- **Example:** "As a customer, I want to view my order history so I can track my past purchases."

A mobile/web application for college students to view events, register, and receive notifications.

- As a student, I want to see the college's logo on the app's home screen.
- Problem: Does not provide clear user value related to event management.

Estimable

- **Definition:** The team should be able to estimate the effort required to complete a user story.
- **Importance:** Estimations help in planning and resource allocation. If a story is too vague to estimate, it might need to be broken down or clarified.
- **Example:** A story like "Improve website performance" is too broad. Specifying "Reduce homepage load time by 2 seconds" makes it estimable.

A mobile/web application for college students to view events, register, and receive notifications.

- As a student, I want the app to load extremely fast and be secure.
- Problem: Vague; no clear criteria for "fast" or "secure."
- Difficult to Estimate

- ❑ **Definition:** User stories should be concise enough to be completed within a single iteration.
- ❑ **Importance:** Smaller stories are easier to estimate, test, and deliver, promoting continuous progress.
- ❑ **Example:** Instead of "Develop user account management," break it into "Implement user login," "Implement password reset," etc.

A mobile/web application for college students to view events, register, and receive notifications.

- As a student, I want to view upcoming events, register, and see my registration history in one place.
- Problem: Multiple features in one story; difficult to complete in one sprint.

- **Definition:** A user story should have clear acceptance criteria to determine when it's complete.
- **Importance:** Testability ensures that the team can verify the functionality, maintaining quality and reliability.
- **Example:** "As a user, I want to receive a confirmation email after registration," with acceptance criteria detailing the email's content and delivery time.

A mobile/web application for college students to view events, register, and receive notifications.

- As a student, I want the app to be fun and easy to use
- Problem: No acceptance criteria or test conditions.

Applying INVEST in Practice

When creating user stories:

- ❑ **Start with the user:** Frame stories from the end-user's perspective to ensure relevance and value.
- ❑ **Collaborate:** Engage stakeholders in discussions to refine stories, ensuring they are negotiable and valuable.
- ❑ **Break down large stories:** If a story seems too big or complex, divide it into smaller, more manageable pieces.
- ❑ **Define clear acceptance criteria:** This makes stories testable and sets clear expectations for completion.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1: Lecture 6

Compiled by
Rajesh Banginwar

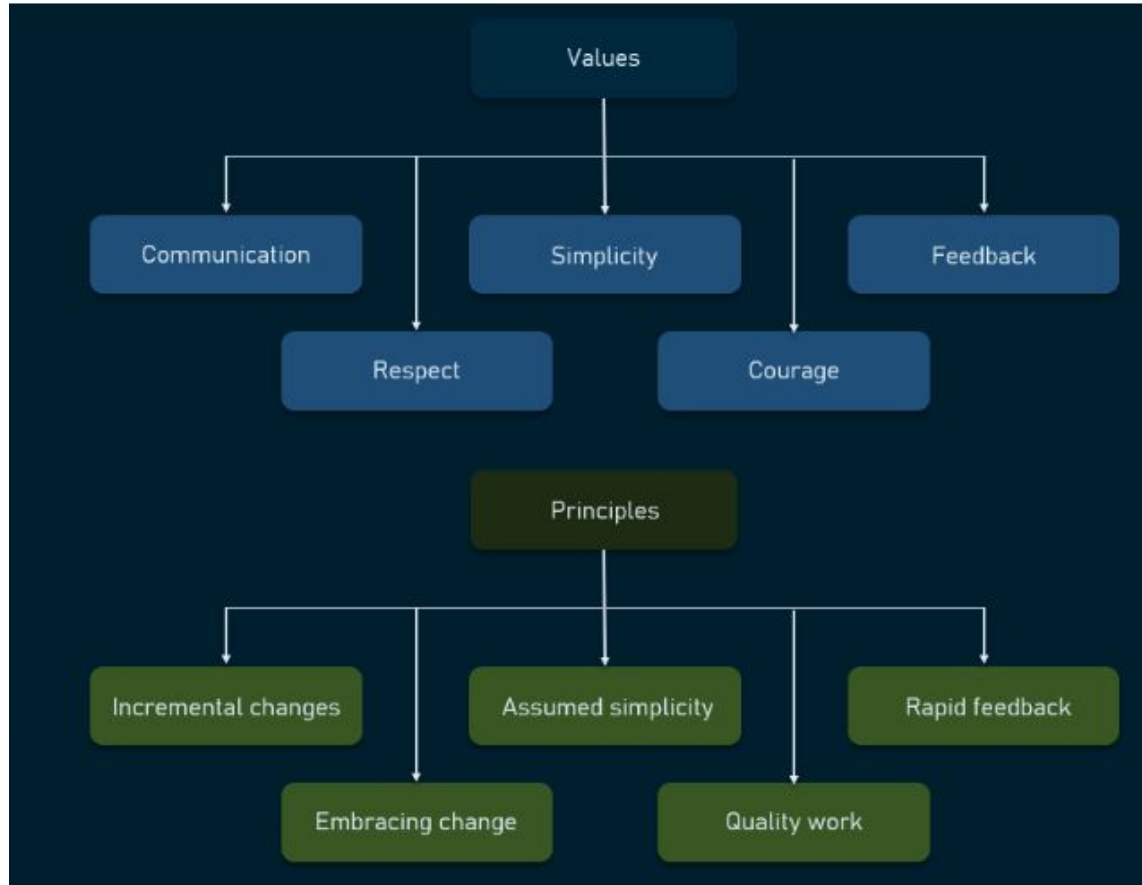
Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Agile Methodologies

- Extreme programming is a software development methodology that's part of what's collectively known as agile methodologies.
- XP is built upon values, principles, and practices, and its goal is to allow small to mid-sized teams to produce high-quality software and adapt to evolving and changing requirements.

XP values and principles



XP core practices

#1 Planning Game

- In XP the main planning process is called Planning Game. There are 2 levels of plans in XP; level one is release planning and level 2 is iteration planning.
- The first phase is release planning. The team and stakeholders/customers collaboratively decide what are the requirements and features that can be delivered into production and when. This is done based on priorities, capacity, estimations and risks factor of the team to deliver.
- In Iteration planning the team will pick up the most valuable items from the list and break them down into tasks, then estimates and a commitment to delivering at the end of the iteration.

XP core practices ...

#2 Simple design

- In XP, the team will not do complex or big architecture and designs upfront; instead the team will start with a simple design and let it emerge and evolve over a period of iterations. The code is frequently refactored so that it will be maintainable and free of technical debt. Simple designs make the 'definition of done' easier.
- XP teams conduct a small test or proof-of-concept workout called spike. The outcome of the spike helps the team to understand the validity of the hypothesis, gauge the complexity of the solution and feel assured to estimate and build something primarily based on the test.

#3 Test-Driven Development (TDD)

- In XP, developers write the unit test cases before starting the coding. The team automates the unit tests and it helps during the build and integration stage. The main benefit of TDD is that programmers have to only write the code that passes the tests.

- Basics steps of TDD,
 - Write the unit test case first
 - Write a minimal amount of code to pass the test.
 - Refactor it by adding the needed feature and functionality, while continuously making sure the tests pass.

#4 Code Standard

- Organizations want their programmers to hold to some well-described and standard style of coding called coding standards. It is a guideline for the development team as in XP. Since there are multiple programming pairs at play, coding standards are very useful to make consistency in code, style, naming convention, exception handling and use of parameters.
- These standards must be defined and agreed before the team starts the coding.
- It will make the code simple to understand and helps detect the problem or issues quickly and also increases the efficiency of the software.

#5 Refactoring

- Refactoring, as the word suggests, is restructuring or reconstructing existing things. In XP, over a period of time, the team produces lots of working code which increases the complexity of the whole code. To avoid this, we should consider the points mentioned as under:
 - Ensure code or functions are not duplicated.
 - No long functions or methods
 - Removing unnecessary variables
 - Proper use of access modifiers etc.

- By refactoring, the programmers look to improve the overall code quality and make it more readable without altering its behavior.

#6 Pair Programming

- Pair programming consists of two programmers operating on the same code and unit test cases, on the same system (one display and one keyboard). One programmer plays the pilot role and focuses on clean code, and compiles and runs. The second one plays the role of navigator and focuses on the big picture and reviews code for improvement or refactoring.
- Every hour or given a period of time this pair is allowed to switch roles so that the pilot will play the role of navigator and vice versa.
- The pairs of pilots and navigators are also not fixed and they are frequently swapped, the main benefits of that over a period of time is that everyone gets to know about the code and functionality of the whole system.

XP core practices ...

#7 Collective code ownership

- By following pair programming practices, the XP team always takes collective ownership of code. Success or failure is a collective effort and there is no blame game. There is no one key player here, so if there is a bug or issue then any developer can be called to fix it.

#8 Continuous Integration

- In XP, Developers do pair programming on local versions of the code. There is a need to integrate changes made every few hours or on a daily basis so after every code compilation and build we have to integrate it where all the tests are executed automatically for the entire project.
- If the tests fail, they are fixed then and there, so that any chance of defect propagation and further problems are avoided with minimum downtime.

XP core practices ...

#9 Small release

- A cross-functional team in XP, releases Minimum Viable Product (MVP) frequently. Small releases also help to breakdown complex modules into small chunks of code. This helps the developer team as well as the on-site customer to demonstrate the product and focus only on the least amount of work that has the highest priority.

#10 System Metaphor

- This is mainly related to user stories; the stories must be simple enough to be easily understood by user and developers and to relate it with code.
- It could be a naming convention practice used in design and code to have a shared understanding between teams. For example `Order_Food()` is easily explained -- this will be used to order food.
- It easy for the team to relate to the functionality of the particular component by just looking at its name.

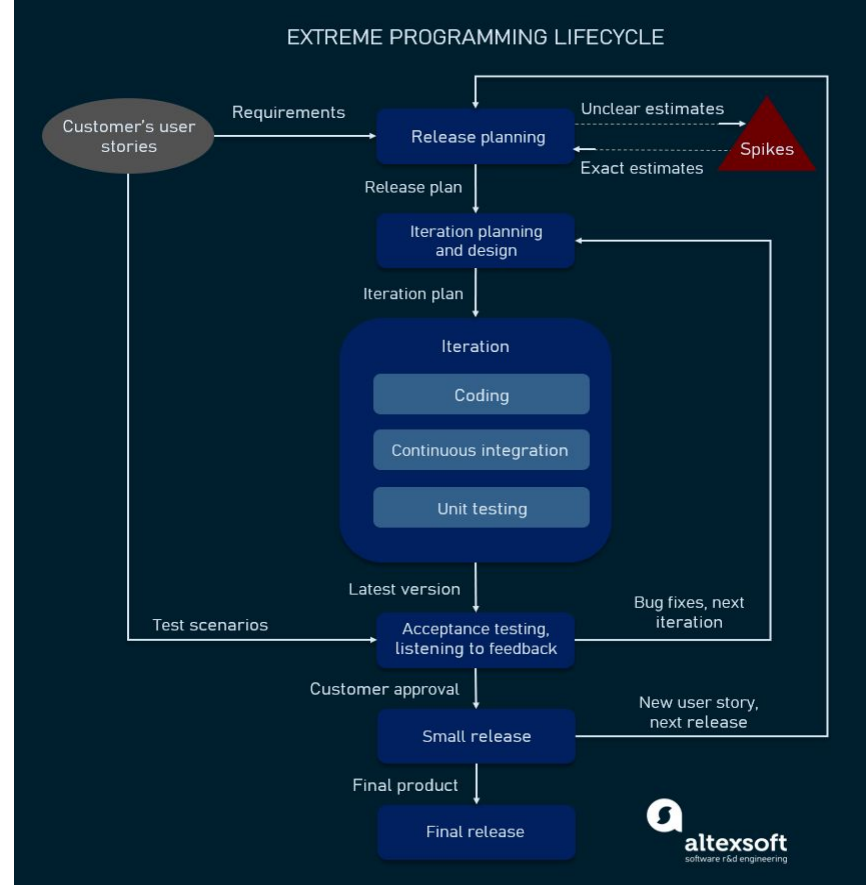
#11 Onsite Customer

- This is a role similar to Product Owner in Scrum. Onsite customer plays a major role here and is responsible for crafting the vision, defining user stories and acceptance criteria, the definition of “done” and release planning.
- They are the experts who know the domain or product and know how to generate a return of investment (ROI) by delivering the minimum viable product (MVP).
- If the onsite customer role is not full time, the role can be filled with product managers, product owners, UI-UX designers and business analysts who are called proxies.
- The word “on-site” implies that the customers or their representatives sit with the rest of the team to ensure that communication flows freely.

#12 Sustainable pace

- This is a people-centric practice. In XP practices like TDD, continuous integration and refactoring of code help to proactively improve the quality and stability of the working software.

- XP maintains a sustainable pace by introducing down time during the iteration. The team is not doing actual development at this time but acts as a buffer to deal with uncertainties and issues. Teams can use the slack time to pay down technical debt by refactoring code or do research to keep up the pace.



Lean Agile

- Lean-Agile is a set of principles and practices for working that aims to minimise waste while maximising value.
- This enables organisations to make quality a priority in their products and services.

Lean Agile Principles

❑ **Eliminate waste**

What's waste? It's anything that doesn't "add value to a product, value as perceived by the customer." Waste is anything produced or sitting around that doesn't get used. It can be "[when] developers code more features than are immediately needed." The time lost shifting development from one group to another? That's waste too.

❑ **Amplify learning**

Tackling a software solution is a lot like trying to put together a new recipe for a top restaurant dish. A chef will iterate and learn from the variations produced. Software development is more complex, and teams can be big, but when you amplify learning, it makes this discovery process possible.

Lean Agile Principles...

❑ **Decide as late as possible**

In something like embedded software development, there can be many unknowns. Which processor is going to fit the product and users' demands best? Should we develop in x, y or z language? Is this screen better than that screen? Functionality likely to change due to user or market feedback? There's no value in speculating. Instead, delay decisions until you have the facts.

❑ **Deliver as fast as possible**

“Design, implement, feedback, improve,” and repeat. “Rapid development” is entirely plausible; its speed enables you to make informed decisions with real feedback. Keeping this cycle short, amplifies learning as well as making sure “customers get what they need now, not what they needed yesterday.”

Lean Agile Principles...

□ Empower the team

The people doing the work are the ones who understand the details the best. Want “excellence”? Make sure your technical teams are involved in “the details of technical decisions”. Enable these teams to use “pull techniques” and “local signalling mechanisms” (like Kanban) to schedule and complete work whilst letting everyone know what’s coming up. So long as a leader guides the team, “they will make better technical decisions and better process decisions than anyone can make for them.”

Lean Agile Principles...

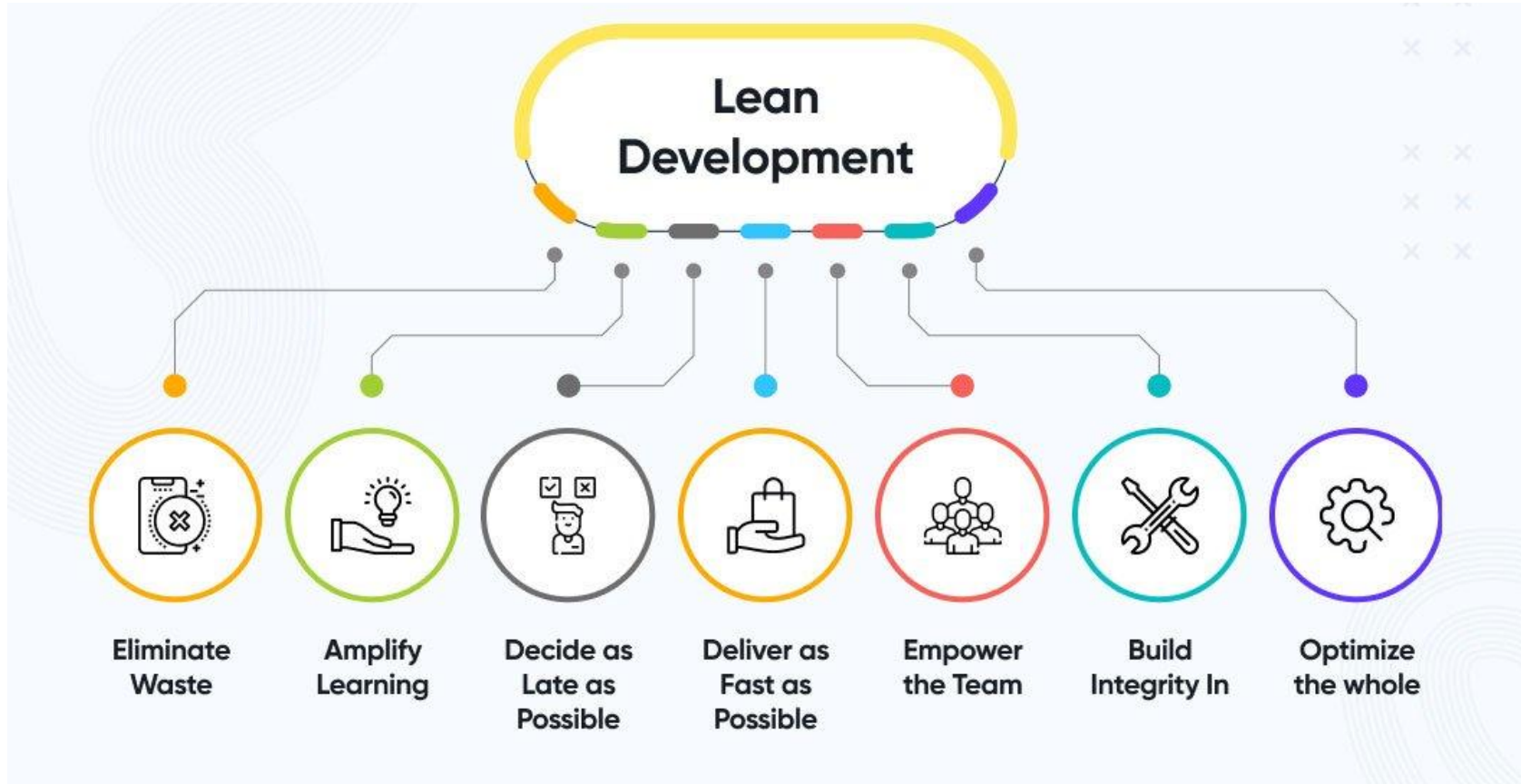
□ Build integrity in

Here the focus is on ensuring quality. This is more than just customers being happy with the final product, its “perceived integrity”. And it’s going further than “conceptual integrity” where the whole works seamlessly together. For software, “building integrity in” means it can evolve smoothly over time, because it has “coherent architecture, scores high on usability and fitness for purpose, and is maintainable, adaptable, and extensible.” For the team, integrity comes “from wise leadership, relevant expertise, effective communication, and healthy discipline”.

□ Optimize(See) the whole

Don’t just look at one area of a project. Step back and look at all areas involved. “Quite often, the common good suffers if people attend first to their own [specialised] interests.” Instead of measuring based on people’s “[specialised] contribution”, look at the overall performance of the project.

Lean Agile Principles...





Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1 – Lecture 7

Compiled by
Rajesh Banginwar

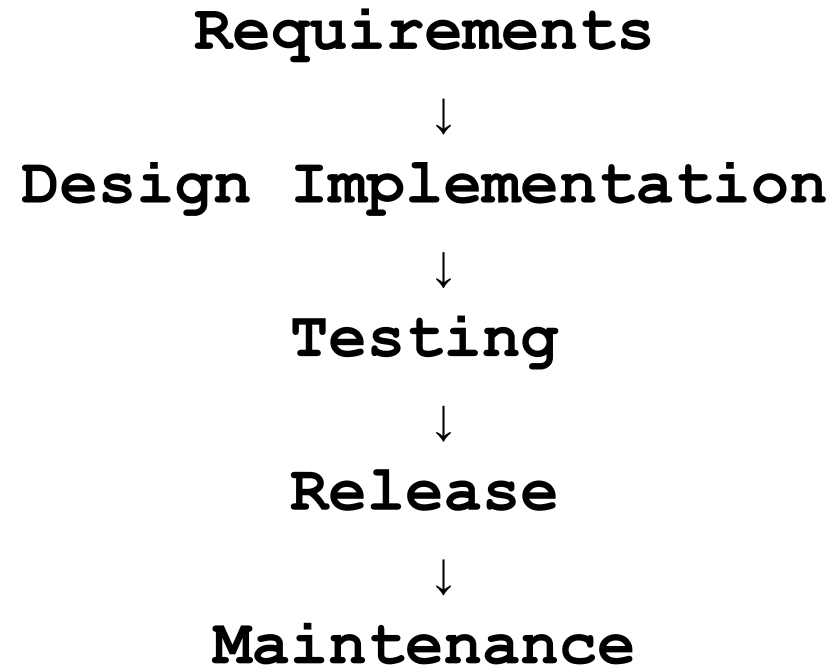
Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Requirements Engineering: Introduction & Elicitation

Recapping where requirements fit in

- Virtually all SDLC models have the following stages



- Requirements are at the top of the list of as we start the journey of product development

Levels of Abstraction

Requirements

High level “what” needs to be done



Architecture (High-Level Design)

High level “how” , mid-level “what”



Design (Low-Level Design, e.g. design patterns)

Mid-Level “how”, low-level “what”

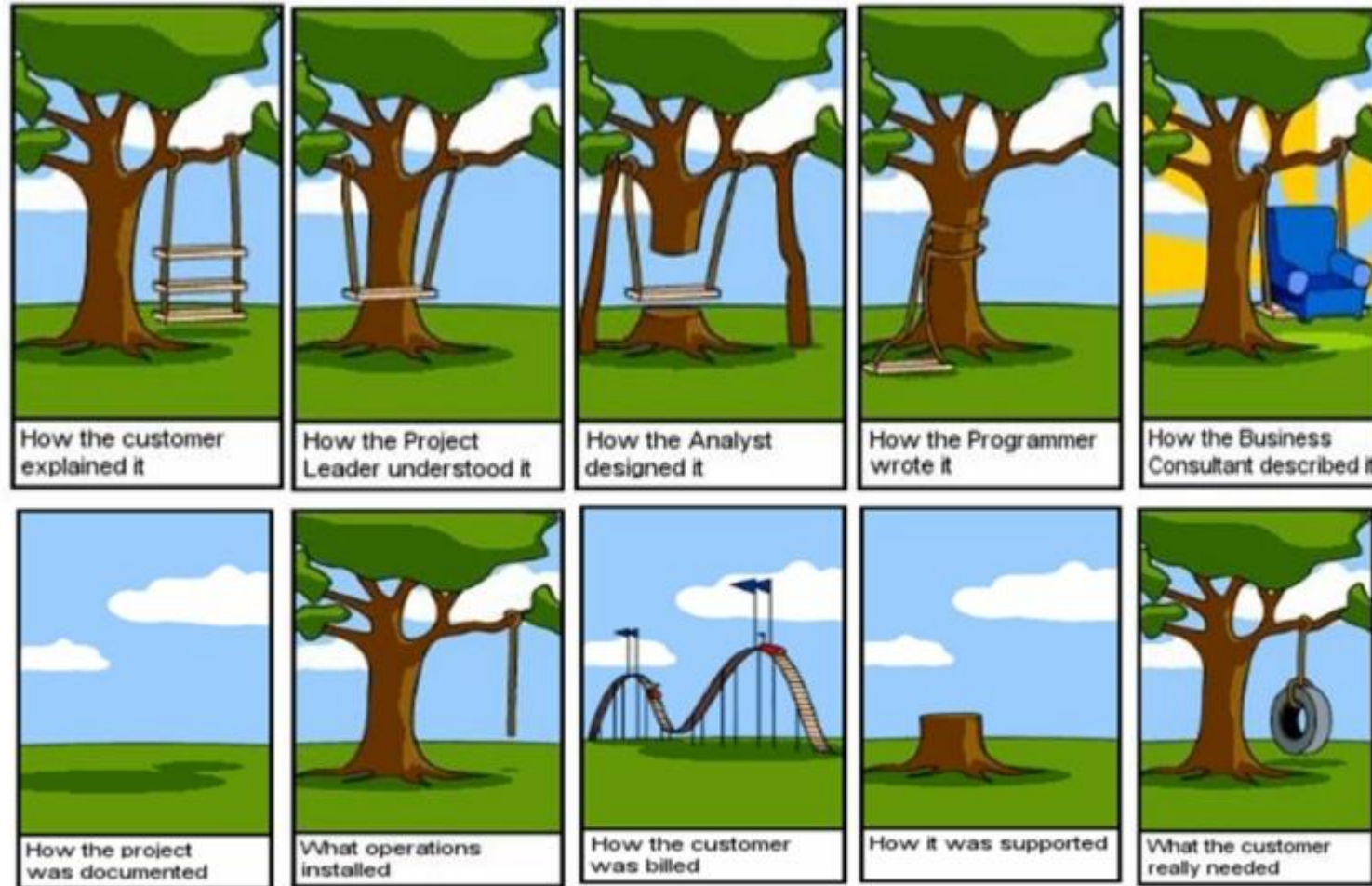


Code

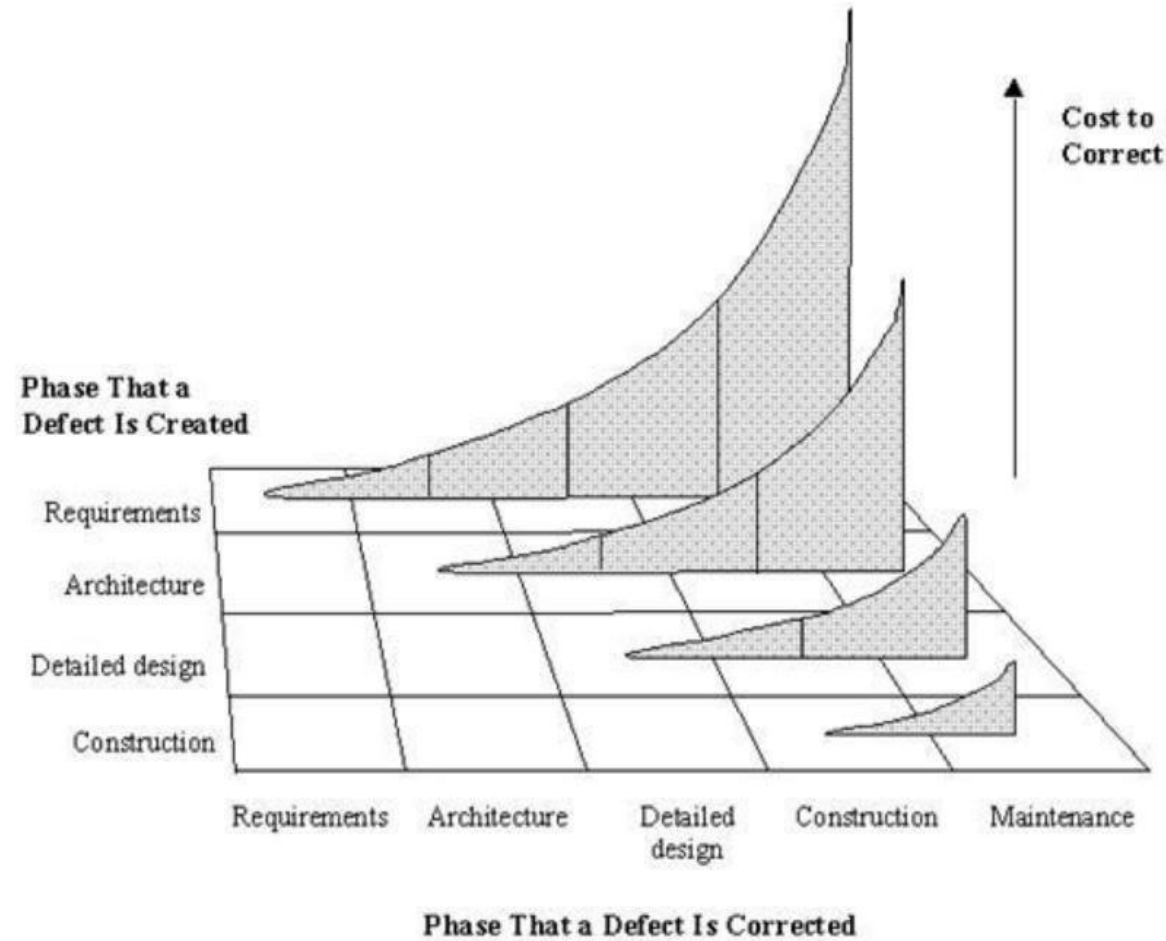
Low-Level “how”

What is Requirement?

- Requirement is the property which must be exhibited by software developed/adapted to solve a particular problem
- Requirement should specify the externally visible behavior of what and not how.
- Requirement engineering is the first step in any software intensive development lifecycle irrespective of model
 - Difficult, error prone and costly
 - Critical for successful development of all down stream activities
 - Requirement errors are expensive to fix

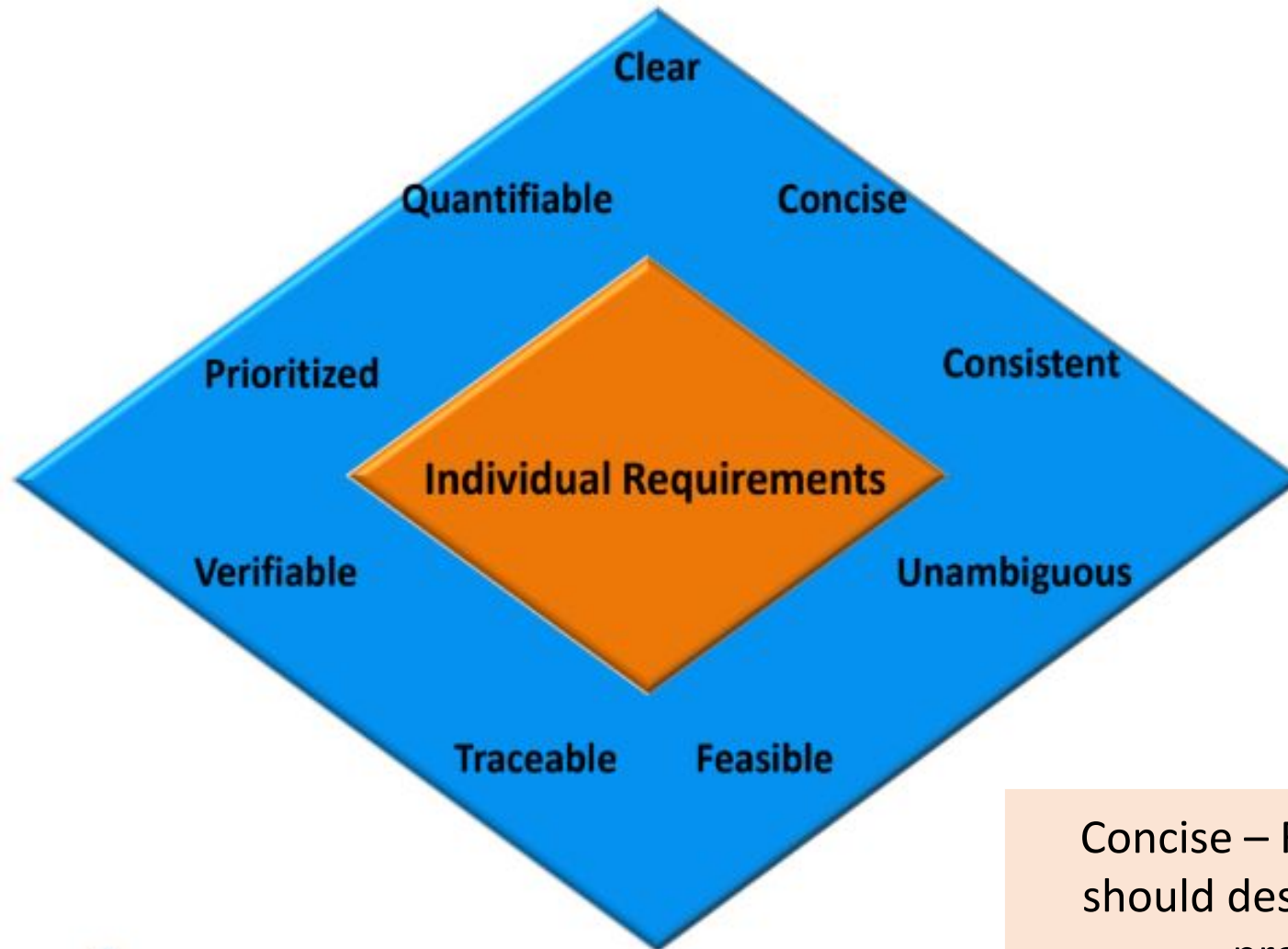


Cost of repair as a function of time



Copyright 1998 Steven C. McConnell. Reprinted with permission from *Software Project Survival Guide* (Microsoft Press, 1998).

Properties of Requirements



Concise – Requirements should describe a single property

Requirements Engineering - In-Class Exercise

Use the properties of requirement to transform the given sentences into requirements.

- **All screens must appear quickly on the monitor**
 - **When the user accesses any screen, it must appear on the monitor within 2 seconds** (Clear, Concise, Unambiguous, Verifiable, Measurable)
- **The replacement control system shall be installed with no disruption to production**
 - **The replacement control system shall be installed causing no more than 2 days of production disruption** (Feasible)
- **The system must generate a batch end report and a discrepancy report when a batch is aborted**
 - **The system must generate a batch end report when a batch is completed or aborted**
 - **The system must generate a discrepancy report when a batch is aborted** (Traceable)
- **The system must be user friendly**
 - **The user interface shall be menu driven. It shall provide dialog boxes, help screens, radio buttons, dropdown list boxes, and spin buttons for user inputs** (Verifiable)

Properties of Set of Requirements



- ☐ When there are many requirements but limited time or budget, choices must be made about which to include or exclude.
- ☐ Factors such as:
 - changes in customer needs
 - improved developer understanding of the products
 - changes in organizational policy

will affect the stability of requirements.

Let's start with some data

From a Standish report on software project

Customer Involvement is 3rd highest factor of success!

CHAOS FACTORS OF SUCCESS

FACTORS OF SUCCESS	POINTS	INVESTMENT
Executive Sponsorship	15	15%
Emotional Maturity	15	15%
User Involvement	15	15%
Optimization	15	15%
Skilled Resources	10	10%
Standard Architecture	8	8%
Agile Process	7	7%
Modest Execution	6	6%
Project Management Expertise	5	5%
Clear Business Objectives	4	4%

https://www.standishgroup.com/sample_research_files/CHAOSReport2015-Final.pdf

The 2015 Factors of Success. This chart reflects our opinion of the importance of each attribute and our recommendation of the amount of effort and investment that should be considered to improve project success.

Successful businesses always start with the user's goal

Marshall Field's

The customer
is always right

Google

Focus on the
user and all
else will follow

amazon

Customer
obsession
rather than
competitor
focus



Understand
and serve the
customer
better than
anyone else

Do's and Don'ts in Requirements gathering

□ Do's:

- Talk to the **customers** -- to learn how they work and need from the product
- Ask questions throughout the process – identify hidden requirements
- Understand “why” part of their activity and not just “what”.
- Expect requirements to change later – during and after development

□ Don'ts:

- Be too specific or detailed for requirements
- Elaborate on implementation complexity or logic behind the system
- Describe “how” it will be implemented including UX
- Try to think of implementation and associated details
- Add unnecessary features not wanted by the customers

Introduction to Requirements Engineering

- The whole process is more formally known as requirements engineering.
- The science of eliciting, analyzing, documenting, and maintaining requirements
- As you collect your class project requirements, consider three categories:
 - Functional requirements
 - e.g., input-output behavior
 - Non-functional requirements
 - e.g., security, privacy, scalability
 - Additional constraints
 - e.g., programming language, frameworks, testing infrastructure

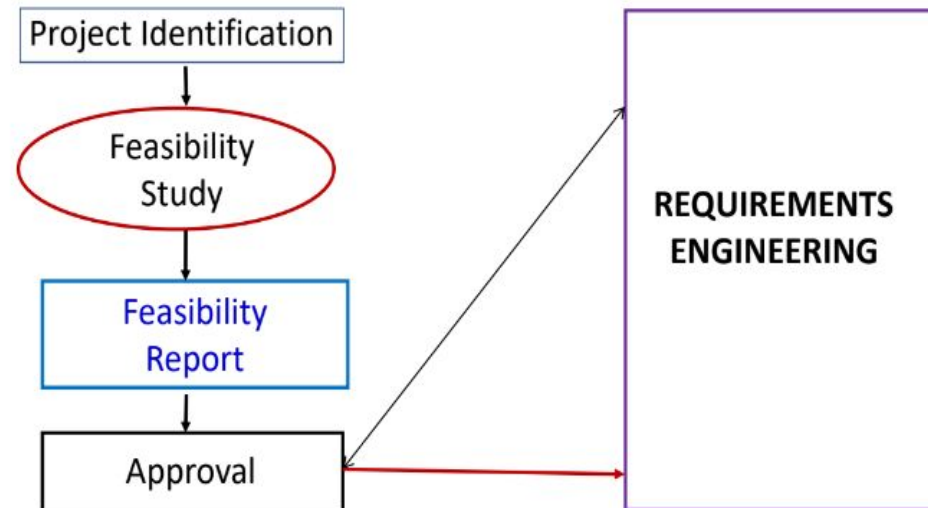
Feasibility Study

□ What is Feasibility Study?

- Short, low-cost study to assess the practicality of the project and whether it should be done.

□ When is Feasibility Study conducted?

- Mostly done before beginning a project.



Activities in Feasibility Study

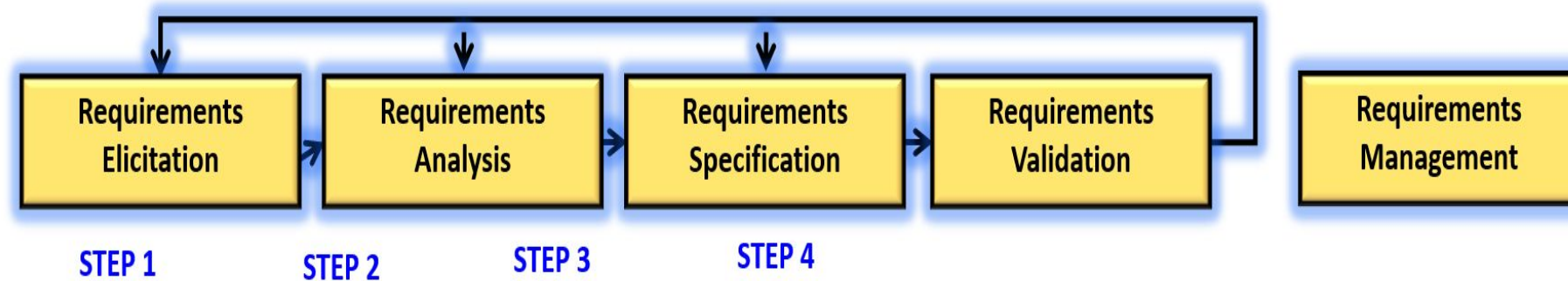
□ Activities in Feasibility Study:

- Figure out the client or the sponsor or the user who would have a stake in the project
- Find the current solution to the problem
- Find the targeted customers and the future market place
- Potential benefits
- Scope
- High level block level understanding of the solution
- Considerations to technology
- Marketing strategy
- Financial projection
- Schedule and high-level planning and budget requirements
- Issues, assumptions, risks and constraints
- Alternatives and their consideration
- Potential project organization
- Ends with **GO** or **NO-GO**

Requirements Engineering Process

A “four + one” set of activities to produce specifications or requirements

It is an iterative process



Requirements Validation – Helps ensure the right requirements are realized

Requirements Elicitation

- It is the process of working proactively with all stakeholders gathering their needs, articulating their problem, identifying and negotiating potential conflicts thereby establishing a clear scope and boundary for a project.

- It involves:
 - Understanding the problem
 - Understanding the domain
 - Identifying clear objectives
 - Understanding the needs
 - Understanding constraints of the system stake holders
 - Writing business objectives for the project

Elicitation Techniques...

- Approach is based on:
 - Nature of the system being developed
 - Background and experience of stakeholders
- Elicitation techniques – **Active** and **Passive**

Active elicitation techniques

Ongoing interaction between the stake holders and users.

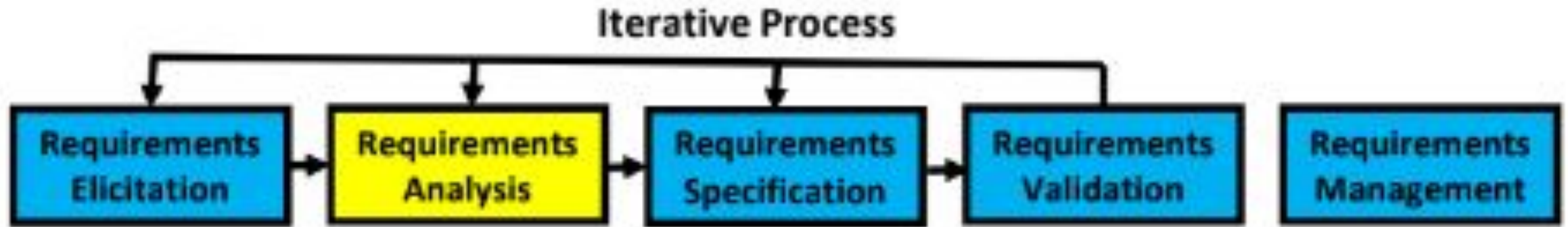
- Interviews
- Facilitated meetings
- Role-playing
- Prototypes
- Ethnography
- Scenarios

Passive elicitation techniques

Infrequent interaction between the stake holders and users.

- Use cases
- Business process analysis & modeling
- Workflows
- Questionnaires
- Checklists
- Documentation

Process of Requirements Analysis...



Process of Requirements Analysis

- ❑ Understand requirements in depth
- ❑ Classify requirements into coherent clusters
- ❑ Model the requirements
- ❑ Analyze requirements using fishbone diagram
- ❑ Recognize and resolve conflicts
- ❑ Negotiate requirements
- ❑ Prioritize requirements – MoSCoW (**Must have, Should have, Could have, Won't have**)
- ❑ Identify risks
- ❑ Decide on build or buy – COTS solution.

Process of Requirements Analysis...

- Understand requirements in depth.

- Classify requirements into coherent clusters:
 - Functional requirements
 - Non-functional requirements
 - User requirements
 - System requirements
 - Domain requirements

Process of Requirements Analysis...

□ Functional requirements

- Functionality or services, the system should provide with different inputs, and expression on how the system should behave in particular situations.

- Examples:

The system must send a confirmation email whenever an order is placed. The system must allow blog visitors to sign up for the newsletter by leaving their email. The system must allow users to verify their accounts using their phone number.

□ Non-functional requirements

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards.

- Examples:

The system should be available and responsive when needed, and should not experience frequent failures or crashes. Usability: Usability requirements describe the ease of use and user-friendliness of the software system.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1 – Lecture 8

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Requirements Engineering: Functional, Non Functional, Security Requirements

Functional Requirements

- Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations
- Describe functionality or system services.
- May state what the system should not do
- Depend on the type of software, expected users and the type of system where the software is used.
- Examples:
 - The system must send a confirmation email whenever an order is placed.
 - The system must allow blog visitors to sign up for the newsletter by leaving their email.
 - The system must allow users to verify their accounts using their phone number.

Requirements Imprecision

- ❑ Problems arise when functional requirements are not precisely stated.
- ❑ Ambiguous requirements may be interpreted in different ways by developers and users.

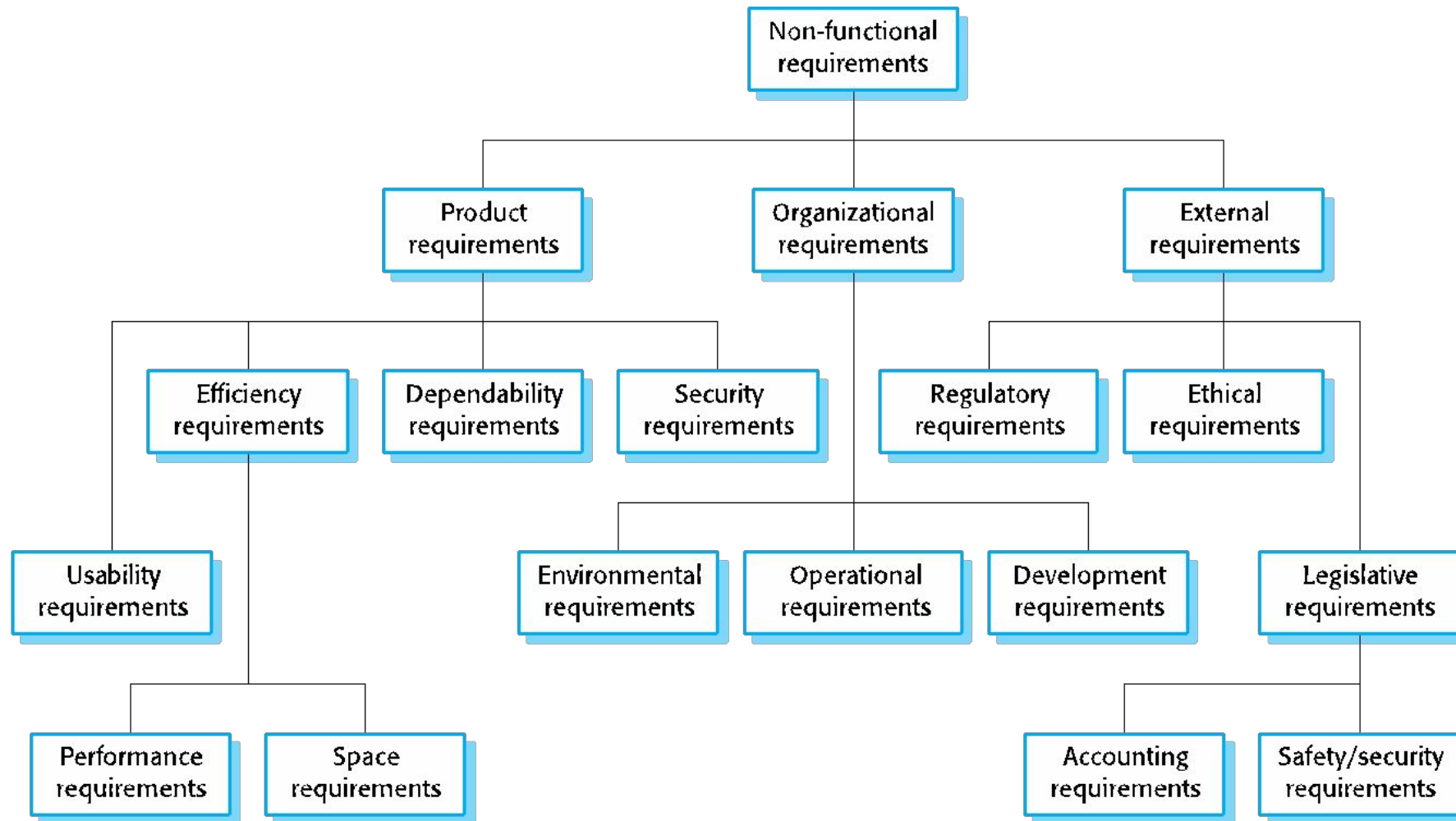
Requirements completeness and consistency

- In principle, requirements should be both complete and consistent.
- Complete
 - They should include descriptions of all facilities required.
- Consistent
 - There should be no conflicts or contradictions in the descriptions of the system facilities.
- In practice, because of system and environmental complexity, it is impossible to produce a complete and consistent requirements document.

Non-Functional Requirements

- These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified mandating a particular IDE, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.
- Often apply to the system as a whole rather than individual features or services.
- Examples:
 - The system should be available and responsive when needed, and should not experience frequent failures or crashes.
 - Usability: Usability requirements describe the ease of use and user-friendliness of the software system.

Types of Non-Functional Requirements



Non-Functional Requirements Implementation

- Non-functional requirements may affect the overall architecture of a system rather than the individual components.
 - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.

- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
 - It may also generate requirements that restrict existing requirements.

Non-Functional Classifications

□ Product requirements

- Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

□ Organisational requirements

- Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.

□ External requirements

- Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Metrics for specifying Non-Functional Requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Security Requirements

Introduction to Security Requirements Phase

- The Requirements Phase establishes *what* security capabilities a system must provide before design and implementation.
- **Why It Matters:** Finding security needs early reduces costly redesign and vulnerabilities.
- **Key Outputs:** Security requirements specification, risk assessment and threat modelling.

Security Risk Assessment (SRA)

- A structured process to identify, quantify, and prioritize security risks.
 - Identify Assets and Stakeholders
 - Identify potential threat and attackers
 - Analyze, Evaluate and categorize risk based on likelihood & Impact Assessment
 - Define mitigations
- Examples:
 - Asset: Customer Credit Card Data
 - Threat: Database injection attack
 - Likelihood: Medium; Impact: High → Risk Rating: High

Assets and Stakeholders

□ Identify Assets and Stakeholders

- Determine what holds value and for whom
- Value can be either tangible, such as economic benefits, or intangible, such as reputational or emotional significance
- Anything which has a value is categorized as asset.
- Anyone who owns asset is categorized as stakeholder
- There can be more than one stakeholders for a given asset
- One stakeholder may have multiple assets

Example: A patient medical data in a hospital database managed by hospital.

Medical data is an asset for patient because of privacy and

Medical data is asset for hospital because of legal and in turn financial implications.

Database is an asset for hospital because it has economical value(initial cost and non availability of database may impact hospital revenue)

Threat and Attackers

- Systematically identify attack paths and potential threat agents.
 - Any weakness which may compromise an asset is considered a potential threat
 - Any actor who may exploit existing adversary to gain something is considered as attackers
 - Gain can be tangible, such as immediate direct benefits (e.g., financial), or intangible (e.g., causing harm to someone).

- Steps involved
 - Visualize processes, data stores, trust boundaries.
 - Identify Actors & Assets: E.g., external user, internal admin, session tokens.
 - Enumerate Threats.
 - Rate Severity

Example: A patient medical data in a hospital database managed by hospital.

Unrestricted access to the database can be considered an existing weakness.

Potential threats include hospital employees (insider attackers), other patients (external attackers), and natural disasters such as a fire at the physical location of the database.

Additionally, a competing organization may act as an external attacker—not for direct tangible gain, but with the intent to corrupt the database and inflict financial damage on a competitor as a form of strategic sabotage.

Risk Categorization

□ Analyze, Evaluate and categorize risk

- Cost of weakness to be exploited
- Probability of weakness to be exploited
- Damage if weakness is exploited
- Risks are analyzed and categorized based on above points and severity is assigned to risk
 - Low Risk = High Cost of exploit, Low probability, low damage
 - Medium Risk = Medium cost of exploit, medium probability, medium damage
 - High risk = Low cost of exploit, high probability and high damage

Example : A patient medical data in a hospital database managed by hospital.

Insider attack because of unrestricted database = Low cost, high probability and high damage so High Risk

External attacker(other patient) = Medium cost, medium probability and medium damage so Medium risk

Security Objectives

- **Translating Risks into Goals:** Convert each high-priority risk into a measurable objective.
- **SMART Criteria:**
 - **Specific:** Clearly state outcome (e.g., encrypt all Personally Identifiable information (PII) in transit).
 - **Measurable:** Define metrics (e.g., 256-bit AES, TLS v1.2+).
 - **Achievable:** Align with team capabilities.
 - **Relevant:** Directly mitigates an identified threat.
 - **Time-bound:** Milestone before design review.
- **Example Objectives:**
 - Encrypt database backups with AES-256 by Q3.
 - Implement brute-force protection limiting login attempts to 5 per hour.

Example :: A patient medical data in a hospital database managed by hospital.

Insider attack risk is almost nil considering doctors trusted

External attack (other patient) risk is almost nil as access protection will not allow database access.

Formulating Clear Security Requirements

- SRA, security Objectives and threat modeling will lead to clearly articulated security requirements for the product

- These are part of the non functional requirements
 - **Use the Risk Analysis output** to determine high-priority threats.
 - **Review Threat Models** to identify vulnerable points and potential attack paths.
 - **Apply Security Objectives** to ensure each critical area is protected by measurable goals.
 - **Draft and Refine Requirements** that are clear, testable, and traceable to specific risks and objectives.

Requirements Analysis

Process of Requirements Analysis...

□ User requirements

- Statements in natural language plus informal context diagrams, system/sub-system and their interconnections and operational constraints. Written for/by customers.

An example:

Screen A accepts production information, including Lot, Product Number, and Date. System B produces the Lab Summary Report. Twenty users can use System C concurrently without noticeable system delays.

□ System requirements

- A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented; so may be part of a contract between client and contractor.

Process of Requirements Analysis...

- User requirements describe what the user needs and wants from the system, while system requirements describe the technical specifications and constraints that the system must meet to fulfill those user requirements.
- Understanding the difference between the two can help ensure that the system meets the needs of its users while also being technically feasible and efficient.
- Domain requirement
 - Constraints on the system from the domain of operations.

An example: (Healthcare related)

The software must be developed in accordance with IEC 60601 standard regarding the basic safety and performance for medical electrical equipment.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1 – Lecture 9-10

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Requirements Engineering: Software Requirement Specification, RTM

Requirement Specification

- After elicitation and analysis, we need to specify the requirements.
- **Requirements specification** is the documentation of a set of requirements that is reviewed and approved by the customer and provides direction for the software construction activities in the next stage of the life cycle.
- The **software requirements specification (SRS)** document is the basis for customers and contractors/suppliers agreeing on what the product will and will not do. It describes both the functional and nonfunctional requirements.
- IEEE defines software requirements specification as, 'a document that clearly and precisely describes each of the essential requirements (functions, performance, design constraints and quality attributes) of the software and the external interfaces.
- A standard template can be found [here](#).

Software Requirement Specification (SRS)



Functionality:

- What is the software supposed to do?



External interfaces:

- How does the software interact with people, the system's hardware, other hardware, and other software?



Non-Functionality:

- This includes all of the Quality criteria which drive the functionality. Example: Performance, Availability, Portability etc.



Design constraints imposed on an implementation:

- Required standards in effect
- Implementation language
- Policies for database integrity
- Resource limits
- Security
- Operating environment(s) etc.

An SRS Template <<to be updated>>

ToC recommended by IEEE for SRS (IEEE Std. 830-1998)

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, acronyms, and abbreviations
- 1.4 References
- 1.5 Overview

2. Overall description

- 2.1 Product perspective
- 2.2 Product functions
- 2.3 User characteristics
- 2.4 Constraints
- 2.5 Assumptions and dependencies

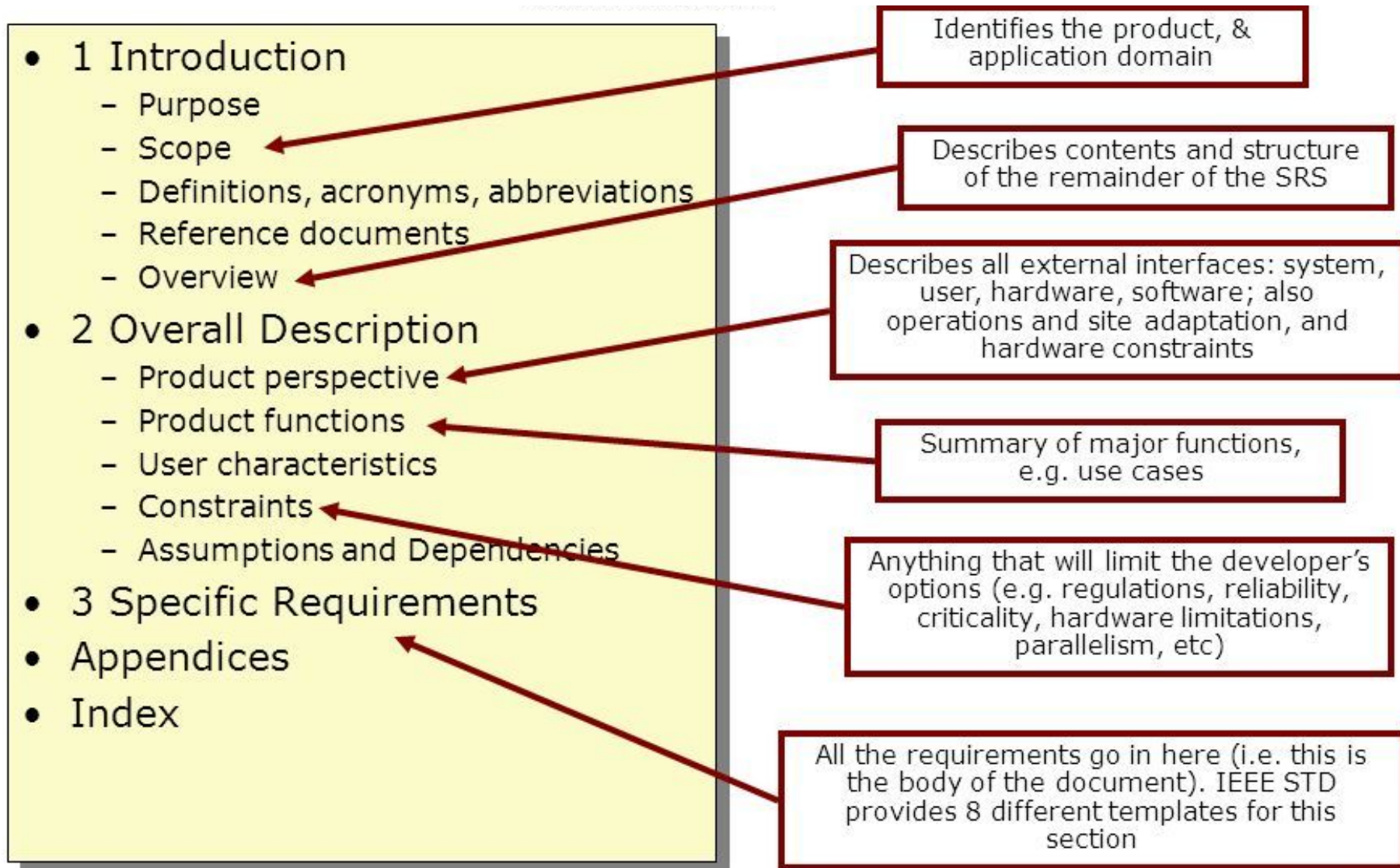
3. Specific requirements

- 3.1 External Interface
- 3.2 Functional Requirements
- 3.3 Non-Functional Requirements
- 3.4 Design Constraints

Appendixes

Index

An SRS Template



Requirements Validation

- The purpose of requirements validation is to help ensure that the requirements do what the customer wants. This is an important phase because repairing requirement errors in downstream phases can be expensive.

□ **VALIDATION & VERIFICATION**

- Validation determines whether the software requirements if implemented, will solve the right problem and satisfy the intended user needs.
- Verification determines whether the requirements have been specified correctly

(Reviews are used for both validation and verification)

Requirement Reviews



Requirements Validation

□ Prototyping

- Prototype facilitates user involvement during requirements engineering phase and ensures engineers and users have the same interpretation of the requirements.
- Prototyping is most beneficial in systems – With many user interactions

Example: Design of online billing systems

- Systems with little or no user interaction may not benefit as much from prototyping.

Example: Batch processing

□ Model Validation

- Ensuring that the models represent all essential functional requirements
- Demonstrating that each model is consistent in itself
- Usage of the Fish Bone Analysis technique for validation

□ Acceptance Criteria

- To check if there are requirements matching with that the Acceptance criteria..

Requirements Management

- Requirements specification is the baseline on which the future lifecycle phases will need to build upon.

- Can you think of reasons why requirements might change?
 - Better understanding of the problem
 - Customer internalizing the problem and solution
 - Evolving environment and technology landscape

- **Facets of Requirements Management**
 - Ensuring that the requirements are all addressed in each phase of the lifecycle
 - Ensuring that the changes in the requirements are handled appropriately

Requirements Traceability Matrix (RTM)

- Requirements are traced across the SDLC using the requirement traceability matrix (RTM)
 - Forward Tracing
 - Backward Tracing
- Every phase of the SDLC progressively fills the RTM

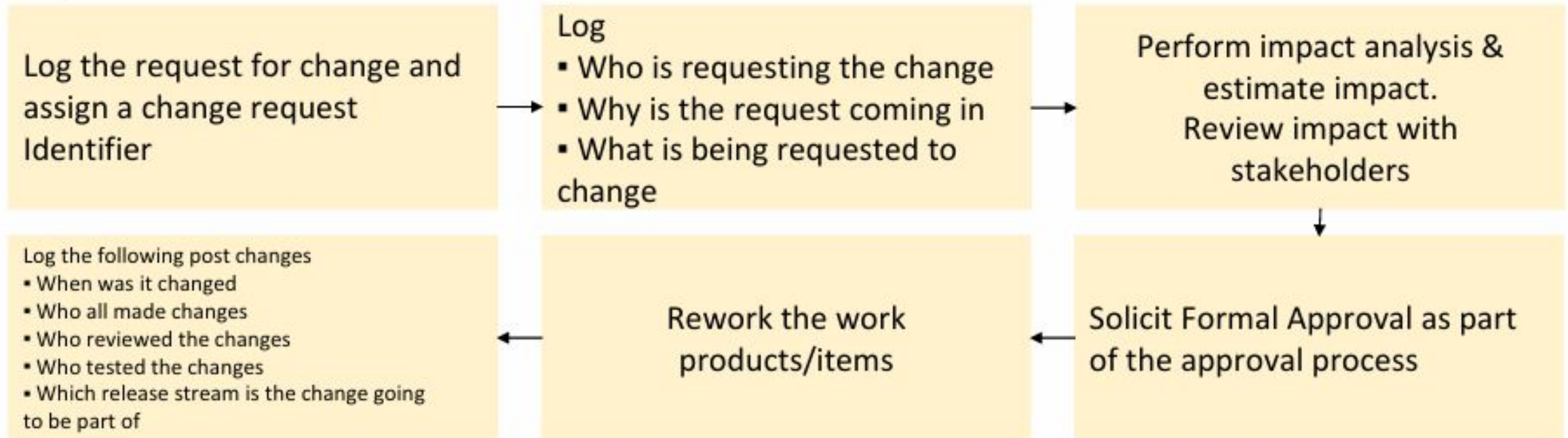
Req ID	Architectural Section	Design Section	File/Implementation	Unit Test ID	Functional Test ID	System Test ID	Acceptance Test ID

Requirements Change Management

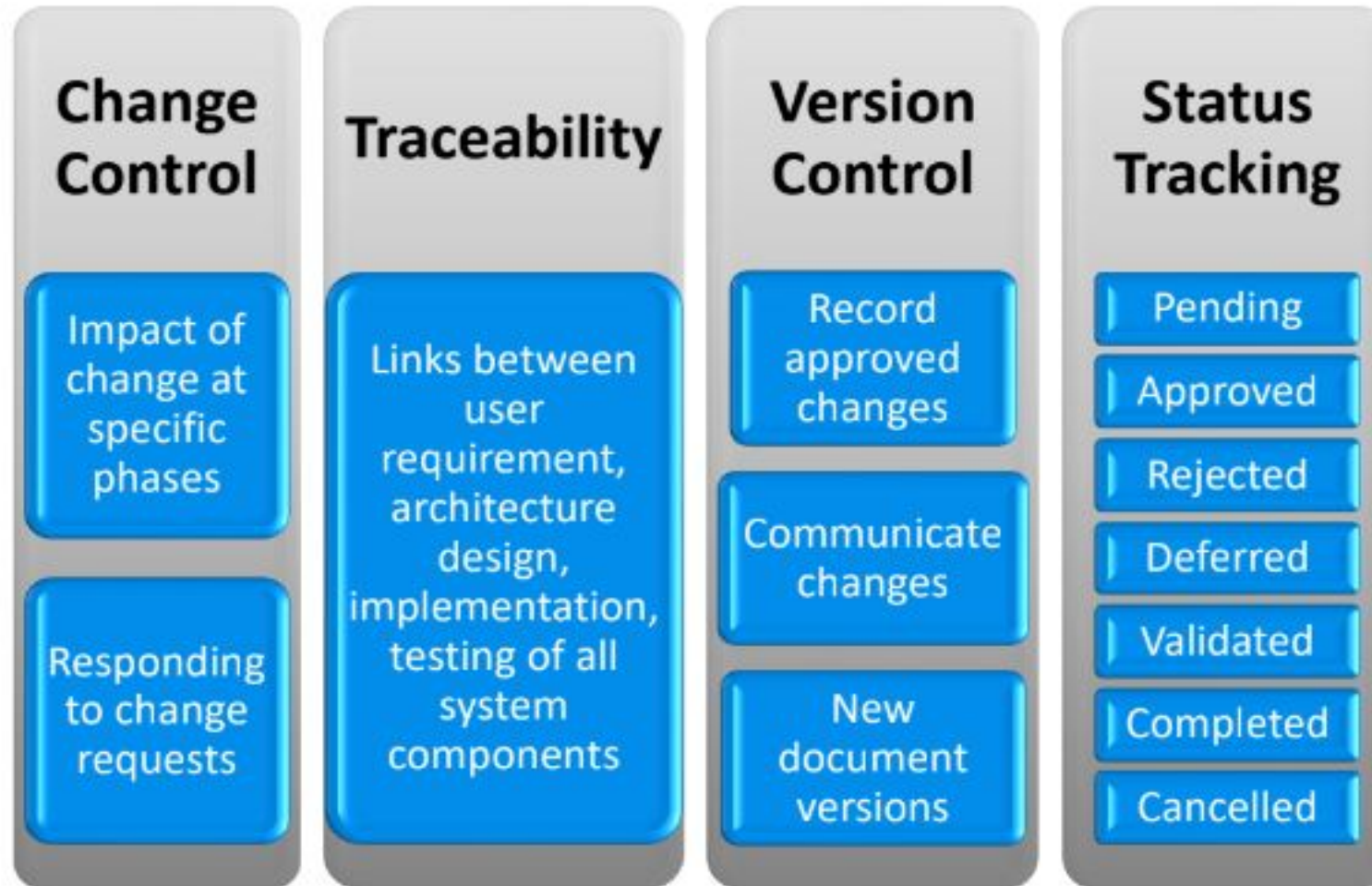
- ❑ Change in the requirements have impacts on plans, work products etc.
- ❑ Uncontrolled changes can have a huge adverse impact on project in terms of cost, schedule, quality and expectations.
- ❑ In the perspective of managing the changes, change requests go through a formal change management process.

Requirements Change Management

REQUIREMENT CHANGE PROCESS



Requirements Change Management



Agile Methods and Requirements

- Many agile methods argue that producing detailed system requirements is a waste of time as requirements change so quickly.
- The requirements document is therefore always out of date.
- Agile methods usually use incremental requirements engineering and may express requirements as “user stories”.
- This is practical for business systems but problematic for systems that require pre-delivery analysis (e.g. critical systems) or systems developed by several teams.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1 – Lecture 11

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

UML and Use Cases

Unified Modeling Language (UML)

□ What is UML?

- The Unified Modeling Language has become the de-facto standard for building Object-Oriented software.

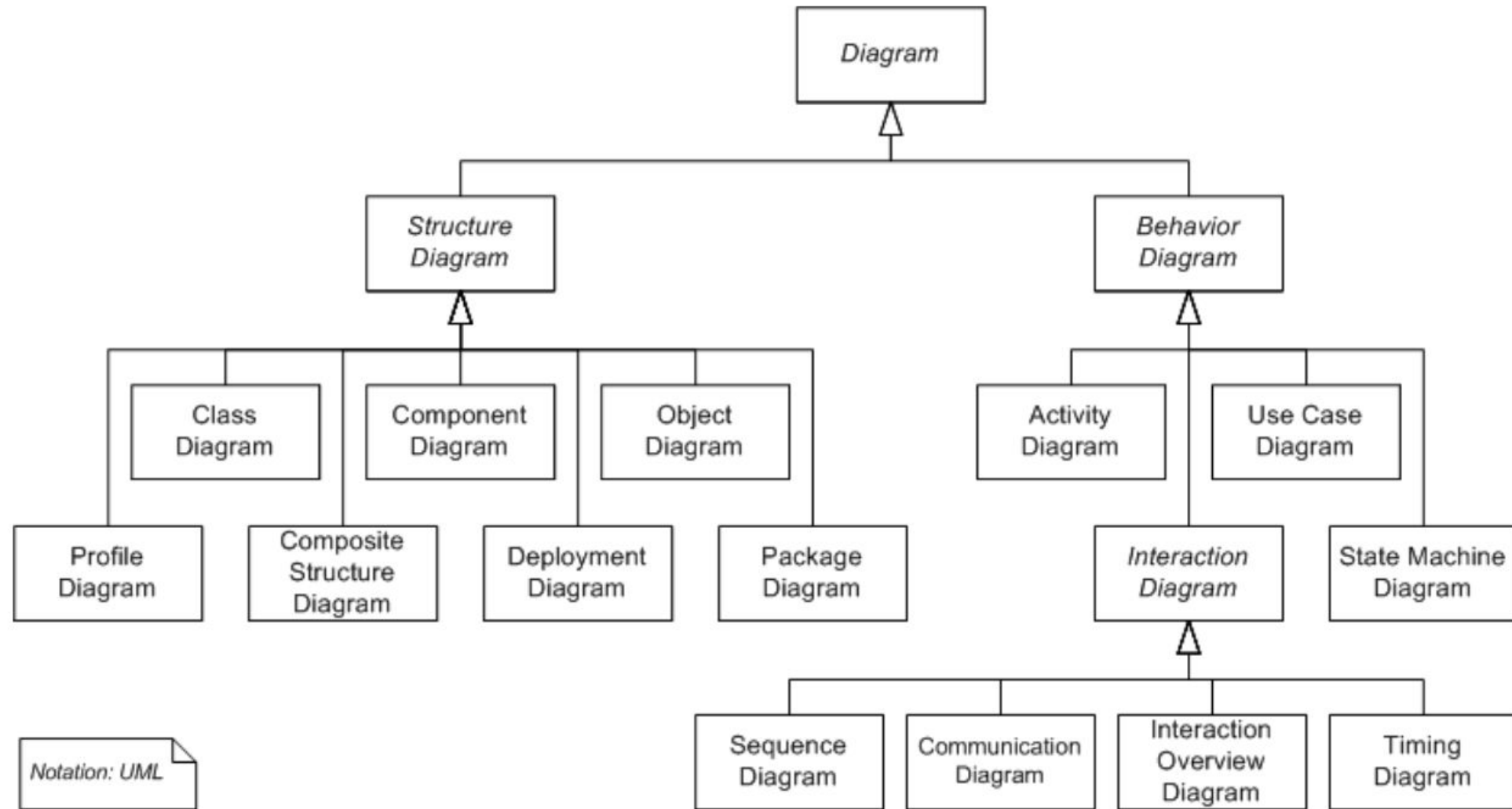
The **OMG** specification states: *"The Unified Modeling Language (UML) is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML offers a standard way to write a system's blueprints, including conceptual things such as business processes and system functions as well as concrete things such as programming language statements, database schemas, and reusable software components."*

UML plays an important role in defining different perspectives of a system

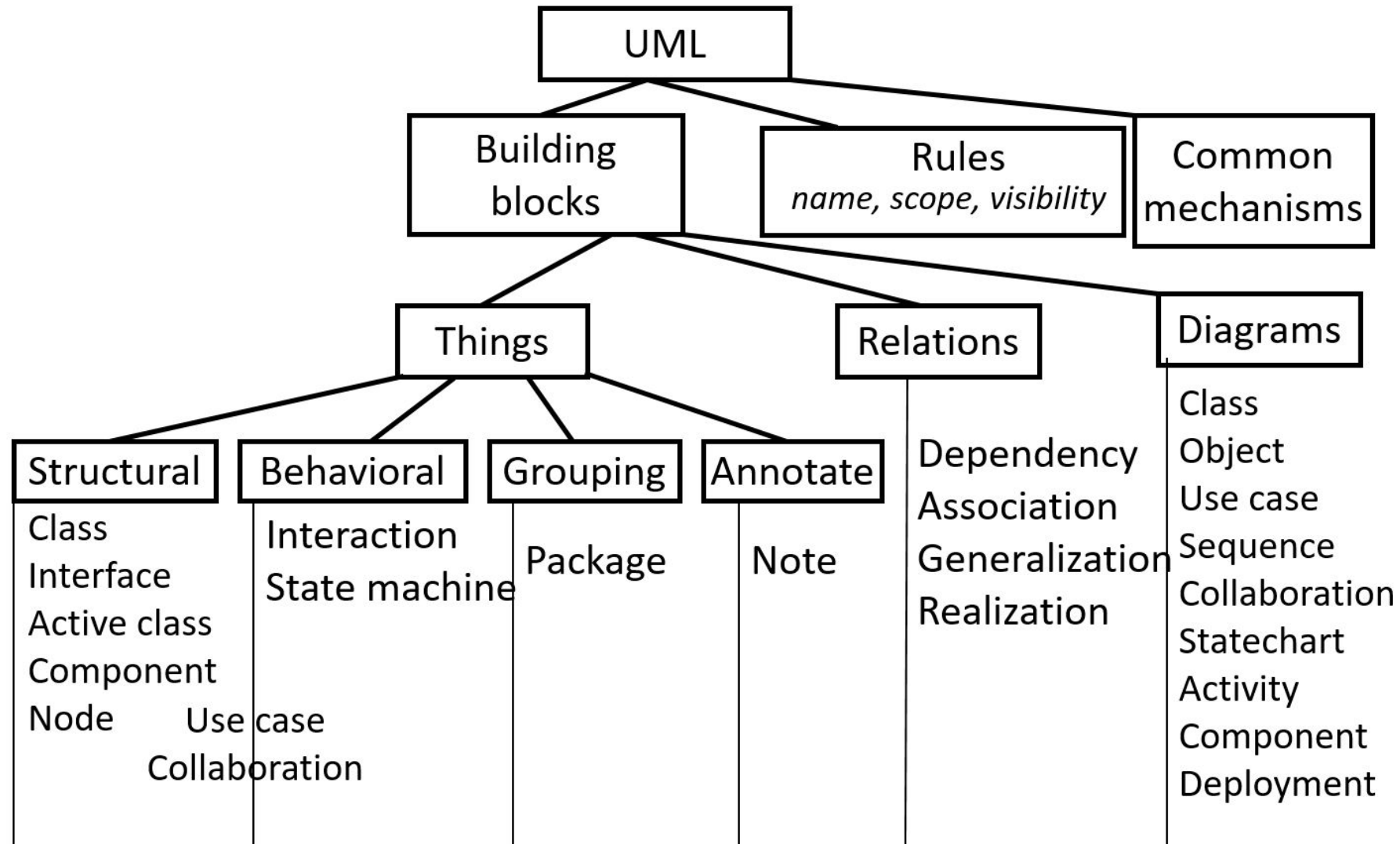
Requirements Design Implementation, Deployment

□ Why is UML used?

- UML Use-case models are predominantly used with Modeling Systems, to discuss the dynamic behavior of the system when it is running/operating. It is often used to used to gather the requirements of a system including internal and external influences.



Conceptual UML model



Using Use Case Diagram

- **Use case** from a user's point of view outlines how the proposed system will perform a task expected to be performed, while responding to a request or task of a role/actor/user.
- **Use case diagrams** are used to visualize, specify, construct, and document the (intended) behavior of the system, during requirements capture and analysis. Used by developers, domain experts and end-users.

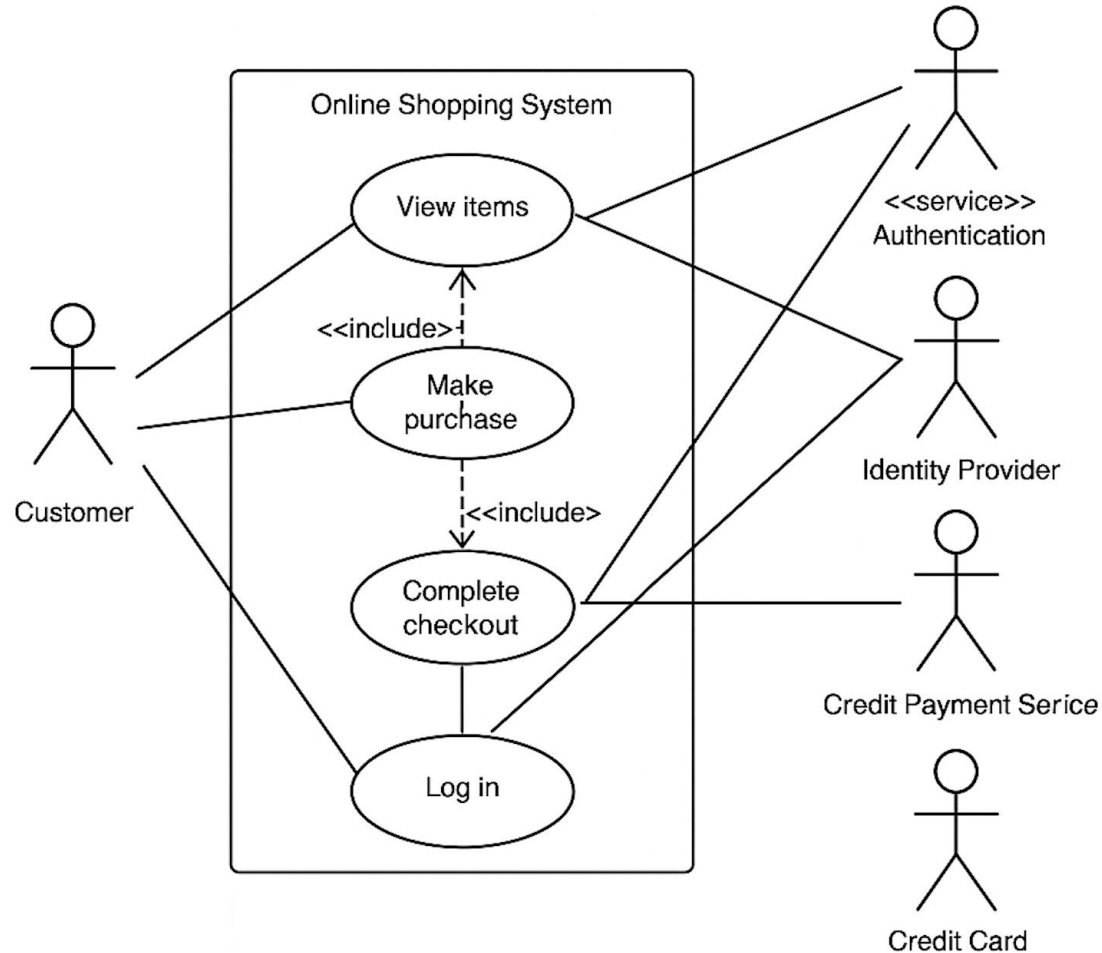
What is Use Case Modelling?

- Describes the **interaction of users and the system**
- Describes **what functionality does a system provides to its users.**
- Use case model has **two important elements - actors and use cases**
- **Actor/s:** One or set of objects who directly interacts with the system.
Every actor has a defined purpose while interacting with the system. An actor can be a person, device or another system.
- **Use case:** A piece of functionality that a system offers to its users.
- Set of all use cases defines the entire functionality of the system. Also define the error conditions that may occur while interacting with the system

How to Create Use Case Diagram

- List main system functions (use cases) in a column
- Draw ovals around the function labels
- Draw system boundary
- Draw actors and connect them with use cases
- Specify include and extend relationships between use cases
 - **"Include"** is used when a use case **always calls another use case** as part of its behavior (e.g., *"Place Order"* always **includes** *"Process Payment"*).
 - **"Extend"** is used when a use case **optionally adds behaviour** to another (e.g., *"Make Payment"* can be **extended** by *"Apply Discount"*, if applicable).
- Use **include** for reuse, and **extend** for optional or conditional behaviour.

A sample Use Case Diagram



Use case diagram symbols and notation

Use cases: Horizontally placed ovals that represent the different functionalities that a user might have.

Actors: Stick figures that represent the people actually employing the use cases.

Associations: A line between actors and use cases.

Use Case Flow

- A use case is an example behaviour of the system
- Written from an actor's point of view, not the system's
- 3-9 clearly written steps (flow) lead to a “main success scenario”
- Also used to describe “variation” and “exception” scenarios

- Use cases are hugely valuable
 - Capture a level of functionality (list of goals)
 - Establish an understanding between the customer and the developers of the requirements (success scenarios)
 - Alert developers of variations (extensions) and exceptions (errors) cases to test

Sample Use Case Flow

Goal	Customer Checkout
Actor	Customer with help from other actors
Main flow	Customer selects to complete checkout. - The system requests the customer to log in. - The customer provides credentials. - The system invokes the Authentication service to verify credentials. - The Authentication service delegates to an Identity Provider for validation. - Upon successful authentication, the customer proceeds. - The system displays payment options (e.g., credit card). - The customer selects Credit Card as the payment method. - The system contacts the Credit Payment Service for authorization. - Payment is successfully authorized. - The system confirms the order and shows an order summary. - Checkout process is completed.
Alternate Flow	Invalid Login Credentials At Step 4, if authentication fails: - System displays an error message and asks the user to retry. - Customer may retry login or cancel checkout.

Use Case

These other/alternate flows are hugely valuable

□ Do's:

- Think about how every step of the use case could be **enhanced or fail**
- Give a plausible response to each extension from the system
- Response should either jump to another step of the case, or end it

□ Don'ts:

- List things outside the scope of the use case (“User’s power goes out”)
- Make unreasonable assumptions (“DB will never fail”)
- List a remedy that your system can’t actually implement
- Go overboard

4 steps for writing a use case

1. Identify actors and their goals

- Actors:
What users and (sub)systems interact with our system?
- Goals:
What does each actor need our system to do?
- Trigger:
What kicks off the interaction with the system

4 steps for writing a use case

1. Identify actors and goals
2. **For each goal, identify what each actor needs the system to do**
 - Main success scenario is the preferred "happy path"
 - Easiest to read and understand
 - Capture each actor's intent and responsibility, from trigger to goal
 - State what information passes between actor(s) and system
 - Number each step (line)

4 steps for writing a use case

1. Identify actors and goals
2. For each goal, identify what each actor needs the system to do
3. **List the variations to the main (success) flow**
 - These are alternate branches from the main path
 - What are some options/enhancements that the user might want/expect
 - Label with step number (success scenario line)

Example variation to step 5:

5.1 <variation>; 5.1 <steps>; 5.1<continue at step 6>

4 steps for writing a use case

1. Identify actors and goals
2. For each goal, identify what each actor needs the system to do
3. List the variations to the main flow
4. **List the exceptions (error) flow**
 - Many steps can fail
 - Describe failure-handling
 - Label with step number (success scenario line)
5.1<failure condition>; 5.1 <actions>; 5.1<continue at failure step 7>

Persona and User Scenarios

- What are techniques used to specify requirements?
 - UML and Use cases
 - **Personas and user scenarios**
 - Storyboarding
 - Paper prototyping
 - Prototyping
 - Feature list

- A **persona** is a fictional yet realistic representation of a user archetype who might use your system. It embodies key characteristics, needs, goals, and pain points of real users.
- **Why Personas?**
 - Ground the design in real user behavior.
 - Help teams empathize with users.
 - Drive user-centered requirements.
- **Example Use Case: Library Catalog System**
 - **Persona A:** First-year undergraduate student
 - **Persona B:** Librarian managing inter-library loans
 - **Persona C:** Faculty member planning course

User Scenarios (User Stories)

- A user scenario is a **narrative** that describes how a persona might interact with the system in a particular situation.
- **Structure Example (similar to user stories):**
 - "As a [persona], I want to [goal] so that [benefit]."
- **Example:**
 - “As a graduate student, I want to filter research articles by citation count so that I can quickly find the most impactful sources.”
- Scenarios help identify missing flows, UI features, and exception cases.

Requirements Discovery

Requirements Discovery

- The process of gathering information about the required and existing systems and distilling the user and system requirements from this information.
- Interaction is with system stakeholders from managers to external regulators.
- Systems normally have a range of stakeholders.

- Formal or informal interviews with stakeholders are part of most RE processes.
- Types of interview
 - Closed interviews based on pre-determined list of questions
 - Open interviews where various issues are explored with stakeholders.
- Effective interviewing
 - Be open-minded, avoid pre-conceived ideas about the requirements and are willing to listen to stakeholders.
 - Prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system.

Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviewers need to be open-minded without pre-conceived ideas of what the system should do
- You need to prompt the use to talk about the system by suggesting requirements rather than simply asking them what they want.

Problems with Interviews

- Application specialists may use language to describe their work that isn't easy for the requirements engineer to understand.

- Interviews are not good for understanding domain requirements
 - Requirements engineers cannot understand specific domain terminology;
 - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

- A social scientist spends a considerable time observing and analysing how people actually work.
- People do not have to explain or articulate their work.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Example: How Flipkart Uses Ethnography for Last-Mile Delivery

□ Problem Statement:

- Flipkart needed to understand why delivery drivers in India faced delays despite optimized routes.
- Traditional data analytics (GPS tracking, delivery times) didn't explain the bottlenecks.

Ethnographic Approach

□ Field Observations

- Researchers rode along with delivery drivers in cities like Bangalore and Mumbai.
- They observed real-world challenges:
 - Traffic & Road Conditions: Unpredictable jams, narrow lanes, and poor addressing systems.
 - Customer Behaviors: Many recipients weren't home, demanded cash-on-delivery (COD) verification, or lived in gated communities with security delays.
 - Driver Workarounds: Drivers developed informal strategies (e.g., calling customers in advance, leaving packages with local shopkeepers).

□ Interviews with Stakeholders

- Spoke with drivers, warehouse managers, and customers to understand pain points.
- Discovered that COD payments caused delays because drivers had to count cash and wait for verification.

□ Artifact Analysis

- Reviewed delivery logs, GPS deviations, and customer feedback to cross-validate findings.

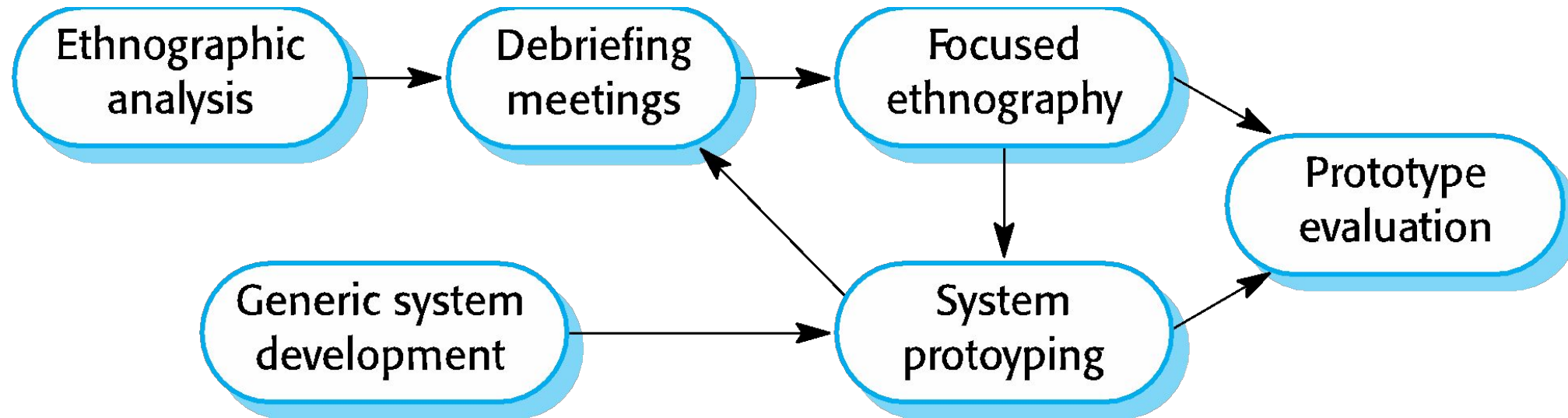
Outcome & Solutions

- ❑ **Dynamic Routing Adjustments:** Flipkart updated algorithms to account for local traffic patterns (e.g., avoiding school zones at pickup times).
- ❑ **Cashless COD:** Introduced digital payment confirmations to reduce cash-handling delays.
- ❑ **Micro-Fulfillment Centers:** Placed small warehouses closer to dense urban areas based on where drivers faced the most delays.
- ❑ **Local Partnerships:** Used kirana (small local stores) as pickup points in hard-to-reach areas.

Key Takeaways

- Data Alone Isn't Enough – GPS logs don't show why drivers take detours.
- Human Insights Drive Innovation – Ethnography reveals workarounds that can be formalized (e.g., digital COD).
- Hyper-Local Solutions Matter – What works in Mumbai may fail in Manila

Ethnography and Prototyping for Requirements Analysis





Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Course Coordinator & Anchor (RR Campus): Vadiraja Acharya
Course Anchor (EC Campus): Prof. M Sheela Devi

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, CMU, WSU class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Unit 1: Lecture 14

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Testing and Test Planning

Software Testing

- Software testing is a process of examining the functionality and behavior of the software through verification and validation.
- Verification and Validation is the process of investigating that a software system satisfies specifications and standards and it fulfills the required purpose.
- Barry Boehm described verification and validation as the following:
 - Verification: Are we building the product right?
 - Validation: Are we building the right product?

Software Testing

- Verification is a process of determining if the software is designed and developed as per the specified requirements.
- Validation is the process of checking if the software (end product) has met the client's true needs and expectations.
- Software testing is incomplete until it undergoes verification and validation processes. Verification and validation are the main elements of software testing workflow because they:
 1. Ensure that the end product meets the design requirements.
 2. Reduce the chances of defects and product failure.
 3. Ensure that the product meets the quality standards and expectations of all stakeholders involved.
- According to a survey, the software testing industry is estimated to grow from \$48 billion in 2025 to \$93 billion in 2030.

Software Testing

- Software testing also involves measuring attributes to build confidence in the software.
- **“Testing software shows only the presence of errors, not their absence”**

Software Testing

What are the real objectives of testing?

Demonstration, Detection and Prevention

□ Demonstration

- System used with acceptable risk
- Functions under special conditions
- Product ready for integration/use

Software Testing

What are the real objectives of testing?

Demonstration, **Detection** and Prevention

□ Detection

- Discover defects, errors and deficiencies
- Determine capabilities and limitations
- Quality of components

Software Testing

What are the real objectives of testing?

Demonstration, Detection and **Prevention**



Prevention

- Information to prevent/reduce errors
- Reduce error propagation
- Clarify System specifications and performance
- Identify ways to avoid risks and problems.

Are we building the product right?

- ❑ Process of checking that software achieves its goals to ensure that products and deliverables meet requirements.
- ❑ Static Testing - Does not include execution of code
- ❑ Checking documents, design, code (reviews, walkthroughs, inspections)
- ❑ Finds the bugs early in development
- ❑ Targets software architecture, design, database, etc.
- ❑ Occurs before validation

Are we building the right product?

- Focuses on product related activities that determine if the system or project deliverables meet customer/client expectations
- Dynamic Testing - Includes execution of code
- Validates the capabilities and features in project scope and requirements
- Typically done by testing team
- Methods: Black Box testing, White Box testing, and non-functional Testing.

Verification and Validation

	Verification	Validation
Definition	It is a process of checking if a product is developed as per the specifications.	It is a process of ensuring that the product meets the needs and expectations of stakeholders.
What it tests or checks for	It tests the requirements, architecture, design, and code of the software product.	It tests the usability, functionalities, and reliability of the end product.
Coding requirement	It does not require executing the code.	It emphasizes executing the code to test the usability and functionality of the end product.
Activities include	A few activities involved in verification testing are requirements verification, design verification, and code verification.	The commonly-used validation activities in software testing are usability testing, performance testing, system testing, security testing, and functionality testing.

Verification and Validation

	Verification	Validation
Types of testing methods	A few verification methods are inspection, code review, desk-checking, and walkthroughs.	A few widely-used validation methods are black box testing, white box testing, integration testing, and acceptance testing.
Teams or persons involved	The quality assurance (QA) team would be engaged in the verification process.	The software testing team along with the QA team would be engaged in the validation process.
Target of test	It targets internal aspects such as requirements, design, software architecture, database, and code.	It targets the end product that is ready to be deployed.

- Different project management and software development methods use verification and validation in different ways.
- For instance, both verification and validation happen simultaneously in agile development methodology due to the need for continuous refinement of the system based on the end-user feedback.

Testing related terminology

- ❑ Testing is the process of identifying defects, where a defect is any variance between actual and expected results.
- ❑ “A mistake in coding is called Error, an error found by a tester is called a Defect, a defect accepted by development team is called a Bug and if a build does not meet the requirements, then it is Failure.”
- ❑ **Defect:**It can be simply defined as a variance between expected and actual. The defect is an error found AFTER the application goes into production.
 - It commonly refers to several troubles with the software products, with their external behavior or with its internal features.
 - In other words, a Defect is a difference between expected and actual results in the context of testing.
 - It is the deviation from the customer requirement.

Testing related Terminology

- **Error:** An error is a mistake, misconception, or misunderstanding on the part of a software developer.
 - In the category of the developer, we include software engineers, programmers, analysts, and testers.
 - For example, a developer may misunderstand a design notation, or a programmer might type a variable name incorrectly – leads to an Error.
 - It is the one that is generated because of the wrong login, loop or syntax.
 - The error normally arises in software; it leads to a change in the functionality of the program.

- **Bug:** A bug is the result of a coding error. An Error found in the development environment before the product is shipped to the customer.
 - A programming error that causes a program to work poorly, produce incorrect results or crash.
 - An error in software or hardware that causes a program to malfunction. A bug is the terminology of Tester.

Testing related Terminology

- ❑ **Failure:** A failure is the inability of a software system or component to perform its required functions within specified performance requirements. When a defect reaches the end customer it is called a Failure. During development, Failures are usually observed by testers.

- ❑ **Fault:** An incorrect step, process or data definition in a computer program that causes the program to perform in an unintended or unanticipated manner. A fault is introduced into the software as the result of an error. It is an anomaly in the software that may cause it to behave incorrectly, and not according to its specification. It is the result of the error.

- ❑ **Issue:** Raised by end user when product doesn't meet expectations.

Characterizing testing

- There are different types of testing (will discuss later).
- How do we characterize these?

Why?

- Observations based on test objective
- Fault identification, Fitness for use

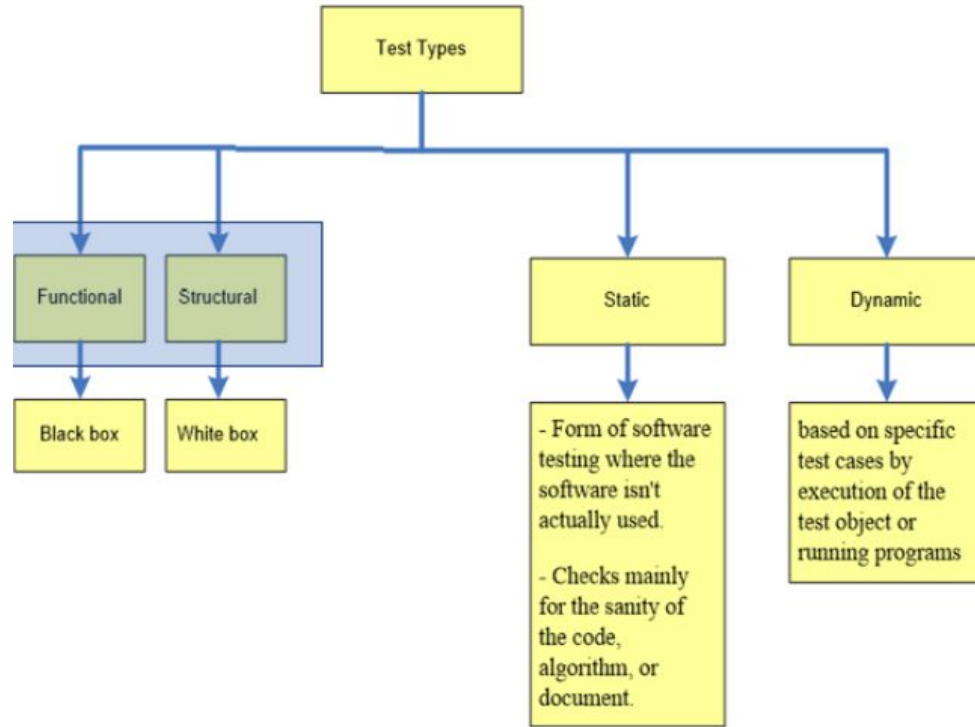
How did we arrive at Test Cases ?

- Based on test strategy or approach

Which of the samples should we observe based on test selection ?

- Random, Ad-Hoc, Algorithmic, Statistical

Broad categorization



Levels (Lifecycle) of testing Types

- UNIT testing
- Integration teting
- System/Functional testing
- Acceptance Testing

Technique based testing type

- Coverage based testing
- Fault based testing

Manual

Automatic

Characterizing testing

☐ What is a test case?

Test cases define how to test a system, software or an application. A test case is a singular set of actions or instructions for a tester to perform that validates a specific aspect of a product or application functionality. If the test fails, the result might be a software defect that the organization can triage.

- ☐ A tester or QA professional typically writes test cases, which are run after the completion of a feature or the group of features that make up the release. Test cases also confirm whether the product meets its software requirements.
- ☐ A group of test cases is organized in a test suite, which tests a logical segment of the application, such as a specific feature.

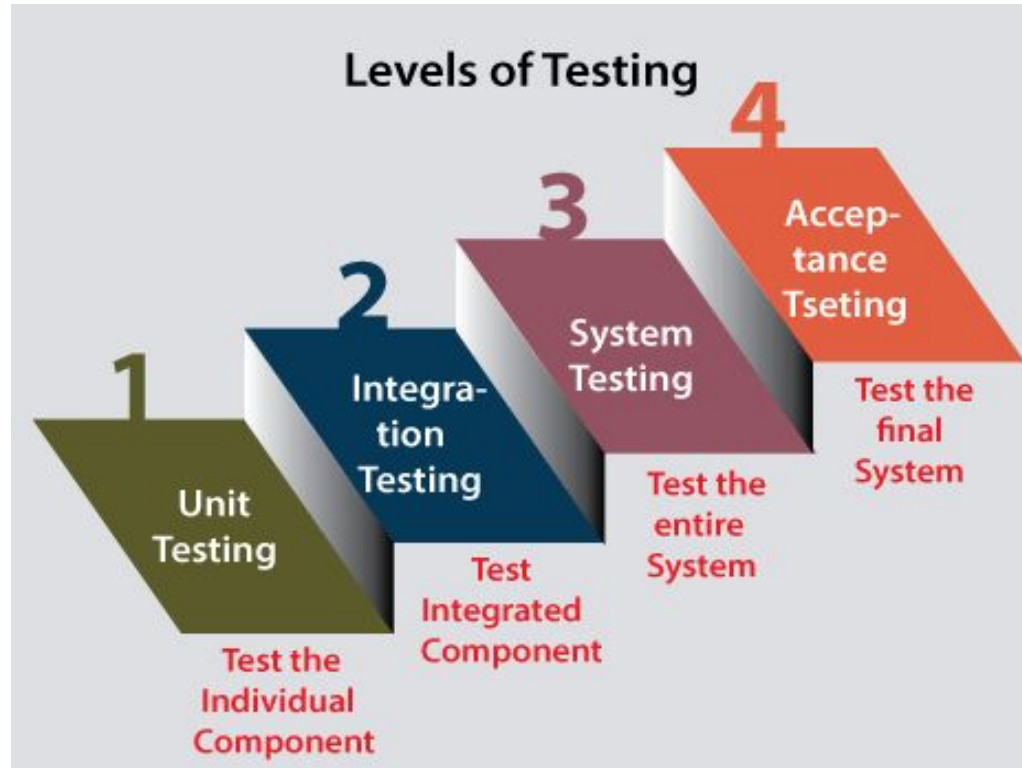
Test case format

- Test case documentation typically includes all the pertinent information to run and collect data from the test. While the specific test case format might differ between organizations, most include the following details:
- **Module name.** This is the module or feature under test.
- **Test ID and/or name.** This is a unique identifier that should follow a standard naming convention.
- **Tester name.** The person conducting the test.
- **Test data.** This describes the dataset(s) to use for the test.
- **Assumptions or preconditions.** Describe the various steps that must be accomplished prior to testing, or what we can assume situationally about the test, such as “after a successful login.”

Characterizing testing

- ❑ **Test priority.** Define whether the test is low, medium or high priority.
- ❑ **Test scenarios.** This is the high-level action from which the test case derives.
- ❑ **Testing environment.** Identify the name and/or characteristics of the environment for testing.
- ❑ **Testing steps.** Detail the steps for the tester to follow in the desired order.
- ❑ **Expected results.** This is the output you expect to receive from the system.
- ❑ **Actual results.** This is the output you actually receive from the system.
- ❑ **Pass/fail determination.** If the actual results match the expected results, the test passes. If not, the test fails.

Levels of Testing



Levels of Testing

- ☐ Unit Testing
 - Verifies proper functioning of the individual unit.

- ☐ Integration testing
 - Focuses on finding interface errors between components
 - Bugs that are not identified during unit testing

- ☐ System Testing
 - Assessment of the entire system behavior
 - Information helps direct product release
 - Discover bugs that cannot be attributed to single component

- ☐ Acceptance Testing
 - Done by system providers/users/customers
 - Determines if system meets the needs

Test planning generics

What is Software Test Planning?

- Software test planning is the process of evolving a test plan which discusses **what**, **when**, **how much** and **how** testing has to be done to ensure quality expectations can be met.
 - The outcome also serves as a blueprint to conduct software testing activities as a defined process
 - Developers, business managers and customers can understand the details of testing
 - This plan is also to be used for monitoring and control.

Test planning process overview

1. Ensuring Context and Scope of the project is understood
2. Establish test adequacy criteria
3. Evolve a test strategy which will be followed
4. Evolving list of deliverables
5. Creation of detailed test schedule
6. Planning, Identification and Allocation of resources
7. Identification of milestones
8. Risk Management
9. Establishment of measures and metrics

- Understanding the context where the product is going to be used by:
 - Reviewing use case scenario in the product deployment environment
 - Discussing with the designer
 - Discussing with the developer
 - Reviewing project/product documentation
 - Play around the product or perform product walk-through

- Optimal amount of testing is based on :
 - Customer requirement
 - Project schedule
 - Project budget
 - Product specification
 - Skills and talent of test team

Test Adequacy

- Testing of the subset of possible combinations does not guarantee absence of issues
- Fixing issues found during testing may not be feasible due to
 - Volume or number of errors
 - Schedule
 - Resources
 - Some issues are enhancements
- Test Adequacy Criteria: Criteria to determine when to stop testing or consider testing complete for that iteration
 - Examples
 - Of the planned tests, x% of lines are executed and y% of branches
 - All planned test cases are complete with no critically high priority issues
 - Total number of severe defects is less than 5

Testing Strategy

- Known as the test approach and defines how testing is carried out and deals with following
 - Testing mindset or test models to be followed
 - Test types which will be used
 - Test environment
 - Automation Strategy
 - Tools
 - Risk analysis with contingency planning

Testing Strategy – Models/Mindsets



Demonstration

- Make sure software runs and solves the problem
- If software passes all tests, establishes satisfaction of specs
- Might only test what succeeds



Preventive

- Prevents faults in early phases through careful planning and design
- Reviews and Test Driven Development



Destruction

- Try and make the software fail to find as many faults
- Good and effective test cases find faults
- Difficult to decide when to stop testing



Evaluation

- Detects faults through the lifecycle phases
- Focuses on analysis and review techniques to detect faults in requirements and design documents

Testing Strategy – Testing Types Chosen

Each lifecycle phase has outcomes which can be tested

- Feasibility phase has acceptance test cases
- Requirements phase has requirements specification
- May have functional and system test cases
- Architecture/Design phase has refined functional and system cases and integration test cases
- Implementation phase has code unit test cases
- Testing phases would involve reviews and execution of test cases
- Maintenance phase would involve review and execution of regression and other tests

Strategies in terms of

- Begin by “testing-in-the-small” and move towards “testing-in-the-large”
- Top Down and Bottom up
- Positive and Negative testing
- Dynamic and Heuristics based approach

Testing Strategy – Test Execution Environment

- ☐ Testing Environment/ Test Execution Environment/ Test Bed: setup of software and hardware for testing teams to execute
 - Configured as per need of the application
 - Could include test data, Database, front end, Operating System, Servers, Storage, and Network
- ☐ Correct setup ensures success of testing or results in delay, cost escalations and so on
- ☐ Environment management involves maintenance and upkeep of test bed, monitoring and modifying components
- ☐ Challenges in terms of planning resource usage, remote environment, setup time, sharing of environment and setup of complex configurations.

Testing Strategy – Automation

- ☐ Defining goals
- ☐ Planning the test approach
- ☐ Selection of Automation framework
- ☐ Selecting Test Tool
- ☐ Test Case Design and Execution
- ☐ Test Result Generation and Analysis
- ☐ Maintaining script

Testing Strategy – Tools which will be used

- ☐ The technology of the Software or application under test (AUT) will need to be compatible to the tools and drives the selection of it.
- ☐ Testers will need to be comfortable with the tool, else it may not be effectively used
- ☐ Tools chosen need to be balanced in terms of the features offered, ability to generate reports/data needed for different stakeholders and the ease of use.
- ☐ Cross platform support is an expectation as automated tests would/may need to run on different platforms.
- ☐ Acceptability/Popularity/prevalence of the tool in the Industry is an indication of availability of support, quality documentation, technical forums and availability of trained personnel.
- ☐ Cost
- ☐ Opensource or Proprietary have different characteristics of cost, features and the support and needs to be balanced

- **Risk**: Probability of an unwanted incident during or towards testing
 - Risks in strategy could be in terms of :
 - Changes to the Business, Technology or competition directions
 - Resources
 - Quality of the software product being developed
 - The test models not being able to be used
 - Some type of testing chosen cannot be used
 - Test environment and its state
 - Automation or Tool issues
- Any risks found in any of them would need to be planned for addressing as part of the mitigation and contingency.

- Test planning is evolving a list of deliverables
- List of Activities and Deliverables will be identified which would need to be executed and delivered
- Could include
 - Test specifications for each of the modules of the product
 - Test cases for different conditions planned

Creation of detailed Test schedule

- Includes estimates for building test strategy/specification/test cases/test environment setup and test execution, test reporting, etc.
- Would include Work Breakdown Schedule and estimation using methods mentioned in project planning
- Calendarize the Work Breakdown estimates



Planning, identification and allocation of resources

- The current step and the previous step of estimation are performed together or iteratively
 - This ensures the schedule factors in characteristics of the planned resources

- Initially involves
 - Identifying the number and type of servers, storage, test tools and network resources
 - Identifying the number and type of people to work on the project
 - Test Manager, Testers, Test Developers, Test Administrators, and SQA
 - Identifying the test environment

- Schedule is reworked after resource and skill identification

Identification of milestones and risks

- Considering the expected deliverables, schedule and commitments, project milestones are identified
 - Milestones track or monitor progress and control overruns
 - Used to identify any risk triggers

- Risks for completion of a task from schedule and quality perspective is identified and analyzed
 - Mitigation plans and triggers for kick-off are identified

Identify measures and metrics

- Measurements are identified
 - Examples: Number of Test Cases planned and Created, Number of test cases run, amount of time spent on creation, execution, number of errors found

- The errors are classified into critical, serious, medium or low impact

- Metrics
 - Examples: Number of test cases executed/day, % of test cases executed, number of issues/KLoC, number of critical issues/KLoC

Security Validation Planning

Introduction to Security Validation Planning

- Ensure that security controls and objectives are **measurable**, **verifiable**, and tested throughout the SDLC.
- Defines *how* and *when* security features and controls will be tested or validated.
- **Why it's important in Requirements Phase:**
 - Helps ensure **testability** of security requirements.
 - Aligns **development**, **QA**, and **security teams** early.
 - Reduces last-minute changes or surprises during testing.
- Results in a documented plan that is tied to both requirements and design artifacts.
 - For our class project, this will be part of the validation plan

Key Elements

- A good security validation plan includes:
 - **Security Requirements Traceability:** Which security requirement is being validated and how.
 - **Validation Objectives:** What needs to be confirmed (e.g., encryption enabled, PII masked).
 - **Validation Methods:** Review, penetration test, fuzzing, static analysis, etc – for which component/requirement.
 - **Ownership:** Who validates—QA, security team, or external auditor?
 - **Timing:** At which SDLC stages will validation occur?
 - **Tools & Environments:** Any tools, testbeds, or configurations needed.

Typical test plan contents

1. Introduction

- i. Scope
 - a) In Scope
 - b) Out of Scope
- ii. Quality Objective
- iii. Roles and Responsibilities

2. Test Methodology

- i. Overview
- ii. Test Levels
- iii. Bug Triage (Sorting of bugs based on criticality)
- iv. Suspension Criteria and Resumption Requirements
- v. Test Completeness

3. Test Deliverables

4. Resource & Environment Needs

- i. Testing Tools
- ii. Test Environment

- A sample software test plan is [here](#).



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the [MIT License](#).
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.